

BI-KAB Kryptografie a bezpečnost

1. Základní pojmy v kryptologii, substituční a transpoziční šifry

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Základní pojmy v kryptologii
- Monoalfabetické substituční šifry
- Polygrafické substituční šifry
- Polyalfabetické substituční šifry
- Transpoziční šifry

Základní pojmy v kryptologii (1)

- **Kryptologie** – tvorba a luštění šifer.
 - ▶ **Kryptografie** – věda o tvorbě šifer.
 - ▶ **Kryptoanalýza** – věda o luštění šifer.
- **Otevřený Text (OT)** – text, který je určen k utajení.
- **Šifrování** – proces převádějící otevřený text do **Šifrového Textu (ŠT)**.
- **Šifra** – metoda, která převádí text do utajené formy.
- **Dešifrování** – proces opačný k procesu šifrování, je založený na znalosti šifry.

Luštění - kryptoanalýza:

- **Identifikace** – jaký šifrovací systém byl použit.
- **Prolomení** – způsob šifrování zprávy, určení neměnných částí systému. Kolik šifrových zpráv je potřebných k prolomení.
- **Nastavení** – určení, jak se mění proměnlivé části kryptosystému.

Základní pojmy v kryptologii (2)

Šifra – utajení obsahu zprávy před nepovolanou osobou.

Kód – **neutajuje se zpráva**, ale upravuje tak, aby ji bylo možné přenést přes nějaký kanál!

- Například kódy pro detekci chyb – paritní, nebo
- samoopravné kódy – Hammingovy kódy.

Šifrovací systémy vytváří šifrovou zprávu z OT pomocí nějakého pravidla – **šifrovacího algoritmu**.

Do nástupu počítačů dominovaly 3 základní metody:

- 1 substituční,
- 2 transpoziční,
- 3 metoda kódové knihy.

Šifrovací systémy byly založeny na jedné z nich nebo na nějaké kombinaci 2 nebo 3 těchto metod.

Šifry využívající substituci

- **Jednoduchá substituce** – jednu a tutéž záměnu (zamíchání) pro celý OT $\Rightarrow$, Caesarova šifra ($A \rightarrow D, B \rightarrow E, \dots W \rightarrow Z, X \rightarrow A, Y \rightarrow B, Z \rightarrow C$). Pokud $A = 0, B = 1, \dots, Z = 25$, p označuje písmeno OT a c označuje písmeno ŠT, pak $c = |p + 3|_{26}$, kde $0 \leq c \leq 25$.
- **Polyalfabetická substituce** – pro každé písmeno OT můžeme abecedu nově zamíchat. Například k písmenům OT popořadě přičítáme (modulo 26) nějaká jiná čísla – písmena nahrazena čísly – **klíč**. Periodicky opakující se heslo $\Rightarrow$ Vigenèrův systém.

Šifry transpoziční – šifrování spočívá v zamíchání písmen OT – nemění je za jiné. Nejjednodušší příklad jsou přesmyčky nebo lištovky.

Substituční a transpoziční šifry jsou **symetrické**, protože používají stejný klíč pro šifrování a dešifrování.

Základní pojmy v kryptologii (4)

Šifrování pomocí kódové knihy

- Slovník s běžnými frázemi tvořenými jedním nebo více písmeny, čísly, nebo slovy, typicky nahrazované čtveřicemi nebo peticemi písmen nebo čísel – **kódové skupiny**.
- K často používaným výrazům nebo písmenům může kódová kniha obsahovat několik kódových skupin. Umožní odesílateli výběr a ztíží identifikaci často používaných frází ("pondělí-2476 nebo 7032 nebo 6845).
- Překlad zprávy do jiného jazyka lze považovat za šifrování pomocí kódové knihy – slovníku. Význam pojmu kódová kniha je v tomto případě rozšířen do krajnosti. Použití málo známého jazyka k předávání zpráv krátkodobého významu (II. světová válka, americká armáda v pacifiku – jazyk Navajů).
- Osobní těsnopis (středověk) k psaní osobních deníků. Pravidelný výskyt symbolů (názvy dnů atd.) poskytují dostatečný klíč k rozluštění.

Posuzování spolehlivosti šifrových systémů

Při posuzování navrhovaného šifrovacího systému je důležité posoudit jeho sílu – odolnost vůči všem známým útokům za předpokladu *znalosti typu tohoto šifrovacího systému*.

Odolnost šifrovacího systému může být posuzována ve 5 různých situacích. Kryptoanalytik provádí luštění se znalostí:

- 1 ŠT (**Ciphertext-only attack**) (systém rozluštěný v této situaci za rozumnou dobu je nepoužitelný).
- 2 ŠT a jeden nebo víc OT (**Known-plaintext attack**).
- 3 ŠT a odpovídající vybraný OT (**Chosen-plaintext attack**).
- 4 ŠT, který je vybrán na základě určitého významu a k tomu dešifrovaný odpovídající OT (**Chosen-ciphertext attack**).
- 5 sjednocení metody 3. a 4. (**Chosen-text attack**).

Základní pojmy v kryptologii (6)

Existují jiné metody skrývání smyslu nebo obsahu zprávy, které nejsou založené na šifrách. Za zmínku stojí **steganografie**, která je založená na principu vkládání zprávy do běžných a nepodezřelých objektů – textových zpráv, programů, obrázků atd.

V dalším textu budeme také používat následující pojmy:

- **Monogram** – jedno písmeno v jakékoliv abecedě.
- **Bigram** – jakákoliv dvojice sousedních písmen v textu.
- **Trigram** – trojice po sobě následujících písmen.
- **Polygram** – nespecifikovaný počet písmen po sobě jdoucích v textu.
- **Symbol** – jakékoliv písmeno, číslice, interpunkční znaménko atd., které se mohlo v textu vyskytnout.
- **Řetězec** – jakákoliv posloupnost po sobě jdoucích symbolů.

Monoalfabetické substituční šifry (1)

Jednoduchá substituční šifra

Šifra použitá Juliem Caesarem je vyjádřena vztahem: $c = |p + 3|_{26}$, kde $0 \leq c \leq 25$. Dále budeme používat označení p pro písmeno OT a c pro písmeno ŠT. Za standard si vezmeme písmena anglické abecedy a přidělíme jim celá čísla od 0 do 25, viz Tabulka

Tabulka: Numerický ekvivalent písmenům anglické abecedy

písmeno	A	B	C	D	E	F	G	H	I	J	K	L	M
num. ekv.	0	1	2	3	4	5	6	7	8	9	10	11	12
písmeno	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
num.ekv.	13	14	15	16	17	18	19	20	21	22	23	24	25

Příklad: Otevřený text seskupíme do bloků po pěti písmenech. Toto seskupení písmen do bloků je jakousi jednoduchou prevencí proti provedení kryptoanalýzy založené na rozpoznávání jednotlivých slov.

Monoalfabetické substituční šifry (2)

Takže zprávu

THIS MESSAGE IS TOP SECRET

převedeme do bloků po pěti písmenech

THISM ESSAG EISTO PSECR ET.

Převodem písmen do jejich numerických ekvivalentů dostaneme

19 7 8 18 12 4 18 18 0 6 4 8 18 19 14 15 18 4 2 17 4 19.

S použitím Caesarovy transformace $c = |p + 3|_{26}$ dostáváme

22 10 11 21 15 7 21 21 3 9 7 11 21 22 17 18 21 7 5 20 7 22

a převodem zpět na písmena obdržíme šifrový text

WKLVP HVVDJ HLVWR SVHFU HW.

Monoalfabetické substituční šifry (3)

Příjemce zašifrované zprávy ji dešifruje následujícím způsobem:

- 1 převede písmena na jejich číselné ekvivalenty.
- 2 na základě vztahu $p = |c - 3|_{26}$, $0 \leq p \leq 25$ převede ŠT v číselné podobě na číselnou podobu OT.
- 3 převede zprávu do písemné podoby.

Dešifrovací postup si můžeme ukázat na dešifrování následující zprávy, která je šifrována Caesarovou šifrou:

WKLVL VKRZZ HGHFL SKHU.

Takže nejdříve převedeme zprávu do číselné podoby a získáme

22 10 11 21 11 21 10 17 25 25 7 6 7 5 11 18 10 7 20.

Dále provedeme transformaci $p = |c - 3|_{26}$, $0 \leq p \leq 25$, a tím získáme otevřený text v číselné podobě

19 7 8 18 8 18 7 14 22 22 4 3 4 2 8 15 7 4 17.

Monoalfabetické substituční šifry (4)

Převodem na písmena dostáváme otevřený text v blocích po pěti písmenech THISI SHOWW EDECI PHER.

Kombinací příslušných písmen vytvoříme slova a naše zpráva je
THIS IS HOW WE DECIPHER.

Jednoduché substituční šifry

Šifry s transformací posunem jsou definovány vztahem

$$c = |p + k|_{26}, \quad 0 \leq c \leq 25,$$

kde k je klíč reprezentující velikost posunutí písmen v abecedě.

Existuje 26 různých transformací tohoto typu šifry, zahrnující také $|k|_{26} = 0$, kde nedochází k žádnému posunu.

Ceasarova šifra je tedy speciálním případem jednoduché substituční šifry s transformací posunem pro $k=3$.

Obecně existuje $26!$ možných způsobů generování monoalfabetické substituční šifry s písmeny abecedy.

Monoalfabetické substituční šifry (5)

O něco málo obecnější šifrou je **šifra s transformací** typu

$$c = |ap + b|_{26}, \quad 0 \leq c \leq 25, \quad (1)$$

kde $a, b \in \mathbb{Z}$ a splňují podmínku $\gcd(a, 26) = 1$. Tato šifra je tzv. **afinní transformace**. Transformace posunem je afinní transformací pro $a = 1$.

Když $\gcd(a, 26) = 1$, pak existuje 12 konstant a ($\varphi(26) = 12$) a 26 b a tedy $12 \cdot 26 = 312$ transformací tohoto typu (zahrnující také $c = |p|_{26}$, pro $a = 1$ a $b = 0$).

Pokud je vztah OT a ŠT popsán vzájemným vztahem (1), potom je inverzní vztah daný

$$p = |a^{-1} \cdot (c - b)|_{26}, \quad 0 \leq p \leq 25. \quad (2)$$

Příklad: Pro $a = 7$ a $b = 10$ a tedy $c = |7p + 10|_{26} \Rightarrow$ pro dešifrování platí $p = |15(c - 10)|_{26} = |15c + 6|_{26}$, protože $15^{-1} = |7|_{26}$.

Monoalfabetické substituční šifry (6)

Pro ilustraci šifrujme OT: "PLEASE SEND MONEY",
který je převeden do ŠT: "LJMKG MGMXF QEXMW".

Obráceně ŠT: "FEXEN ZMBMK JNHMG MYZMN"
odpovídá OT: "DONOT REVEA LTHES ECRET"

a po seskupení slov máme "DO NOT REVEAL THE SECRET".

Tabulka: Vzájemný vztah písmen pro šifru: $c = |7p + 10|_{26}$

OT	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
ŠT	K	R	Y	F	M	T	A	H	O	V	C	J	Q
	10	17	24	5	12	19	0	7	14	21	2	9	16
OT	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	13	14	15	16	17	18	19	20	21	22	23	24	25
ŠT	X	E	L	S	Z	G	N	U	B	I	P	W	D
	23	4	11	18	25	6	13	20	1	8	15	22	3

Monoalfabetické substituční šifry (7)

Dále se budeme zabývat některými jednoduchými postupy **kryptoanalýzy šifer** založených na afinních transformacích.

Pokus prolomit nějakou znakovou šifru může začít porovnáním četnosti výskytu písmen v ŠT a OT. Takto získáme informaci o vztahu mezi písmeny ŠT a OT. Různá frekvence výskytu 26 písmen anglické abecedy v OT je uvedena v % v tabulce.

Tabulka: Četnost výskytu jednotlivých písmen v běžném anglickém textu.

písmeno četnost[%]	A	B	C	D	E	F	G	H	I	J	K	L	M
	7	1	3	4	13	3	2	3	8	<1	<1	4	3
písmeno četnost[%]	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	8	7	3	<1	8	6	9	3	1	1	<1	2	<1

Vidíme, že typický anglický text má největší výskyt písmen E, T, N, R, I, O a A. E má 13% a T, N, R, I, O a A mají výskyt v rozmezí 7% až 9%. Tuto informaci můžeme dobře využít pro určení koeficientů afinní šifry.

Monoalfabetické substituční šifry (8)

Mějme k šifrování šifru s posunem: $c = |p + k|_{26}$, $0 \leq c \leq 25$. Nechť ŠT určený ke kryptoanalýze na základě uvedených předpokladů je

YFXMP CESPZ CJTDF DPQFW QZCPY NTASP CTYRX PDDL R PD.

V tabulce jsou uvedeny hodnoty četností výskytu písmen ŠT.

písmeno	A	B	C	D	E	F	G	H	I	J	K	L	M
četnost	1	0	4	5	1	3	0	0	0	1	0	1	1
písmeno	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
četnost	1	0	7	2	2	2	3	0	0	1	2	3	2

Relativně největší výskyt písmen v ŠT je P, D, C, F, T a Y. Odhad: P v ŠT reprezentuje E v OT. Potom $|4 + k|_{26} = 15$ a z toho $k = 11$.

Po dosazení máme pro $c = |p + 11|_{26}$ a obráceně $p = |c - 11|_{26}$.

Použitím tohoto vztahu můžeme z uvedeného ŠT psát následující OT:

NUMBER THEORY IS USEFUL FOR ENCIPHERING MESSAGES.

Z toho je vidět, že naše předpoklady byly správné.

Monoalfabetické substituční šifry (9)

Dále předpokládejme použití afinní transformace podle vztahu (1), kterou byla šifrována následující zpráva

USLEL JUTCC YRTPS URKLT YGGFV
ELYUS LRYXD JURTU ULVCU URJRK
QLLQL YXSRV LBRYZ CYREK LVEXB
RYZDG HRGUS LJLLM LYPDJ LJ TJU
FALGU PTGVT JULYU SLDAL TJRWU
SLJFE OLPU

V tabulce jsou uvedeny četnosti výskytu jednotlivých písmen v ŠT.

písmeno	A	B	C	D	E	F	G	H	I	J	K	L	M
četnost	2	2	4	4	5	3	6	1	0	10	3	22	1
písmeno	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
četnost	0	1	4	2	12	7	8	16	5	1	3	10	2

Monoalfabetické substituční šifry (10)

Znovu předpokládejme, že písmeno L, které se nejvíce vyskytuje v ŠT je písmenem E v OT a písmeno U v ŠT, které má druhou největší četnost, odpovídá písmenu T v OT. Potom podle vztahu (1) dostáváme soustavu dvou kongruencí.

$$|4a + b|_{26} = 11$$

$$|19a + b|_{26} = 20$$

Řešení této soustavy je: $|a|_{26} = 11$ a $|b|_{26} = 19$. Pokud jsou to konstanty šifrovací transformace (1) $\Rightarrow$ dostáváme podle (2) (vzhledem k tomu, že $|11^{-1}|_{26} = 19$) pro dešifrování transformaci:

$$p = |19(c - 19)|_{26} = |19c - 361|_{26} = |19c + 3|_{26}, \quad 0 \leq p \leq 25.$$

Pomocí tohoto předpisu se pokusíme šifrový text dešifrovat. Jak je vidět, po prvních slovech zjistíme, že náš předpoklad a následný výpočet byl správný

THEBE STAPP ROACH TOLEA RNNUM ...

Polygrafické substituční šifry (1)

Motivace

- Afinní šifry zranitelné při použití kryptoanalýzy založené na frekvenční analýze zašifrovaného textu.
- K eliminaci těchto nevýhod byl vyvinut systém, který nahrazuje bloky určité délky otevřeného textu $\Rightarrow$ šifry polygrafické – blokové.
- Dále: polygrafické – blokové šifry založené na modulární aritmetice, které byly vyvinuté Hillem v roce 1930.

Polygrafické – blokové substituční šifry

Uvažujeme šifru s blokem o délce jednoho bigramu OT, který je převáděn do ŠT s dvoupísmenovými bloky. Na konec zprávy přidáváme písmeno X v případě, že zpráva končí blokem s jedním písmenem. Zpráva

THE GOLD IS BURIED IN ORONO

je seskupená jako

TH EG OL DI SB UR IE DI NO RO NO.

Polygrafické substituční šifry (2)

V dalším jsou tato písmena přeložena do jejich číselných ekvivalentů

19 7 4 6 14 11 3 8 18 1 20 17 8 4 3 8 13 14 17 14 13 14.

Každý blok dvou čísel p_1, p_2 otevřeného textu je převeden do bloku dvou čísel c_1, c_2 šifrovaného textu pomocí (3). c_1 jako nejmenšího nezáporného zbytku modulo 26 lineární kombinace p_1 a p_2 a c_2 jako nejmenšího nezáporného zbytku modulo 26 lineární kombinace p_1 a p_2 . Například:

$$\begin{aligned}c_1 &= |5p_1 + 17p_2|_{26}, & 0 \leq c_1 \leq 25 \\c_2 &= |4p_1 + 15p_2|_{26}, & 0 \leq c_2 \leq 25.\end{aligned}\tag{3}$$

První blok 19 7 je převeden do bloku 6 25, protože

$$\begin{aligned}c_1 &= |5 \cdot 19 + 17 \cdot 7|_{26} = 6, \\c_2 &= |4 \cdot 19 + 15 \cdot 7|_{26} = 25.\end{aligned}$$

Polygrafické substituční šifry (3)

Po aplikaci této transformace na všechny bloky zprávy otevřeného textu je šifrový text

6 25 18 2 23 13 21 2 3 9 25 23 4 14 21 2 17 2 11 18 17 2.

Pokud tyto bloky převedeme na písmena, máme šifrový text

GZ SC XN VC DJ ZX EO VC RC LS RC.

Dešifrovací předpis pro tento šifrovací systém získáme řešením soustavy kongruencí (3), tj. řešením soustavy dvou lineárních rovnic v modulární aritmetice s modulem 26 pro neznámé p_1 a p_2

$$\begin{aligned} p_1 &= |17c_1 + 5c_2|_{26}, & 0 \leq p_1 \leq 25, \\ p_2 &= |18c_1 + 23c_2|_{26}, & 0 \leq p_2 \leq 25, \end{aligned} \quad (4)$$

kde pro determinant Δ soustavy (3) platí $\gcd(\Delta, 26) = 1$.

Polygrafické substituční šifry (4)

V maticovém zápisu můžeme rovnici (3) vyjádřit následovně

$$\left| \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} 5 & 17 \\ 4 & 15 \end{pmatrix} \cdot \begin{pmatrix} p_1 \\ p_2 \end{pmatrix} \right|_{26} \quad (5)$$

a pro rovnici (4) je maticový zápis

$$\left| \begin{pmatrix} p_1 \\ p_2 \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} 17 & 5 \\ 18 & 23 \end{pmatrix} \cdot \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} \right|_{26} \quad (6)$$

Obecně, v Hillově šifrovacím systému, tj. blokové – polygrafické šifře, je ŠT získaný seskupením písmen OT do bloků o velikosti n , převodem písmen do číselných ekvivalentů a generováním ŠT podle vztahu

$$|\mathbf{c}|_{26} = |\mathbf{A}\mathbf{p}|_{26}, \quad (7)$$

kde $\mathbf{A}$ je matice dimenze $n \times n$ a platí $\gcd(\det \mathbf{A}, 26) = 1$, elementy $c_1, c_2, \dots, c_n$ vektoru $\mathbf{c}$ představují blok ŠT odpovídající bloku OT, který je reprezentován elementy $p_1, p_2, \dots, p_n$ vektoru $\mathbf{p}$ a nakonec je ŠT vyjádřen v číselných ekvivalentech převeden do písmen.

Polygrafické substituční šifry (5)

Pro dešifrování je použita inverzní matice $\mathbf{A}^{-1}$ k matici $\mathbf{A}$ modulo 26, kterou lze získat například GJ eliminačním procesem aplikovaným na matici $\mathbf{A}$ a současně matici jednotkovou $\mathbf{E}$ v modulární aritmetice s modulem 26. Můžeme psát za použití platnosti $|\mathbf{A}\mathbf{A}^{-1}|_{26} = |\mathbf{E}|_{26}$

$$|\mathbf{A}^{-1}\mathbf{c}|_{26} = |\mathbf{A}^{-1}(\mathbf{A}\mathbf{p})|_{26} = |(\mathbf{A}^{-1}\mathbf{A})\mathbf{p}|_{26} = |\mathbf{p}|_{26}.$$

Takže, pro získání otevřeného textu ze šifrovaného textu použijeme následující vztah

$$|\mathbf{p}|_{26} = |\mathbf{A}^{-1}\mathbf{c}|_{26}. \quad (8)$$

Ilustrujme si popsanou proceduru na příkladě pro $n = 3$ se šifrovací maticí

$$\mathbf{A} = \begin{pmatrix} 11 & 2 & 19 \\ 5 & 23 & 25 \\ 20 & 7 & 1 \end{pmatrix}.$$

Polygrafické substituční šifry (6)

Vzhledem k tomu, že determinant matice $\det \mathbf{A} = 5$ a platí $\gcd(5, 26) = 1$ existuje inverzní matice k matici $\mathbf{A}$ a můžeme pro šifrování s bloky o velikosti třech písmen psát

$$\left| \begin{pmatrix} c_1 \\ c_2 \\ c_2 \end{pmatrix} \right|_{26} = \mathbf{A} \left| \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix} \right|_{26}.$$

K zašifrování zprávy STOP PAYMENT nejdříve seskupíme zprávu do bloků po třech písmenech s přidáním fiktivního písmena X na konec posledního bloku (výplň — padding). Takto dostáváme OT v blocích

STO PPA YME NTX.

Převodem těchto písmen do jejich číselných ekvivalentů dostáváme:

18 19 14 15 15 0 24 12 4 13 19 23.

Polygrafické substituční šifry (7)

První blok šifrového textu vypočítáme následovně:

$$\left| \begin{pmatrix} c_1 \\ c_2 \\ c_2 \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} 11 & 2 & 19 \\ 5 & 23 & 25 \\ 20 & 7 & 1 \end{pmatrix} \begin{pmatrix} 18 \\ 19 \\ 14 \end{pmatrix} \right|_{26} = \begin{pmatrix} 8 \\ 19 \\ 13 \end{pmatrix}$$

Zašifrováním celé zprávy otevřeného textu dostáváme

8 19 13 13 4 15 0 2 22 20 11 0

a konečně šifrový text v písmenné podobě je

ITN NEP ACW ULA.

Pro dešifrovací proces použijeme transformaci

$$\left| \begin{pmatrix} p_1 \\ p_2 \\ p_2 \end{pmatrix} \right|_{26} = \mathbf{A}^{-1} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} \Big|_{26},$$

Polygrafické substituční šifry (8)

kde

$$\left| \mathbf{A}^{-1} \right|_{26} = \left| \begin{pmatrix} 6 & 21 & 11 \\ 21 & 25 & 16 \\ 19 & 3 & 7 \end{pmatrix} \right|_{26}.$$

- S blokovými šiframi je možné provádět kryptoanalýzu založenou na frekvenční analýze bloků n písmen (jednopísmenná frekvenční analýza je neúčinná).
- Pro blokovou šifru se dvěma písmeny v jednom bloku existuje 26^2 kombinací dvou písmen v bloku.
- Existují studie výskytu dvou písmen v textech různých jazyků. Bylo zjištěno, že nejčastěji se vyskytující dvojice písmen v anglickém textu je TH, a potom dvojice HE.

Pokud Hillův blokový šifrovací systém o délce bloků dvou písmen byl použit pro zašifrování zprávy, ve které se nejčastěji vyskytuje dvojice písmen KX a následně dvojice VZ, můžeme se domnívat, že dvojicím písmen KX a VZ v ŠT odpovídají dvojice písmen TH a HE v OT.

Polygrafické substituční šifry (9)

Bloky 19 7 a 7 4 mohou být v ŠT bloky 10 23 a 21 25. Pokud **A** je šifrovací matice, potom na základě těchto zjištění můžeme psát

$$\left| \mathbf{A} \begin{pmatrix} 19 & 7 \\ 7 & 4 \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} 10 & 21 \\ 23 & 25 \end{pmatrix} \right|_{26}.$$

Pokud platí

$$\left| \begin{pmatrix} 4 & 19 \\ 19 & 19 \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} 19 & 7 \\ 7 & 4 \end{pmatrix}^{-1} \right|_{26},$$

pak

$$\mathbf{A} = \left| \begin{pmatrix} 10 & 21 \\ 23 & 25 \end{pmatrix} \begin{pmatrix} 4 & 19 \\ 19 & 19 \end{pmatrix} \right|_{26} = \begin{pmatrix} 23 & 17 \\ 21 & 2 \end{pmatrix},$$

a

$$\left| \mathbf{A}^{-1} \right|_{26} = \left| \begin{pmatrix} 2 & 9 \\ 5 & 23 \end{pmatrix} \right|_{26}.$$

ve vztahu (8) dešifrujeme ŠT, a po této operaci vidíme, jestli náš předpoklad byl správný, tj. jestli je takto získaný OT smysluplný.

Polygrafické substituční šifry (10)

Zobecnění: Pokud víme, že bloky o velikosti n znaků ŠT $c_{1j}, c_{2j}, \dots, c_{nj}$, $j = 1, 2, \dots, n$, a bloky o velikosti n znaků OT $p_{1j}, p_{2j}, \dots, p_{nj}$, $j = 1, 2, \dots, n$ si OT vzájemně odpovídají $\Rightarrow$ obdržíme soustavu n lineárních kongruencí

$$\left| \mathbf{A} \begin{pmatrix} p_{11} & \dots & p_{1j} & \dots & p_{1n} \\ p_{21} & \dots & p_{2j} & \dots & p_{2n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ p_{n1} & \dots & p_{nj} & \dots & p_{nn} \end{pmatrix} \right|_{26} = \left| \begin{pmatrix} c_{11} & \dots & c_{1j} & \dots & c_{1n} \\ c_{21} & \dots & c_{2j} & \dots & c_{2n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ c_{n1} & \dots & c_{nj} & \dots & c_{nn} \end{pmatrix} \right|_{26}$$

a maticově

$$|\mathbf{AP}|_{26} = |\mathbf{C}|_{26},$$

kde $\mathbf{P}$ a $\mathbf{C}$ jsou matice dimenze $n \times n$ s elementy p_{ij} a c_{ij} . Pokud platí $\gcd(\det \mathbf{P}, 26) = 1$, $\Rightarrow$ pro $\mathbf{A}$ platí

$$|\mathbf{A}|_{26} = |\mathbf{CP}^{-1}|_{26},$$

kde $\mathbf{P}^{-1}$ je inverzní matice k matici $\mathbf{P}$ modulo 26.

Polygrafické substituční šifry (11)

Shrnutí:

- Kryptoanalýza blokových šifer s použitím výskytu četnosti jednotlivých kombinací písmen v bloku o velikosti n má smysl jen tehdy, když n je malé číslo.
- Například, pokud $n = 10$, existuje $26^{10} \doteq 1,4 \times 10^{14}$ kombinací písmen s touto délkou bloků.
- Pro tak velký počet různých kombinací deseti písmen v bloku je extrémně obtížné použít analýzu založenou na srovnání relativních četností těchto kombinací v šifrovaném textu s výsledky relativní četnosti získanými z obyčejného textu.
- Hillova šifra není odolná ani vůči kryptoanalýze 2. typu (known-plaintext), protože můžeme využít lineární závislosti ŠT na OT k získání matice **A**!
- Pokud zvolíme OT tak, aby matice **P** byla jednotková, pak dostáváme přímo klíč, tj. matici **C** = **A** (útok 3. typu, chosen-plaintext)!

Polyalfabetické substituční šifry (1)

- Monoalfabetické šifry jsou málo bezpečné, protože distribuce frekvence výskytu elementů ŠT je odrazem distribuce frekvence výskytu elementů OT.
- Polyalfabetické šifry řeší tento nedostatek.

Systém polyalfabetických šifer nad abecedou $\mathbb{Z}_N$ tvoří konečná případně nekonečná posloupnost monoalfabetických transformací $(T_1, T_2, \dots, T_n, \dots)$ nad abecedou $\mathbb{Z}_N$. Tyto posloupnosti tvoří prostor klíčů tohoto systému

$$K = \{T_1, T_2, \dots, T_n, \dots\}$$

Speciálním případem polyalfabetických šifer je **systém Vigenèrovských šifer**. Tyto šifry tvoří konečnou posloupnost transformací s posunem. Naříklad při základní abecedě mohou být posloupnosti klíčů:

$$K = \{k_1, k_2, \dots, k_n\}$$

kde $k_i \in \mathbb{Z}_N$, pro $i \in \langle 1, 2, \dots, n \rangle$.

Polyalfabetické substituční šifry (2)

Kryptografická transformace OT $(p_1, p_2, \dots)$ odvozená od klíče K je

$$c_j = |p_j + k_{|j|_n}|_N$$

Číslo n se nazývá periodou Vigenèrovské šifry, nebo také délkou klíče.

Příklad: Necht' $K = ('B', 'R', 'I', 'D', 'G', 'E') = (1, 17, 8, 3, 6, 4)$. OT je

THE LITERATURE OF CRYPTOGRAPHY HAS A CURIOUS HISTORY

ŠT tohoto OT v 5 písmenových blocích Vigenèrovy šifry s klíčem K je:

THELI	TERAT	UREOF	CRYPT	OGRAP	HYHAS	ACURI	...
BRIDG	EBRID	GEBRI	DGEBR	IDGEB	RIDGE	BRIDG	...
UYMOO	XFIIW	...					

Tento algoritmus snižuje použitím klíče K se 6 písmeny dostatečným způsobem vliv četnosti každého písmene OT na nerovnoměrnost distribuce proto, že má k dispozici 6 různých transformací. Vyhovující vyhlazení distribuce jsou schopná i klíče o 3 znacích. Konečný klíč je slabina, rovněž klíč, který je slovem přirozeného jazyka je slabina.

Transpoziční šifry (1)

Transpozice – permutace písmen ve zprávě

- Cílem substituce je **konfúze**, tj. ztížení určení způsobu transformace zprávy a klíče na ŠT.
- Transpozice je šifrování, kde dochází ke změně uspořádání písmen zprávy.
- Cílem transpozice je **difúze**, tj. rozptyl informace zprávy nebo klíče po celé šíři ŠT.
- Transpozice odstraňuje vzniklé systematické struktury.
- Transpozice je častokrát označována za permutaci, protože mění uspořádání symbolů zprávy.

Sloupcová transpozice

Sloupcová transpozice přerozděluje znaky OT do sloupců.

Transpoziční šifry (2)

Příklad: Mějme pětisloupcovou transpozici $\Rightarrow$ znaky OT se rozdělí do samostatných bloků po peticích a zapíší se po sobě:

$$\begin{array}{ccccc} p_1 & p_2 & p_3 & p_4 & p_5 \\ p_6 & p_7 & p_8 & p_9 & p_{10} \\ p_{11} & p_{12} & p_{13} & \dots & \\ \dots & & & & \end{array}$$

Výsledný ŠT je potom

$$p_1 p_6 p_{11} \dots p_2 p_7 p_{12} \dots p_3 p_8 p_{13} \dots$$

Mějme dále OT:

THIS IS THE MESSAGE TO SHOW HOW A COLUMNAR
TRANSPOSITION WORKS

Tento OT text můžeme zapsat pomocí sloupcové transpozice například do 5 sloupců takto:

Transpoziční šifry (3)

T	H	I	S	I
S	T	H	E	M
E	S	S	A	G
E	T	O	S	H
O	W	H	O	W
A	C	O	L	U
M	N	A	R	T
R	A	N	S	P
O	S	I	T	I
O	N	W	O	R
K	S	X	X	X...

Výsledný ŠT je potom

TSEEO AMROO KHTST WCNAS NSIHS OHOAN
IWXSE ASOLR STOXI MGHWU TPIRX

BI-KAB Kryptografie a bezpečnost

2. Exponenciální šifra, zřízení společného klíče a problém diskrétního logaritmu

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Exponenciální šifry
- Zřízení společného klíče
- Problém diskretního logaritmu

Exponenciální šifry (1)

Exponenciální šifry

- Založené na modulárním umocňování.
- Byly uvedeny v roce 1978 Pohligem a Hellmanem.
- Značně odolné vůči kryptoanalýze.

Postup šifrování pomocí exponenciální šifry:

- Necht' m je prvočíslo,
- přirozené číslo e je **klíč** a
- platí $\gcd(e, m - 1) = 1$.
- Pro zašifrování zprávy je nejdříve OT převeden do dvojciferného číselného ekvivalentu pomocí tabulky:

písmeno	A	B	C	D	E	F	G	H	I	J	K	L	M
num. ekv.	00	01	02	03	04	05	06	07	08	09	10	11	12
písmeno	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
num.ekv.	13	14	15	16	17	18	19	20	21	22	23	24	25

Exponenciální šifry (2)

- Seskupíme získaná čísla do dekadických číslic délky $2s$ tvořících číslo, kde s je počet písmen v jednom bloku.
- Tyto bloky tvoří čísla menší než m .
- Když $2525 < m < 252525$, potom $s = 2$.

Každý blok p s s písmeny OT, který reprezentuje celé číslo s $2s$ dekadickými číslicemi, převedeme na blok c ŠT vztahem

$$c = |p^e|_m, \quad 0 \leq c < m. \quad (1)$$

ŠT se skládá z takových bloků, že celá čísla, která je reprezentují, jsou menší než m . Hodnoty e určují rozdílné šifry, a proto e můžeme pojmenovat **šifrovacím klíčem**.

Příklad: Nechť $m = 2633$ (m je prvočíslo) a nechť šifrovací klíč $e = 29$, kde $\gcd(e, m - 1) = \gcd(29, 2632) = 1$.

Exponenciální šifry (3)

K zašifrování OT zprávy

THIS IS AN EXAMPLE OF AN EXPONENTIATION CIPHER,

kde její numerický ekvivalent se 4 číslicovými bloky (2 písmena) je

1907 0818 0818 0013 0423 0012 1511 0414 0500 1304
2315 1413 0413 1908 0019 0814 1302 0815 0704 1723.

Na konec zprávy bylo kvůli zarovnání do bloku přidáno písmeno X (hodnota 23). Každý blok p OT je převeden do bloků ŠT s použitím vztahu

$$c = |p^{29}|_{2633}, \quad 0 < c < 2633.$$

Například k zašifrování prvního bloku otevřeného textu počítáme

$$c = |1907^{29}|_{2633} = 2199.$$

Exponenciální šifry (4)

Pro efektivní modulární umocňování použijeme algoritmus "Square & Multiply". Zašifrováním všech bloků OT dostáváme ŠT:

2199 1745 1745 1209 2437 2425 1729 1619 0935 0960
1072 1541 1701 1553 0735 2064 1351 1704 1741 1459.

K dešifrování bloku c ŠT potřebujeme znát dešifrovací klíč, tj. celé číslo d , pro které platí $de \equiv 1 \pmod{m-1} \Rightarrow d$ je multiplikativní inverze e modulo $(m-1)$, která existuje, pokud $\gcd(e, m-1) = 1 \Rightarrow$ dešifrovací vztah pro náš blok p OT získáme:

$$|c^d|_m = |(p^e)^d|_m = |p^{ed}|_m = |p^{k(m-1)+1}|_m = |(p^{m-1})^k p|_m = |p|_m,$$

kde $de = k(m-1) + 1$, pro nějaké celé číslo k , protože $de \equiv 1 \pmod{m-1}$. Při odvození vztahu pro dešifrování bloku ŠT jsme použili malou Fermatovu větu: $p^{m-1} \equiv 1 \pmod{m}$.

Exponenciální šifry (5)

Příklad: K dešifrování bloku šifrovaného textu, který byl vygenerován s použitím modula $m = 2633$ a šifrovacího klíče $e = 29$, potřebujeme vypočítat inverzi e modulo $m - 1 = 2632$.

Podle rozšířeného Eukleidova algoritmu – **Extended Euclidean algorithm (EEA)** vypočítáme inverzi d modulo $m - 1$

$$|d|_{m-1} = |e^{-1}|_{m-1} = |29^{-1}|_{2632} = 2269.$$

Potom dešifrování bloku ŠT c je vyjádřeno výpočtem p podle vztahu

$$p = |c^{2269}|_{2633}.$$

Pro dešifrování bloku 2199 šifrovaného textu potom platí

$$p = |2199^{2269}|_{2633} = 1907.$$

Exponenciální šifry (6)

Časová složitost

- Číslo m můžeme vyjádřit pomocí $k = \lceil \log_2(m) \rceil$ bitů
- Pro každý blok p OT zašifrovaný pomocí $|p^e|_m$ potřebujeme $O(k^3)$ bitových operací
- Před dešifrováním potřebujeme nalézt inverzi d čísla e modulo $m - 1$. Výpočet proveden EEA potřebuje $O(\log 2^k)$ bitových operací. Tento výpočet je potřebné provést jen jednou.
- K převodu bloku ŠT na blok OT potřebujeme vypočítat $|c^d|_m$.
- To je možné vykonat pomocí algoritmu pro modulární umocňování se složitostí $O(k^3)$ bitových operací. Takto je celý proces šifrování a dešifrování proveden značně rychle.
- Na druhé straně kryptoanalýza zprávy šifrované s použitím exponenciální šifry obecně **nemůže být vykonána v přijatelné době** se standardními výpočetními prostředky.

Exponenciální šifry (7)

Předpokládejme, že je použit pro exponenciální šifru modul m , a navíc víme, jak vypadá blok OT p a blok ŠT c , a platí

$$c = |p^e|_m. \quad (2)$$

Pro úspěšné provedení kryptoanalýzy potřebujeme najít šifrovací klíč e . Když vztah (2) platí, říkáme, že e je **logaritmus c o bázi p modulo m** .

- Nalezení logaritmu o dané bázi modulo nějaké prvočíslo představuje **problém diskrétního logaritmu**.
- Nejrychlejší algoritmy potřebují přibližně $\exp(\sqrt{\log m \log \log m})$ bitových operací.
- K nalezení logaritmu modulo nějaké prvočíslo při použití nejrychlejších známých algoritmů pro výpočet diskrétního logaritmu potřebujeme přibližně stejný počet bitových operací jako pro faktorizaci tohoto čísla s použitím nejrychlejších algoritmů pro faktorizaci.

Exponenciální šifry (8)

Příklad: Pro m se 100 dekadickými číslicemi nalezení logaritmu modulo m vyžaduje přibližně desítky let a pro m s 200 dekadickými číslicemi to je přibližně 10^8 let.

- Délka klíče je jedním z parametrů, které určují bezpečnost šifry $\Rightarrow$
- Jeden z útoků na prolomení šifry je zjistit hodnotu klíče.
- Útok prováděný hrubou silou (brute-force attack) je vyzkoušení všech 2^n možných kombinací hodnot klíče (n je # bitů klíče).
V průměru to je ale jen $2^{(n-1)}$ kombinací.

V orientační tabulce jsou uvedeny délky klíče n v bitech a k nim odpovídající přibližný čas prolomení pro různé kategorie luštitelů.

	hacker	firma	taj. služba	lidé	???
# μ PC	1	10^3	10^5	10^8	10^{51}
# testů/s	10^4 (2020?)	10^6	10^9	10^{12}	10^{19}
čas [s]	1 W [10^6]	1 M [10^7]	1 Y [10^8]	100 Y [10^{10}]	1000 Y [10^{11}]
# operací	10^{10}	10^{16}	10^{22}	10^{30}	10^{81}
n	34	54	73	100	269

Zřízení společného klíče (1)

- Délka klíče je přímo úměrná kvalitě šifry.
- Když m je prvočíslo a $m - 1$ je součinem malých prvočísel (tj. slabá instance) $\Rightarrow$ je možné použít speciálních metod pro nalezení logaritmu modulo m s menším počtem binárních operací než $O(\log_2^2 m) \Rightarrow$
- je vhodné jako modula pro exponenciální šifrovací systémy používat čísla $m = 2q + 1$, kde q je prvočíslo.

Zřízení společného klíče

Exponenciální šifra je vhodná pro zřízení **společného klíče** pro dva nebo více subjektů. Společný klíč může být například použit jako šifrovací klíč pro šifrovací systém nějaké datové komunikace a je sestaven tak, že neautorizovaný subjekt ho nemůže rozluštit v nějakém rozumném počítačovém čase.

Nechť m je velké prvočíslo a nechť a je generátorem grupy $\mathbb{Z}_m^*$. Každý subjekt v síti si náhodně zvolí celé číslo k_i jako klíč tak, že $\gcd(k_i, m - 1) = 1$.

Zřízení společného klíče (2)

Když si dva subjekty s klíči k_1 a k_2 chtějí vygenerovat klíč, tak nejdříve první subjekt pošle druhému subjektu celé číslo y_1 takové, že platí

$$y_1 = |a^{k_1}|_m, \quad 0 < y_1 < m$$

a druhý subjekt vypočítá společný klíč K na základě vztahu

$$K = |y_1^{k_2}|_m = |a^{k_1 k_2}|_m, \quad 0 < K < m.$$

Podobně, druhý subjekt vyšle prvnímu subjektu celé číslo y_2 , kde

$$y_2 = |a^{k_2}|_m, \quad 0 < y_2 < m$$

a první subjekt si vypočítá společný klíč K pomocí vztahu

$$K = |y_2^{k_1}|_m = |a^{k_1 k_2}|_m, \quad 0 < K < m.$$

Neautorizované subjekty nacházející se v komunikační síti nemohou najít společný klíč K v rozumném čase, protože jsou nuceni hledat logaritmus modulo m pro nalezení K .

Zřízení společného klíče (3)

Příklad: Mějme subjekt A, B a $m = 13$, $a = 2$, $k_1 = 7$, $k_2 = 11 \Rightarrow$

- ① A pošle B $y_1 = |a^{k_1}|_m = |2^7|_{13} = 11$
- ② B pošle A $y_2 = |a^{k_2}|_m = |2^{11}|_{13} = 7$
- ③ A vypočítá společný klíč $K = |y_2^{k_1}|_m = |7^7|_{13} = 6$
- ④ B vypočítá společný klíč $K = |y_1^{k_2}|_m = |11^{11}|_{13} = 6$

Podobným způsobem je možné společný klíč K sdílet i s více než dvěma subjekty. V případě n subjektů má každý z těchto subjektů vlastní klíč $k_1, k_2, \dots, k_n$. Potom pro společný klíč K platí

$$K = |a^{k_1 k_2 \dots k_n}|_m, \quad 0 < K < m.$$

Úloha: Navrhněte schéma pro zřízení společného klíče K pro 3 subjekty A, B a C.

Zřízení společného klíče (4)

Předchozí schéma zřízení společného klíče je v podstatě

Diffie-Hellmanův kryptografický algoritmus veřejného klíče

- 1 Volba veřejných prvků účastníkem A: prvočíslo m a celé číslo $0 < a < m$ (a je generátorem Z_m^*).
- 2 Generování parametrů klíče účastníkem A: volba čísla $k_1 < m - 1$ a výpočet $y_1 = |a^{k_1}|_m$,
A odešle prostřednictvím komunikačního kanálu B čísla a , m a y_1 .
- 3 Generování parametrů klíče účastníkem B: volba čísla $k_2 < m - 1$ a výpočet $y_2 = |a^{k_2}|_m$,
B odešle prostřednictvím komunikačního kanálu A číslo y_2 .
- 4 Dopočítání sdíleného klíče účastníkem A: $K = |y_2^{k_1}|_m$.
- 5 Dopočítání sdíleného klíče účastníkem B: $K = |y_1^{k_2}|_m$.

Veřejné prvky jsou v tomto případě čísla m a a .

Motivace

- Při práci s reálnými čísly není výpočet mocniny výrazně jednodušší, než výpočet logaritmu.
- Jiná situace nastává v případě, když pracujeme v konečné grupě, např. v $\mathbb{Z}_n^*$ (s operací násobení modulo n), která obsahuje prvky nesoudělné s n .
- S použitím metody „Square & Multiply“ je výpočet mocniny v konečné grupě rychlý.
- Ale když máme prvek $m \in \mathbb{Z}_n^*$, o kterém víme, že je ve tvaru b^k , kde $b \in \mathbb{Z}_n^*$ je známý prvek, tak neznáme rychlý algoritmus na nalezení hodnoty $k = \log_b m$.

Definice

- Necht' G je grupa, $b \in G$ a $y \in G$ je mocninou prvku b . Potom diskrétní logaritmus prvku y o základu b je každé číslo k takové, že $b^k = y$, značíme ho $\log_b y$.
- Problém nalezení k se nazývá **problém diskrétního logaritmu**, dále PDL.
- Pro libovolné prvky $g, h \in G$ nemusí vždy existovat číslo k takové, že platí $g^k = h$. Např. v grupě $\mathbb{Z}_{11}^*$ neexistuje číslo k takové, že $5^k = 7$. Tj. v grupě $\mathbb{Z}_{11}^*$ neexistuje $\log_5 7$.
- Avšak když grupa G bude cyklická a prvek $b \in G$ bude jejím generátorem, tak existenci prvku $\log_b m$ máme zaručenou pro všechna $m \in G$. Např. grupa $\mathbb{Z}_{11}^*$ je cyklická a její generátor je 2. Proto v ní existuje $\log_2 m$ pro všechna $m \in \mathbb{Z}_{11}^*$.

Vlastnosti

- Souvislost diskrétního logaritmu s „běžným“ logaritmem (v $\mathbb{R}$) je přímočará.
- Mějme cyklickou grupu G řádu $n \in \mathbb{N}$ a necht' g je její generátor. Potom platí:
 - 1) $\log_g 1 = 0$, $\log_g g^k = k$ pro $k \in \mathbb{N}$,
 - 2) $\log_g a^k = k \cdot \log_g a$ pro všechny $a \in G$, $k \in \mathbb{N}$,
 - 3) $\log_g(a \cdot b) = \log_g a + \log_g b$ pro všechny $a, b \in G$
- Důkaz bodu 1) plyne přímo z definice.
- Důkaz bodu 2): $\log_g a^k = \log_g (g^l)^k = \log_g g^{lk} = lk = k \cdot \log_g a$.
- Důkaz bodu 3): $\log_g(a \cdot b) = \log_g(g^k \cdot g^l) = \log_g g^{k+l} = k + l = \log_g g^k + \log_g g^l = \log_g a + \log_g b$.

Příklady

1. Uvažujme cyklickou grupu $\mathbb{Z}_{17}^*$ s operací násobení modulo 17 a její generátor $g = 3$. Diskrétním logaritmem čísla 7 je potom hodnota $\log_3 7 = 11$, protože $3^{11} \equiv 7 \pmod{17}$.
 2. Dále mějme cyklickou grupu $\mathbb{Z}_{17}$ s operací sčítání modulo 17. Generátorem této grupy je např. $g = 3$. Diskrétním logaritmem čísla 7 je potom hodnota $\log_3 7 = 8$, protože $3 \cdot 8 \equiv 7 \pmod{17}$.
- Všimněme si, že výpočetní složitost diskretního logaritmu závisí na konkrétní grupě.
 - V 1. případě $\mathbb{Z}_p^*$ neexistuje známý algoritmus na řešení PDL, který by měl polynomiální složitost v nejhorším případě.
 - V 2. případě $\mathbb{Z}_p$ můžeme PDL vyřešit lehce pomocí Euklidova algoritmu v polynomiálním čase.

Útok hrubou silou

- Uvažujme cyklickou grupu G s generátorem $g \in G$ a počtem prvků n . Chceme pro daný prvek $y \in G$ spočítat $\log_g y$.
- Nejjednodušším, ale neefektivním postupem výpočtu diskretního logaritmu je postupný výpočet hodnot $g, g^2, g^3, \dots$ až do momentu, kdy najdeme k , pro které bude platit $g^k = y$. Hodnota k potom bude hledaným diskretním logaritmem $\log_g y = k$.
- Je zřejmé, že v nejhorším případě třeba vykonat n operací a porovnání a uvedený algoritmus má exponenciální složitost vzhledem k délce vstupu $d = \log_2 n$, tj. složitost tohoto útoku je $\mathcal{O}(2^d) = \mathcal{O}(n)$.
- Mezi efektivnější, ale stále exponenciální algoritmy patří např. Baby step - giant step, Pollardův ρ algoritmus, Pohligův-Hellmanův algoritmus, Index kalkulus nebo Funkční síto, které je momentálně nejrychlejším dodnes známým algoritmem.

Využití v kryptografii

- Diskrétní logaritmus je díky své vysoké časové náročnosti využíváný v mnoha kryptografických transformacích, hlavně v asymetrické kryptografii.
- Známým příkladem je Diffie-Hellmanův protokol výměny klíčů. Využití diskrétního logaritmu spočívá v tom, že i kdyby útočník znal hodnoty g^a a g^b , tak nemůže rychle spočítat g^{ab} bez znalosti a nebo b .
- Diskrétní logaritmus se dále využívá u známých kryptosystémů jako jsou ECDSA anebo ElGamal.
- Bezpečnost uvedených algoritmů do velké míry závisí na zvolené grupě. Běžnou volbou bývají cyklické grupy $\mathbb{Z}_p^*$ pro dostatečně velká prvočísla.
- Např. u Diffie-Hellmannova algoritmu lze použít grupy $\mathbb{Z}_p^*$, kde prvočíslo $p \approx 2^{4096}$.

BI-KAB Kryptografie a bezpečnost

3. Rozdělení šifer, proudové šifry, RC4, Salsa20, ChaCha, A5/1

prof. Ing. Róbert Lórencz, CSc.

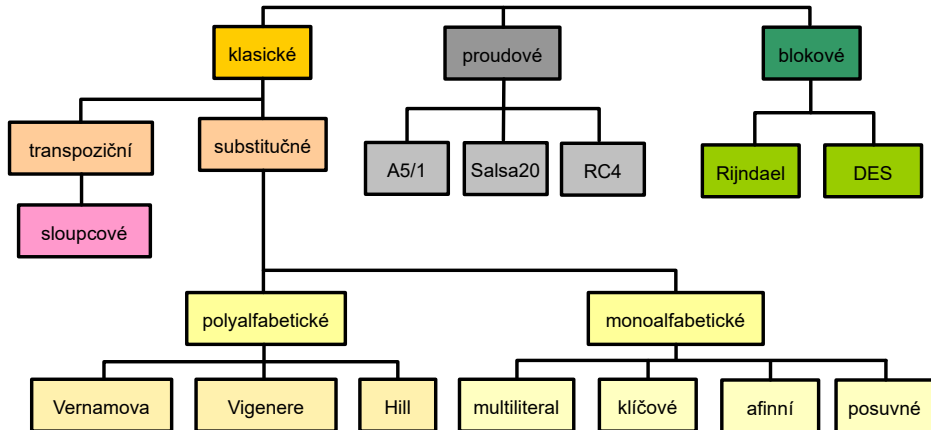


**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Rozdělení šifer
- Proudové šifry, RC4
- Salsa20
- Šifra A5/1

Rozdělení symetrických šifer



Proudové šifry (1)

Proudové šifry

- Z hlediska použití klíče ke zpracování OT rozeznáváme dva základní druhy symetrických šifer - **proudové a blokové**.
- Necht' OT používá vstupní abecedu A o q symbolech. Proudová šifra šifruje zvlášť jednotlivé znaky abecedy, zatímco bloková šifra zpracovává najednou bloky (řetězce) délky t znaků (to ale není hlavní rozdíl).
- Podstatné na blokových šifrách však je, že všechny bloky jsou šifrovány (dešifrovány) stejnou transformací $E_k(D_k)$, kde k je šifrovací klíč.
- Ale proudové šifry nejprve z klíče k vygenerují posloupnost $h_1, h_2, \dots$ a každý znak otevřeného textu šifrují jinou transformací E_{h_i} .
- Proudové šifry by mohly být chápány i jako blokové šifry s blokem délky $t = 1$, **ale** u proudových šifer je každý tento "blok" zpracováván jiným způsobem, jinou substitucí.

Definice symetrické proudové šifry

- Nechť A je abeceda q symbolů, nechť $M = C$ je množina všech konečných řetězců nad A a nechť K je množina klíčů.
- Proudová šifra se skládá z transformace (generátoru) G , zobrazení E a zobrazení D .
- Pro každý klíč $k \in K$ generátor G vytváří posloupnost hesla $h_1, h_2, \dots$ přičemž prvky h_i reprezentují libovolné substituce $E_{h_1}, E_{h_2}, \dots$ nad abecedou A .
- Zobrazení E a D každému klíči $k \in K$ přiřazují transformace zašifrování E_k a odšifrování D_k .
- Zašifrování OT $m = m_1, m_2, \dots$ probíhá podle vztahu $c_1 = E_{h_1}(m_1), c_2 = E_{h_2}(m_2), \dots$
- Dešifrování ŠT $c = c_1, c_2, \dots$ probíhá podle vztahu $m_1 = D_{h_1}(c_1), m_2 = D_{h_2}(c_2), \dots$, kde $D_{h_i} = E_{h_i}^{-1}$.

Proudové šifry (3)

- Z historických důvodů nazýváme G generátor hesla, neboť $h_1, h_2, \dots$ bývá proud znaků abecedy A a substituce E_{h_i} posunem v abecedě A o h_i pozic, tj. $c_i = |m_i + h_i|_q$.
- Proudové šifry jsou příkladem historických tzv. heslových systémů. V anglické literatuře se heslo $h_1, h_2, \dots$ nazývá running-key nebo key-stream (keystream), tj. proud klíče, i když se jedná o derivát originálního klíče k .
- Pokud se proud hesla začne od určité pozice opakovat, říkáme, že jde o periodické heslo a periodickou šifru (Vigenèrova šifra).
- Moderní proudové šifry pracují nad abecedou $A = \{0, 1\}$, tj. $q = 2$. Sčítání/odčítání modulo 2 je binárním sčítáním/odčítáním. Platí $|a + b|_2 = |a - b|_2$ a vyjadřuje diferenci bitů. Označuje se zkratkou **xor** nebo operátorem $\oplus$.
- Jedinou smysluplnou substitucí E_{h_i} nad bitem abecedy m_i je transformace $E_{h_i}(m_i) = m_i + h_i$ nebo $E_{h_i}(m_i) = m_i + h_i + 1$.

Proudové šifry (4)

- U moderních proudových šifer ŠT vzniká tak, že jednotlivé bity proudu hesla jsou postupně sloučeny s jednotlivými bity proudu OT binárním sčítáním.
- Vzhledem k rovnosti binárního sčítání a odčítání je transformace pro zašifrování a odšifrování také stejná.
- Jako u všech symetrických šifer, odesílatel i příjemce musí mít k dispozici tentýž klíč (heslo).
- Heslo může být vytvořeno zcela náhodně jako u Vernamovy šifry nebo může být vygenerováno deterministicky nějakým šifrovacím algoritmem na základě šifrovacího klíče.

Vernamova šifra

- Vernamova šifra používala náhodné heslo stejně dlouhé jako OT $\Rightarrow$ se heslo ničilo (nikdy nebylo použito k šifrování 2 různých OT).
- Na OT (5b na písmeno v 32znakovém Baudotově kódu) se bit po bitu binárně načítá náhodná posloupnost bitů klíče (děrná páska).

Proudové šifry (5)

Vernamova šifra má vlastnost **absolutní bezpečnosti**, tj. dokonalého utajení. ŠT nenese žádnou informaci o OT – **definice absolutně bezpečné šifry**.

Algoritmické proudové šifry

- Heslo se “vypočítá” na základě tajného klíče (distribuuje se).
- Aby klíč nemusel být měněn příliš často $\Rightarrow$ princip náhodně se měnícího inicializačního vektoru (IV).
- IV je pro každou zprávu vybírán náhodně a je přenášen před ŠT v otevřené podobě.
- IV (za účasti tajného klíče nebo bez něj) nastavuje příslušný algoritmus (konečný automat, šifrátor) vždy do jiného (náhodného) počátečního stavu $\Rightarrow$ i při stejném tajném klíči je generována pokaždé jiná heslová posloupnost.
- Za různost hesla zodpovídá IV, za utajenost zodpovídá tajný šifrovací klíč. (podobný princip se využívá i u blokových šifer).

Proudové šifry (6)

Proudová šifra RC4

- Jedna z nejpoužívanějších šifer na internetu (Rivest – 1987). Dnes je považována za prolomenou.
- Nevyužívá IV $\Rightarrow$ na každé spojení generuje náhodně nový tajný klíč (pomocí asymetrické metody).
- Její popis nebyl oficiálně publikován, i když je znám.
- RC4 zveřejněna neznámým hackerem v roce 1994 (získán disassemblováním z programu BSAFE společnosti RSA).
- Šifra RC4 = “Arcfour” (z důvodu ochrany autorských práv)
- RC4 používá mnoho protokolů a standardů (S/MIME a SSL).
- RC4 umožňuje volit délku klíče, 40b a 128b jsou nejpoužívanější.
- Šifrovací klíč se používá pouze k vygenerování tajné substituce $\{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$, tedy substituci bajtu za bajt.
- Pomocí tabulky S se pak konečným automatem generují jednotlivé bajty hesla $h_0, h_1, \dots$, které se xorují na OT nebo ŠT.

Proudové šifry (7)

Princip generování náhodné permutace

- 1 Naplníme identickou permutací: $P_i = i$ pro $i = 0, 1, 2, \dots, 255$.
- 2 Pomocí posloupnosti r promícháme permutaci P .
- 3 Míchání provádíme postupně tak, že v každém kroku i ($i = 0, 1, 2, \dots, 255$) v permutaci P vyměníme hodnoty na pozicích i a r_i , tj. hodnoty P_i a P_{r_i} vzájemně vyměníme.

$P(0)=0, P(1)=1, \dots, P(255)=255$

```
for i = 0 to 255
```

```
{
```

```
    vyměň mezi sebou hodnoty  $P(i)$  a  $P(r(i))$ 
```

```
}
```

- P zůstává stále permutací.
- Výměna postihne každou její pozici.
- Výsledek je nová permutace závislá na **posloupnosti r** .

Proudová šifra RC4 — inicializace permutace S

- 1 Naplníme identickou permutací: $S_i = i$ pro $i = 0, 1, 2, \dots, 255$.
- 2 Pomocí posloupnosti bajtů klíče k (délky n) promícháme permutaci S .
- 3 Míchání provádíme postupně tak, že v každém kroku i ($i = 0, 1, 2, \dots, 255$) v permutaci S vyměníme hodnoty na pozicích podle následujícího předpisu.

$S(0)=0, S(1)=1, \dots, S(255)=255$

$j = 0$

for $i = 0$ to 255

{

$j = |j + S(i) + k(|i|_n)|_{256}$

vyměň mezi sebou hodnoty $S(i)$ a $S(j)$

}

Proudová šifra RC4 — princip tvorby hesla RC4

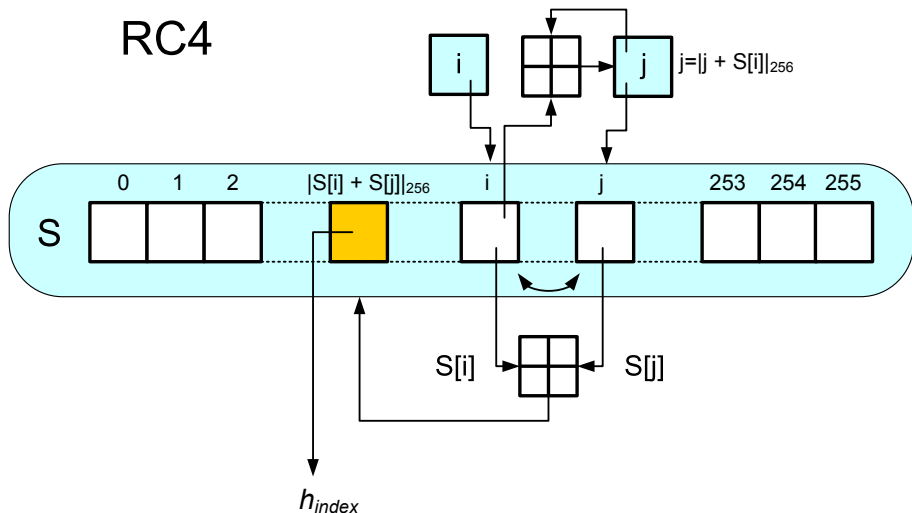
- Počítání s bajty $\Rightarrow$ redukce modulo 256.
- i se systematicky zvyšuje modulo 256.
- j je náhodný na klíči závislý index.
- Hodnota h_{index} obsahuje heslovou posloupnost generovanou tímto algoritmem.

```
i = j = 0
for index = 0 to n-1
{
    i = |i + 1|256
    j = |j + S(i)|256
    vyměň mezi sebou hodnoty S(i) a S(j)
    h(index) = S(|S(i) + S(j)|256)
}
```

Proudové šifry (10)

Proudová šifra RC4 — princip generování náhodné posloupnosti

RC4



Dvojití použití hesla u aditivních šifer

- U proudových šifer, kde každá parciální substituce je posunem otevřeného textu v abecedě A o h_i , postačí k luštění 2 ŠT (c, c'), šifrované tímtež heslem. Obdobně to platí i pro xor a další invertibilní operace.
- Mějme $c_i = |m_i + h_i|_q$ a $c'_i = |m'_i + h_i|_q$.
- Odečtením ŠT od sebe eliminujeme heslo a dostáváme $|c_i - c'_i|_q = |m_i - m'_i|_q$
- Z posloupnosti $|m_i - m'_i|_q$ pro $i = 0, 1, \dots$ pak původní texty m a m' luštíme jako při použití knižní šifry, kde v roli původního otevřeného textu vystupuje m a v roli knižního hesla m' .
- Někdy se používá metoda předpokládaného slova, kdy za m' zkusíme nějaké běžné slovo (např. v angličtině the) postupně od první do poslední pozice, odpočítáme m a sledujeme, zda dává smysl.

Použití

- Linkové šifrátory – do komunikačního kanálu přicházejí jednotlivé znaky v pravidelných nebo nepravidelných časových intervalech
⇒ v daném okamžiku je nutné tento znak okamžitě přenést.
- Příklad tzv. terminálového spojení, kdy jsou spojeny dva počítače, přičemž to, co uživatel píše na klávesnici na jedné straně, se objevuje na monitoru počítače druhého uživatele.
- Šifrovací zařízení má omezenou paměť na průchozí data.
- Výhodou proudových šifer oproti blokovým je malá “propagace chyby”. Vzniklá chyba na komunikačním kanálu v jednom znaku ŠT se projeví u proudových šifer pouze v jednom odpovídajícím znaku otevřeného textu, u blokové šifry má vliv na celý blok znaků.

Proudové šifry (13)

Synchronní a asynchronní proudové šifry

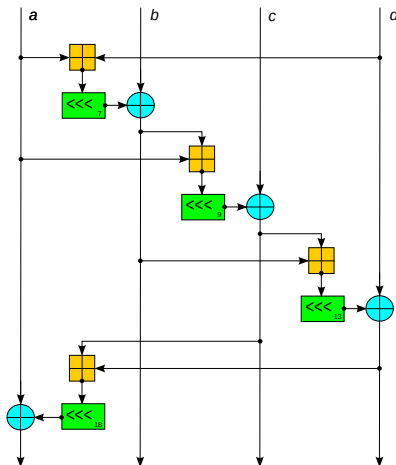
- Pokud proud hesla nezávisí na OT ani ŠT $\Rightarrow$ synchronní proudové šifry $\Rightarrow$ příjemce i odesílatel jsou přesně synchronizováni (jinak výpadek jednoho znaku ŠT naruší veškerý následující OT).
- Šifry eliminující takové chyby se nazývají asynchronní nebo samosynchronizující se šifry. U nich dojde v krátké době k synchronizaci a správné dešifraci zbývajících OT.
- To se může docílit například tím, že proud hesla je generován pomocí klíče a n předchozích znaků ŠT: $h_i = f(k, c_{i-n}, \dots, c_{i-1})$.
- K synchronizaci dojde, jakmile se přijme souvislá posloupnost $n + 1$ správných znaků ŠT.
- Historická asynchronní šifra – **Vigenèrův autokláv**.
- Heslo je pouze jedno písmeno klíče, znaky hesla byly tvořeny už přímo předchozím znakem ŠT (sčítání v modulu 26): $c_1 = p_1 + h_1$, kde $h_1 = k$ a $c_i = p_i + h_i$, $i = 2, 3, \dots$, kde $h_i = c_{i-1}$.

Proudová šifra Salsa20 (1)

- Proudová šifra – 2005 D. J. Bernsteinem.
- Modifikovaná verze Salsa20 o 12 rundách – kandidát projektu eSTREAM.
- eSTREAM (2004-08) – nové proudové šifry Profil 1 /Profil 2 – SW/HW.
- Profil 1 – Proudové šifry pro SW aplikace s požadavkem na vysokou propustnost (HC-128, Rabbit, Salsa20/12 SOSEMANUK).
- Profil 2 – Proudové šifry pro HW aplikace s omezenými prostředky, jako je omezené úložiště, počet bran nebo spotřeba energie (Grain, MICKEY, Trivium).
- Salsa20 nabízí rychlosti kolem 4–14 cyklů na B v SW (procesorech x86) a přiměřený výkon HW.

Proudová šifra Salsa20 (2)

- Základ – pseudonáhodná funkce s operacemi:
 - ▶ Add-Rotate-XOR (ARX) — 32bitové sčítání, bitové sčítání (XOR) a operace rotace.
 - ▶ Základní funkce mapuje 256bitový klíč, 64bitový nonce a 64bitový čítač na 512bitový blok keystreamu.
 - ▶ Existuje verze Salsa se 128bitovým klíčem $\Rightarrow$ Salsa20 (ChaCha) má výhodu pro uživatele v efektivním hledání libovolné pozici v keystreamu v konstantním čase.



Proudová šifra Salsa20 (3)

- Interně šifra používá bitové sčítání $\oplus$ (exkluzivní OR), 32bitové sčítání mod 2^{32} $\boxplus$ a operace rotace s konstantní vzdáleností $\lll$ na vnitřním stavu šestnácti 32bitových slov.
- Použití pouze operací add-rotate-xor zabrání možnosti časových útoků v SW implementacích.
- Vnitřní stav je tvořen 16 32bitovými slovy v matici 4×4 .
- Počáteční stav: 8 **klíčových slov**, 2 slova **pozice keystreamu**, 2 **slova nonce** (v podstatě dalších bitů pozice proudu) a 4 pevných slov:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

„expa“	Key	Key	Key
Key	„nd 3“	Nonce	Nonce
Pos.	Pos.	„2-by“	Key
Key	Key	Key	„te k“

Proudová šifra Salsa20 (4)

- 4 pevná slova tvoří výraz "expand 32-byte k", kde je každé ze slov ("expa", "nd 3", "2-by", "te k") zapsáno v ASCII.
- Základní operací v Salsa20 je čtvrtinová runda (Quarter-Round) $QR(a, b, c, d)$, které vstup tvoří 4 slova a výstupem jsou rovněž 4 slova.

$$b := b \oplus (a \boxplus d) \lll 7$$

$$c := c \oplus (b \boxplus a) \lll 9$$

$$d := d \oplus (c \boxplus b) \lll 13$$

$$a := a \oplus (d \boxplus c) \lll 18$$

- Liché rundy aplikují $QR(a, b, c, d)$ na každý ze 4 sloupců v matici.
- Sudé rundy jej aplikují na každý ze 4 řádků.
- 2 po sobě jdoucí rundy (runda po sloupcích a po řádcích) se nazývají dvourundy.

Proudová šifra Salsa20 (5)

- Lichá runda

QR(0, 4, 8, 12) $\rightarrow$ sloupec 1

QR(5, 9, 13, 1) $\rightarrow$ sloupec 2

QR(10, 14, 2, 6) $\rightarrow$ sloupec 3

QR(15, 3, 7, 11) $\rightarrow$ sloupec 4

- Sudá runda

QR(0, 1, 2, 3) $\rightarrow$ řádek 1

QR(5, 6, 7, 4) $\rightarrow$ řádek 2

QR(10, 11, 8, 9) $\rightarrow$ řádek 3

QR(15, 12, 13, 14) $\rightarrow$ řádek 4

- Zopakuje se 10 krát, tj. dohromady je to 20 rund.
- Nakonec se k výsledku přičte vstup. Přidání smíšeného pole k originálu znemožňuje obnovení vstupu. (Stejná technika je používána v hashovacích funkcích).
- Výstup je 64B keystreamu.

Proudová šifra Salsa20 (6)

Varianty Salsa20

- Salsa20 provádí na vstupu 20 rund. Existují i varianty Salsa20/8 (8 rund) a Salsa20/12 (12 rund) s redukováným počtem rund.
- Toto jsou doplňující varianty k Salsa20, neslouží jako její náhrada.
- V benchmarcích eSTREAM dosahují lepších výsledků než varianta Salsa20, ale s odpovídající nižší bezpečností.
- Bernstein (2008) – XSalsa20 se 192bitovými nonces, která je prokazatelně bezpečná vzhledem k bezpečnosti Salsa20. Je vhodnější pro aplikace, kde jsou požadovány delší nonces.

Kryptoanalýza Salsa20 (1)

- Od vzniku Salsa20 do roku 2015 byly provedené různé kryptoanalytické útoky na Salsa20.
- 2005 P. Crowley – útok na Salsa20/5 se složitostí 2^{165} pomocí zkrácené diferenciální kryptoanalýzy (DC) (použitá i dalšími autory). 1000 US \$ za "most interesting Salsa20 cryptanalysis".

Kryptoanalýza Salsa20 (2)

- 2006 Fischer et al. – útok na Salsa20/6 se složitostí $\approx 2^{177}$ a Salsa20/7 s příbuzným klíčem (related-key) se složitostí $\approx 2^{217}$.
- 2007 Tsunoo et al. – prolomení 8 z 20 rund Salsa20 a obnovení 256 bitů tajného klíče za 2^{255} operací s použitím $2^{11.37}$ párů klíčů (srovnatelné s "brute force attack").
- 2008 Aumasson et al. – útok na Salsa20/7 se složitostí 2^{151} a na Salsa20/8 se složitostí $\approx 2^{251}$. Nový koncept pravděpodobnostně neutrálních bitů klíče pro pravděpodobnostní detekci zkráceného diferenciálu.
- 2012 Shi et al. – vylepšil proti Salsa20/7 (128bitový klíč) na časovou složitost 2^{109} a na Salsa20/8 (256bitový klíč) na 2^{250} .
- 2013 Mouha et al. – důkaz: 15 rund Salsa20 je 128bitově bezpečných proti DC $\Rightarrow$ nemá žádnou diferenciální charakteristiku s pravděpodobností $> 2^{-130}$, to je víc než 2^{128} (brute force attack).

Proudová šifra ChaCha (8)

- Proudová šifra – 2008 D. J. Bernsteinem.
- Cíl zvýšení rozptylu v jedné rundě při zachování nebo vylepšení výkonu.
- 2008 Aumasson et al. – útočí kromě na Salsa20 také na ChaCha. Závěrem tvrdí, že navrhovaný útok nedokáže prolomit 128bitovou ChaCha7.
- Stejně jako Salsa20 obsahuje počáteční stav ChaCha 128bitovou konstantu, 256bitový klíč, 64bitový čítač a 64bitovou nonce.
- Tyto bity jsou uspořádané znovu do matice 4×4 32bitových slov.
- ChaCha však některá slova v počátečním stavu přeuspořádává.

Proudová šifra ChaCha (9)

● Inicializace ChaCha

„expa“	„nd 3“	„2-by“	„te k“
Key	Key	Key	Key
Key	Key	Key	Key
Count.	Count.	Nonce	Nonce

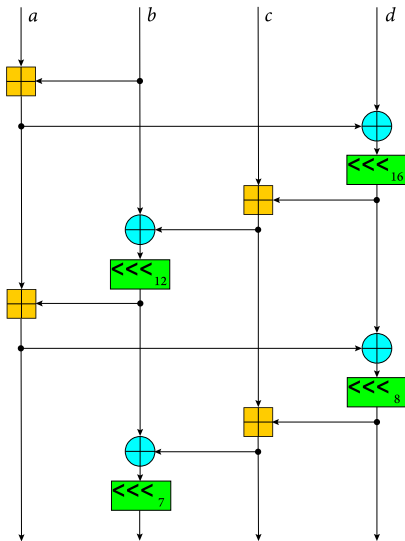
- Konstanta je stejná jako u Salsa20 ("expand 32-byte k").
- ChaCha nahrazuje čtvrt-rundový QR(a, b, c, d) Salsa20 pomocí:

$$a := a \boxplus b; \quad d := d \oplus a; \quad d \lll 16$$

$$c := c \boxplus d; \quad b := b \oplus c; \quad b \lll 12$$

$$a := a \boxplus b; \quad d := d \oplus a; \quad d \lll 8$$

$$c := c \boxplus d; \quad b := b \oplus c; \quad b \lll 7$$



Proudová šifra ChaCha (10)

- Tato verze QR ChaCha aktualizuje každé slovo $2\times$ (QR verze Salsa20 aktualizuje každé slovo pouze jednou).
- QR ChaCha navíc změny šíří rychleji. V průměru se po změně 1 vstupního bitu u QR Salsa20 změní 8 výstupních bitů, zatímco u QR ChaCha se změní 12,5 výstupních bitů.
- QR ChaCha má stejný počet sčítání, xorů a bitových rotací jako QR Salsa20.
- 2 z rotací jsou násobky 8 $\Rightarrow$ malá optimalizace na některých architekturách včetně x86.
- Změněno formátování vstupu, které podporuje efektivní optimalizaci implementace SSE.
- 32bitová slova jsou do QR plněná po sloupcích a po úhlopříčkách (bez posuvu pro jednotlivá plnění).

Proudová šifra ChaCha (11)

- Lichá runda

QR(0, 4, 8, 12) → sloupec 1

QR(1, 5, 9, 13) → sloupec 2

QR(2, 6, 10, 14) → sloupec 3

QR(3, 7, 11, 15) → sloupec 4

- Sudá runda

QR(0, 5, 10, 15) → diagonála 1 (hlavní)

QR(1, 6, 11, 12) → diagonála 2

QR(2, 7, 8, 13) → diagonála 3

QR(3, 4, 9, 14) → diagonála 4

- Zopakuje se 10 krát, tj. dohromady je to 20 rund.
- Nakonec se k výsledku přičte vstup. Přičtení vstupu k výstupu znemožňuje obnovení vstupu. (Stejná technika je používána v hashovacích funkcích).
- Výstup je 64B keystreamu.

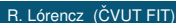
Proudová šifra ChaCha (12)

- ChaCha je základem hašovací funkce BLAKE, finalista soutěže hašovacích funkcí NIST (rychlejších verze BLAKE2 a BLAKE3).
- Existuje XChaCha, která je analogická k XSalsa20.
- Google vybral ChaCha20 a Bernsteinův kód pro ověřování zpráv Poly1305 a navrhl autentizované šifrování – ChaCha20-Poly1305, které jsou používány v protokolu QUIC (součást HTTP/3) a v šifře chacha20-poly1305@openssh.com v OpenSSH.
- ChaCha20 je generátorem náhodných čísel pro arc4random v operačních systémech Free/Open/NetBSD (nahradil RC4).
- Implementace IETF upravila ChaCha – změnou 64bitové nonce a 64bitového čítače na 96bitovou nonce a 32bitový čítač bloků $\Rightarrow$ maximální délka zprávy je 2^{32} bloků po 64 bajtech (256 GiB).
- ChaCha20 nahrazuje Advanced Encryption Standard (AES) v systémech, kde procesor nemá akceleraci AES (mobilní zařízení s procesory založenými na architektuře ARM).

Úvod

- A5/1 je synchronní proudová šifra, která se používá na zabezpečení komunikace mobilních telefonů, které jsou v souladu se standardem GSM.
- Šifra vznikla v roce 1987 a dlouhou dobu byla její struktura utajována. Až v roce 1999 byla pomocí reverzní analýzy kompletně zrekonstruována.
- Šifra se skládá z kombinace tří LFSR (Linear feedback shift register) a nelineárního mechanismu určujícího posunutí registrů v daném čase.
- Strukturu šifry A5/1 názorně ilustruje následující obrázek.

→ → → Shift direction → → →



Mechanismus posunu registrů

- Zvýrazněné bity $R_1[8]$, $R_2[10]$ a $R_3[10]$ z předešlého obrázku se nazývají taktovací bity a na jejich hodnotách závisí, které registry se v následujícím taktu posunou.
- Je zřejmé, že mohou-li tři bity nabývat každý jen dvou hodnot, musí jedna hodnota převládat.
- Např. když $R_1[8] = 0$, $R_2[10] = 0$ a $R_3[10] = 1$, tak převládá hodnota 0, protože se vyskytuje u většiny taktovacích bitů.
- V každém taktu se posunou jen ty registry, ve kterých hodnota taktovacího bitu převládá.
- Tento mechanismus posunu registrů je jediným nelineárním prvkem šifry a na něm je postavena bezpečnost šifry.

Vlastnosti registrů

- Aktualizace samotných registrů je určena bity zpětné vazby, které jsou dány koeficienty primitivního polynomu
- Použití primitivního polynomu u LFSR zaručuje maximální možnou periodu výstupní posloupnosti registru.
- Bity zpětné vazby a další vlastnosti LFSR jsou popsány v následující tabulce.

registr	délka registru	taktovací bit	bity zpětné vazby
R_1	19	8	18,17,16,13
R_2	22	10	20,21
R_3	23	10	22,21,20,7

Takt šifry

- V jednom taktu se vyprodukuje jeden bit keystreamu.
- Takt šifry vypadá následovně:
 - ▶ Spočítá se většinová hodnota h taktovacích bitů.
 - ▶ Posunou se ty registry, u kterých je taktovací bit rovný h .
 - ▶ Spočte se výstupní bit šifry k_i jako XOR bitů registrů s nejvyšším indexem, tj. $k_i = R_1[18] \oplus R_2[21] \oplus R_3[22]$.
- Bity k_i tvoří tzv. keystream. Šifrový text c získáme tak, že na otevřený text p „naxorujeme“ keystream k , tj. $c_i = p_i \oplus k_i$.
- Před samotným produkováním keystreamu probíhá inicializace šifry, kdy se registry naplní 64-bitovým tajným klíčem K a 22-bitovým inicializačním vektorem IV .

Inicializace šifry

- 1: Nastav všechna políčka všech třech registrů na nulu.
- 2: **for** $i = 0$ **to** 63 **do**
- 3: $R_1[0] = R_1[0] \oplus K[i]$
- 4: $R_2[0] = R_2[0] \oplus K[i]$
- 5: $R_3[0] = R_3[0] \oplus K[i]$
- 6: Posuň všechny tři registry.
- 7: **end for**
- 8: **for** $i = 0$ **to** 21 **do**
- 9: $R_1[0] = R_1[0] \oplus IV[i]$
- 10: $R_2[0] = R_2[0] \oplus IV[i]$
- 11: $R_3[0] = R_3[0] \oplus IV[i]$
- 12: Posuň všechny tři registry.
- 13: **end for**
- 14: **for** $i = 0$ **to** 99 **do**
- 15: Vykonej takt šifry (s použitím Majoritní funkce).
- 16: **end for**

Šifrování

- Komunikace spočívá v přenosu posloupnosti rámců, každý délky 228 bitů, přičemž 114 bitů se využívá na zašifrování dat vyslaných z GSM centra k uživatelskému mobilnímu telefonu a pomocí dalších 114 bitů se zašifrují data vyslané z mobilního telefonu do centra.
- Pro jeden rámec (frame) se vykonají následující kroky:
 - ▶ Na začátku proběhne inicializační fáze, ve které se registry naplní tajným klíčem a inicializačním vektorem a 100-krát se vykoná takt šifry, přičemž se zahodí výstupní bit šifry.
 - ▶ Pak se vykoná 228 taktů a vyprodukuje se 228 bitů keystreamu, které se použijí k šifrování.
- Keystream z jednoho rámce zabezpečí řádově milisekundy hovoru.
- Pro další rámec stejného hovoru se použije jiný inicializační vektor, ale stejný tajný klíč.

- M.Bricero, I.Goldberg, D.Wagner, A pedagogical implementation of A5/1, <http://www.scard.org/gsm/a51.html>, 1999.
- E.Biham, O.Dunkelman, Cryptanalysis of the A5/1 GSM stream Cipher, Lecture notes in Computer Science, vol-1977, 2000, pp. 43-51, (Indocrypt 2000).
- Animation of A5/1 cipher, <http://www.youtube.com/watch?v=LgZAI3DdUA4>
- <https://cs.wikipedia.org/wiki/Salsa20>

Kryptografie a bezpečnost

4. Blokované šifry, DES, 3DES, AES, operační módy blokových šifer

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Blokové šifry
- DES, 3DES
- AES
- Operační módy

Blokové šifry (1)

Bloková šifra

- Nechť A je abeceda q symbolů, $t \in \mathbb{N}$ a $M = C$ je množina všech řetězců délky t nad A . Nechť K je množina klíčů.
- Bloková šifra je šifrovací systém (M, C, K, E, D) , kde E a D jsou zobrazení, definující pro každé $k \in K$ transformaci zašifrování E_k a dešifrování D_k tak, že zašifrování bloků OT $m_1, m_2, m_3, \dots$, kde $m_i \in M$, probíhá podle vztahu $c_i = E_k(m_i)$ a dešifrování podle vztahu $m_i = D_k(c_i)$ pro každé $i \in \mathbb{N}$.
- Pro blokovou šifru je podstatné, že všechny bloky OT jsou šifrovány toutéž transformací a všechny bloky ŠT jsou dešifrovány toutéž transformací.
- Za určitých okolností můžeme za blokové šifry považovat šifry substituční a transpoziční.

Blokové šifry (2)

Bloková šifra

- Dnes nejběžnější šifry používají blok 128 bitů, na který se přešlo na přelomu letopočtu ze 64 bitů.
- Historicky nejznámější blokové šifry používaly (a používají) blok o délce 64b: DES (Data Encryption Standard), TripleDES, IDEA, CAST aj.,
- Některé blokové šifry využívají principy **algoritmů Feistelova typu** umožňující postupnou aplikací relativně jednoduchých transformací vytvořit složitý kryptografický algoritmus.
- Tento princip je využíván v symetrických šifrovacích algoritmech, nejznámějším algoritmem tohoto typu je algoritmus DES.
- Tento přístup je využíván také v jiných oblastech. Například u zabezpečovacích kódů jsou využívány tzv. zřetězené kódy, které ze 2 relativně jednoduchých standardních zabezpečovacích kódů vytvářejí výkonné zabezpečovací kódy.

Blokové šifry (3)

Základní principy algoritmů Feistelova typu (1)

Jednoduchý příklad (ne obecný princip):

Mějme 2 permutace

$$f_1 = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 1 & 2 & 4 \end{pmatrix} \quad f_2 = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix},$$

kde zápis funkce f_1 znamená, že 1. bit se přesune na 2. pozici, 2. bit na 3. pozici, 3. bit na 1. pozici, atd. Mějme následující způsob šifrování.

Původní 8b zprávu $m \in M$ rozdělíme na 2 4b bloky $m = (m_0 \ m_1) \Rightarrow$ vytvoříme novou 8b zprávu $c_1 = (m_1 m_2)$, kde $m_2 = m_0 \oplus f_1(m_1)$ a tento postup ještě jednou zopakujeme, tj. vytvoříme $c_2 = (m_2 \ m_3)$, kde $m_3 = m_1 \oplus f_2(m_2)$.

Pro $m = 1001 \ 1101$ dostáváme postupně:

$c_1 = (1101 \ 1110)$, kde $m_2 = 1001 \oplus f_1(1101) = 1001 \oplus 0111 = 1110$

$c_2 = (1110 \ 0000)$, kde $m_3 = 1101 \oplus f_2(1110) = 1101 \oplus 1101 = 0000$

Blokové šifry (4)

Základní principy algoritmů Feistelova typu (2)

Dešifrování je možné realizovat opačným postupem:

$m_1 = m_3 \oplus f_2(m_2)$ a $m_0 = m_2 \oplus f_1(m_1)$, přičemž dešifrovaná hodnota je $m = 1001\ 1101$.

Pro $c_2 = (m_2\ m_3) = 1110\ 0000$ dostáváme:

$$m_1 = 0000 \oplus f_2(1110) = 0000 \oplus 1101 = 1101.$$

A pro m_0 dostáváme:

$$m_0 = m_2 \oplus f_1(m_1) = 1110 \oplus f_1(1101) = 1110 \oplus 0111 = 1001 \text{ a } \Rightarrow \\ m = (m_0\ m_1) = 1001\ 1101$$

Z předcházejících příkladů je zřejmé, že funkce f_1 a f_2 nemusí být prosté, protože není potřebné počítat jejich inverzi. Všech možných funkcí $f : V_4 \rightarrow V_4$ je $(2^4)^{2^4} = 16^{16} \doteq 1.85 \cdot 10^{19}$.

Blokové šifry (5)

Základní principy algoritmů Feistelova typu (3)

- Všechny funkce lze do Feistelovské šifry použít.
- Ale ne všechny funkce f jsou pro šifrování vhodné.
- Uvedené způsoby šifrování jsou speciálním případem tzv. Feistelových kryptosystémů.

Definice – Feistelův kryptosystém

Nechť množina zpráv M je složená z všech možných $2n$ -tic V_{2n} a prostor klíčů K tvoří všechny možné h -tice funkcí $k = \{f_1, f_2, \dots, f_h\}$, $f_i : V_n \rightarrow V_n$ pro každé $i = 1, 2, \dots, h$ a prostor zašifrovaných textů $C = V_{2n}$. Zobrazení $T_k : K \times V_{2n} \rightarrow V_{2n}$, definované rekurentně vztahy

$$\begin{aligned} m_{i+1} &= m_{i-1} + f_i(m_i), \quad \text{pro } i = 1, 2, \dots, h \\ T_k(m) &= (m_h \ m_{h+1}), \end{aligned}$$

kde $m = (m_0 \ m_1) \in M$, definuje Feistelův kryptosystem.

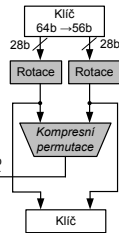
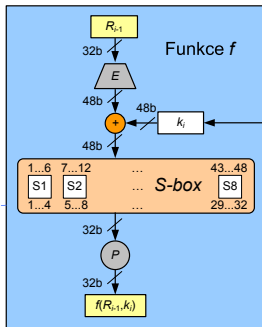
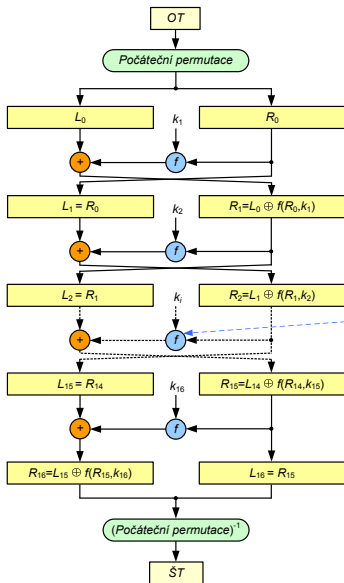
Algoritmus DES

- Veřejná soutěž (1977): šifrovací standard (FIPS 46-3) v USA pro ochranu citlivých ale neutajovaných dat ve státní správě.
- Součást průmyslových, internetových a bankovních standardů.
- 1977: varování – příliš krátký klíč 56b, který byl do původního návrhu IBM zanesen vlivem americké tajné služby NSA.
- DES – intenzivní výzkum a útoky $\Rightarrow$ objeveny teoretické negativní vlastnosti jako: tzv. slabé a poloslabé klíče, komplementárnost a teoreticky úspěšná lineární a diferenciální kryptoanalýza.
- V praxi jedinou zásadní nevýhodou je pouze krátký klíč.
- 1998: stroj – DES-Cracker, lušticí DES hrubou silou.
- DES jako americký standard skončil (jen v "dobíhajících" systémech a kvůli kompatibilitě) a místo něj: Triple-DES, (FIPS 46-3).
- Od 26. 5. 2002 – šifrovací standard nové generace AES.

Stavební části DES (1)

- DES je iterovaná šifra typu $E_{k_{16}}(E_{k_{15}}(\dots(E_{k_1}(m_i)\dots)))$.
- Používá 16 rund a 64b bloky OT a ŠT. Šifrovací klíč k má délku 56 b (vyjadřuje se ale jako 64b číslo, kde každý 8. bit je bit parity)
- 56b klíč k je v inicializační fázi nebo za chodu algoritmu expandován na 16 rundovních klíčů k_1 až k_{16} , které jsou řetězci 48 bitů, každý z těchto bitů je některým bitem původního klíče k .
- Místo počátečního zašumění OT se používá bezklíčová pevná permutace *Počáteční permutace* a místo závěrečného zašumění permutace k ní inverzní (*Počáteční permutace*)⁻¹.
- Po *počáteční permutaci* je blok rozdělen na dvě 32b poloviny (L_0, R_0) . Každá ze 16 rund $i = 1, 2, \dots, 16$ transformuje (L_i, R_i) na novou hodnotu $(L_{i+1}, R_{i+1}) = (R_i, L_i \oplus f(R_i, k_{i+1}))$, liší se jen použitím jiného rundovního klíče k_i .
- Ve smyslu definice Feistelova kryptosystému je v tomto případě $h = 16$ a $2n = 64$.

Algoritmus DES



Stavební části DES (2)

- Po 16. rundě dochází ještě k výměně pravé a levé strany:
 $(L_{16}, R_{16}) = (R_{15}, L_{15} \oplus f(R_{15}, k_{16}))$ a závěrečné permutaci
(Počáteční permutace)⁻¹.
- Dešifrování probíhá stejným způsobem jako zašifrování, pouze se obrátí pořadí výběru rundovních klíčů.

Rundovní funkce f

- Rundovní funkce se skládá z binárního načtení klíče k_i na vstup. 48b klíč k_i je vytvořen po kompresi ze 2 28b rotovaných částí původního klíče k , kde počet bitů rotace je závislý na čísle rundy.
- Tento klíč k_i je dál xorován s expandovanou 32b částí R_{i-1} , která je expanzně permutovaná v bloku **E** z 32b na 48b. Tato operace kromě rozšíření daného 32b slova také permutuje bity tohoto slova tak, aby se dosáhlo lavinového efektu.

Stavební části DES (3)

- Následně je prováděna pevná **nelineární** substituce na úrovni 6b znaků do 4b znaků s následnou transpozicí na úrovni bitů. Těmito operacemi se dosahuje dobré difúze i konfúze.
- Použité substituce se nazývají substituční boxy: **S-boxy**, jsou jediným nelineárním prvkem schématu. Pokud bychom substituce vynechali, mohli bychom vztahy mezi ŠT, OT a klíčem popsat pomocí operace binárního sčítání $\oplus$, tedy lineárními vztahy.
- Tato nelinearita je překážkou jednoduchého řešení rovnic, vyjadřujících vztah mezi OT, ŠT a K.
- Následně je 32b výsledné slovo z S-boxu permutováno v bloku **P**. Tato permutace převádí každý vstupní bit do výstupu, kde žádný vstupní bit se nepoužije $2\times$.
- Nakonec se výsledek permutace sečte modulo 2 s levou 32b polovinou a začne další runda.

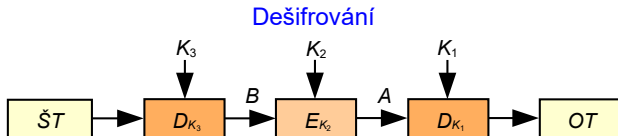
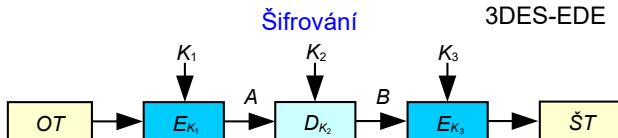
TripleDES (1)

TripleDES

- TripleDES (3DES) prodlužuje klíč originální DES tím, že používá DES jako stavební prvek celkem $3 \times$ s 2 nebo 3 různými klíči.
- Nejčastěji se používá varianta EDE této šifry, která je definována ve standardu FIPS PUB 46-3 (v bankovní normě X9.52).
- Vstupní data OT jsou zašifrována podle vztahu $\check{S}T = E_{K_3}(D_{K_2}(E_{K_1}(OT)))$, kde K_1 , K_2 a K_3 jsou buď 3 různé klíče nebo $K_3 = K_1$. Varianta EDE byla zavedena z důvodu kompatibility $\rightarrow$ při rovnosti všech klíčů 3DES = DES.
- Klíč 3DES je tedy buď 112 bitů (2 klíče) nebo 168 bitů (3 klíče). 3DES je spolehlivá $\rightarrow$ klíč je dostatečně dlouhý a teoretickým slabínám (komplementárnost, slabé klíče) se dá předcházet.
- 3DES lze, jako jakoukoliv jinou blokovou šifru, použít v různých operačních modech (CBC mod $\Rightarrow$ 3DES-EDE-CBC).

TripleDES (2)

Algoritmus 3DES



$K_1 \neq K_2 \neq K_3 \Rightarrow 3 \times \text{klíč} = 168\text{b} \rightarrow 3\text{DES}$

$K_1 = K_3 \neq K_2 \Rightarrow 2 \times \text{klíč} = 112\text{b} \rightarrow 3\text{DES}$

$K_1 = K_2 = K_3 \Rightarrow 1 \times \text{klíč} = 56\text{b} \rightarrow \text{DES}$

AES (1)

- Po útocích hrubou silou na DES americký standardizační úřad připravil náhradu - **Advanced Encryption Standard (AES)**.
- 2. 1. 1997 výběrové řízení na AES – 15 kandidátů.
- Z 5 finalistů byl vybrán algoritmus Rijndael [rájndol] (autoři J. Daemen a V. Rijmen).
- Jako AES byl přijat s účinností od 26. května 2002 a byl vydán jako standard v oficiální publikaci FIPS PUB 197.
- AES je bloková šifra s délkou bloku 128 bitů, čímž se odlišuje od dřívějších blokových šifer, které měly blok 64bitový.
- AES podporuje tři délky klíče: 128, 192 a 256 bitů $\Rightarrow$ se částečně mění algoritmus (počet rund je po řadě 10, 12 a 14).
- Větší délka bloku a klíče zabraňují útokům, které byly aplikované na DES. AES nemá slabé klíče, je odolný proti známým útokům a metodám lineární a diferenciální kryptoanalýzy.
- **AES není Feistelovského typu!**

- Algoritmus zašifrování i dešifrování se dá výhodně programovat na různých typech procesorů, má malé nároky na paměť i velikost kódu a je vhodný i pro paralelní zpracování.
- AES bude pravděpodobně platným šifrovacím standardem několik desetiletí a bude mít obrovský vliv na počítačovou bezpečnost.
- Označíme-li délku klíče N_k jako počet 32b slov, máme $N_k = 4, 6$ a 8 pro délku klíče 128, 192 a 256 bitů.
- AES je iterativní šifra, počet rund N_r se mění podle délky klíče: $N_r = N_k + 6$, tj. je to 10, 12 nebo 14 rund.
- Tato skutečnost odráží nutnost zajistit konfúzi vzhledem ke klíči. Algoritmus pracuje s prvky Galoisova tělesa $GF(2^8)$ a s polynomy, jejichž koeficienty jsou prvky z $GF(2^8)$. Bajt s bity $(b_7, \dots, b_0)$ je zde chápán jako polynom $b_7x^7 + \dots + b_1x^1 + b_0$ a operace “násobení bajtů” odpovídá násobení těchto polynomů modulo $m(x) = x^8 + x^4 + x^3 + x^1 + 1$.

- AES využívá $1 + N_r$ rundovních 128b klíčů, které se definovaným způsobem odvozují ze šifrovacího klíče.
- Před zahájením 1. rundy zašifrování se provede úvodní zašumění, kdy se na OT naxoruje první rundovní klíč (128b na 128b) $\Rightarrow$
- N_r shodných rund (s výjimkou poslední, kdy se neprovede operace **MixColumns**), při kterých výstup z každé předchozí rundy slouží jako vstup do rundy následující. Tím dochází k postupnému mnohonásobnému zesložit'ování výstupu.
- Na počátku každé rundy se vždy vstup (16 B) naplní postupně shora dolů a zleva do prava po sloupcích do matice 4×4 B
 $\mathbf{A} = (a_{ij}) \ i, j = 0, 1, 2, 3.$
- Na každý bajt matice $\mathbf{A}$ se zvlášť aplikuje substituce, daná pevnou substituční tabulkou **SubBytes**.

- Následuje operace **ShiftRows**: Řádky matice **A** se cyklicky posunou postupně o 0-3 bajty doleva, 1. řádek o 0, druhý o 1, třetí o 2 a čtvrtý o 3, čímž dochází k transpozici na úrovni bajtů.
- Dále se na každý jednotlivý sloupec matice aplikuje operace **MixColumns**, která je substitucí 32 bitů na 32 bitů. Tuto substituci lze však popsat lineárními vztahy – všechny výstupní bity jsou nějakou lineární kombinací vstupních bitů. Označíme-li jednotlivé bajty v rámci daného sloupce matice **A** (shora dolů) jako a_0 až a_3 , pak výstupem budou jejich nové hodnoty b_0 až b_3 , podle vztahů

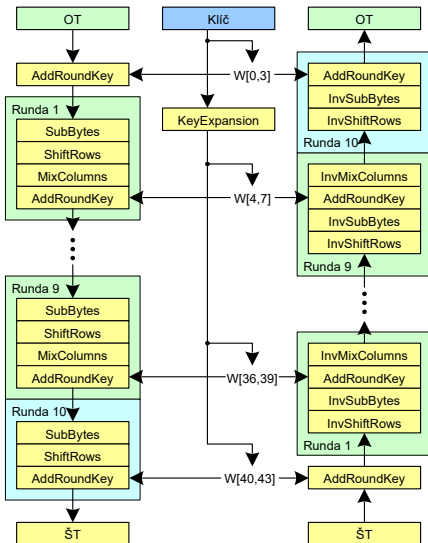
$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} '02' & '03' & '01' & '01' \\ '01' & '02' & '03' & '01' \\ '01' & '01' & '02' & '03' \\ '03' & '01' & '01' & '02' \end{pmatrix} \times \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

- Násobení je násobení prvků $GF(2^8)$. Konstantní prvky tohoto pole jsou vyjádřeny hexadecimálně.
- Jako poslední operace rundy se vykoná transformace **AddRoundKey**, v rámci níž se na jednotlivé sloupce matice **A** zleva doprava naxorují odpovídající rundovní klíč (4×32 bitů). Tím je jedna runda popsána a začíná další. Po poslední rundě se ŠT jen vyčte z matice **A**.
- Při odšifrování se používají operace inverzní k operacím použitým při zašifrování, neboť všechny jsou reverzibilní.
- Nelinearity v AES se objevují pouze v substituci **SubBytes**. V roce 2002 bylo zjištěno, že vzájemné vztahy výstupních ($y_1, \dots, y_8$) a vstupních ($x_1, \dots, x_8$) bitů lze popsat implicitními rovnicemi $(x_1, \dots, x_8, y_1, \dots, y_8) = 0$ pouze druhého řádu.

AES (6)

Algoritmus AES

AES – Struktura šifrování a dešifrování



Operační módy blokových šifer (1)

- Operační módy blokových šifer jsou způsoby použití blokových šifer v daném kryptosystému, kde OT není jen 1 blok blokové šifry, ale obecně posloupnost znaků dané abecedy.
- U moderních blokových šifer chápeme jako znaky bajty, i když se délka bloku N uvádí v bitech. Obvykle $N = 64$ nebo 128 . Pomocí operačních modů můžeme získat nové zajímavé vlastnosti a využití blokových šifer. Mody: ECB, CFB, OFB, CBC, CTR a CBC-MAC.

Mod ECB (Electronic Codebook)

- Tento operační modus se nazývá elektronická kódová kniha a je základním modelem.
- Posloupnost bloků otevřeného textu $OT_1, OT_2, \dots, OT_n$ se šifruje tak, že každý blok je šifrován zvlášť, což lze vyjádřit vztahem $ST_i = E_K(OT_i), i = 1, 2, \dots, n$.

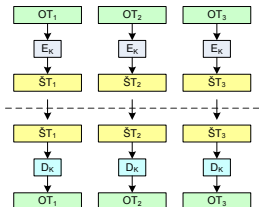
Operační módy blokových šifer (2)

- Nevýhodou takového typu šifrování je, že stejné bloky OT mají vždy stejný šifrový obraz.
- Pokud nalezneme několik shodných bloků $\Rightarrow$ to může v určitém kontextu dokonce rozkrývat i hodnotu otevřeného bloku (například prázdné sektory na disku jsou vyplněny hodnotou 0x00 apod.
- Integrita a mod ECB – Ve zprávě šifrované modelem ECB může útočník bloky ŠT vyměňovat, vkládat nebo vyjímat, a tak snadno docílovat pro uživatele nežádoucích změn v OT, zejména pokud nějakou dvojici (OT, ŠT) už zná.
- Tím je opět ilustrován v praxi často opomíjený fakt, že integrita OT se šifrováním nezajistí.

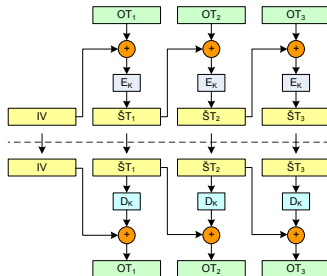
Synchronní proudové šifry mají nedostatečnou vlastnost difúze neboť pracují jen nad jednotlivými znaky abecedy. **Moderní blokové šifry naproti tomu dosahují velmi dobrých vlastností jak difúze, tak konfúze.**

Operační módy blokových šifer (3)

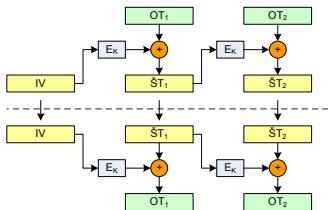
ECB – Electronic Code Book



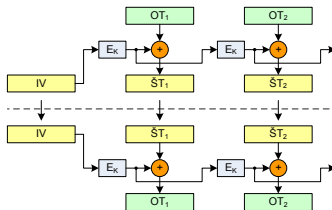
CBC – Cipher Block Chaining



CFB – Cipher FeedBack



OFB – Output FeedBack



Operační módy blokových šifer (4)

Mod CBC (Cipher Block Chaining)

- Pro rozšíření difúze na více bloků byl definován modus řetězení šifrového textu (CBC). Každý blok OT se v něm nejprve modifikuje předchozím blokem ŠT, a teprve poté se šifruje.
- To zajišťuje, že běžný šifrový blok závisí na celém předchozím OT z důvodu řetězení této závislosti přes předchozí ŠT.
- CBC je nejpoužívanějším operačním módem blokových šifer. Eliminuje některé slabiny modu ECB a zajišťuje difúzi celého předchozího OT do daného bloku ŠT.
- 1. blok OT je modifikován náhodnou hodnotou IV. Šifrování se provádí podle vztahů

$$\check{S}T_0 = IV, \check{S}T_i = E_K(OT_i \oplus \check{S}T_{i-1}), \quad i = 1, 2, \dots, n$$

a dešifrování podle vztahu

$$\check{S}T_0 = IV, OT_i = \check{S}T_{i-1} \oplus D_K(\check{S}T_i), \quad i = 1, 2, \dots, n$$

Operační módy blokových šifer (5)

- Náhodný IV způsobí, že budeme-li šifrovat jeden a tentýž, byť i velmi dlouhý, OT $2\times$, obdržíme naprosto odlišný ŠT.
- Řetězení mírně znesnadňuje útoky v porovnání s ECB.
- Z definice modu CBC však vyplývá vlastnost samosynchronizace. Proces dešifrování je schopen se zotavit a produkovat správný OT už při 2 za sebou jdoucích správných blocích ŠT (ŠT_{i-1} a ŠT_i).

Mody CFB a OFB (Cipher Feedback, Output Feedback)

- Převádí blokovou šifru na proudovou. Inicializační hodnota IV nastavuje konečný automat do náhodné polohy.
- Automat produkuje posloupnost hesla, které se jako u proudových šifer *xoruje* na OT. 1. blok hesla se získá zašifrováním IV.
- Vzniklé heslo (OFB) nebo vzniklý ŠT (CFB) se přivádějí na vstup blokové šify a jejich zašifrováním je získán další blok hesla.
- OFB má vlastnost čisté (synchronní) proudové šify – heslo je generováno zcela autonomně bez vlivu OT a ŠT.

Operační módy blokových šifer (6)

- CFB je kombinací vlastností CBC a proudové šifry.
- Předpis pro zašifrování v CFB:

$$\check{S}T_0 = IV, \check{S}T_i = OT_i \oplus E_K(\check{S}T_{i-1}), \quad i = 1, 2, \dots, n$$

a dešifrování v modu CFB:

$$\check{S}T_0 = IV, OT_i = \check{S}T_i \oplus E_K(\check{S}T_{i-1}), \quad i = 1, 2, \dots, n$$

- Předpis pro zašifrování v OFB:

$$\check{S}T_0 = IV, H = E_K(IV), \{\check{S}T_i = OT_i \oplus H, H = E_K(H)\}, \quad i = 1, 2, \dots, n$$

a dešifrování v modu OFB:

$$\check{S}T_0 = IV, H = E_K(IV), \{OT_i = \check{S}T_i \oplus H, H = E_K(H)\}, \quad i = 1, 2, \dots, n$$

- V modech OFB a CFB se bloková šifra používá jen jednosměrně, tj. jen transformace $E_K \Rightarrow$ výhodné při HW realizaci.

Operační módy blokových šifer (7)

- Jako výstup lze použít část bloku hesla/ŠT, např. b bitů $\Rightarrow$ se b bitů hesla (OFB) nebo b bitů vzniklého ŠT (CFB) vede zprava do vstupního registru, přičemž původní obsah vstupního registru se posune doleva o b bitů (b bitů nejvíce vlevo z registru vypadne).
- CFB je samosynchronní, a to podle délky zpětné vazby. Je-li b bitů, pak postačí 2 nenarušené b -bitové bloky ŠT, aby se OT zesynchronizoval.
- OFB $\rightarrow$ synchronní proudová šifra. Heslo generuje konečným automatem, který má maximálně 2^N vnitřních stavů $\Rightarrow$
- se produkce hesla musí opakovat. Délka periody hesla je proto maximálně 2^N , její konkrétní délka je určena hodnotou IV a může se pohybovat náhodně v rozmezí od 1 do 2^N .
- Struktura hesla je značně závislá na tom, zda zpětná vazba je plná nebo nikoli. Pro $b < N$ je střední hodnota délky periody pouze cca $2^{N/2}$, zatímco pro $b = N$ je to 2^{N-1} .

Čítačový mod CTR (Counter Mode)

- Podobný OFB, převádí blokovou šifru na synchronní proudovou. Není problém s neznámou délkou periody hesla (je dána předem).
- IV se načte do vstupního registru (čítače) T . Po jeho zašifrování vzniká první blok hesla. Poté dojde k aktualizaci čítače T , nejčastěji přičtením jedničky a ke generování dalšího bloku hesla.
- Heslo se může využít v plné šíři bloku nebo jen jeho $b < N$ bitů. Způsob aktualizace čítače je definován volně, inkrementovat se může jen například dolních B bitů čítače T .
- V žádných zprávách šifrovaných tímto klíčem nesmí dojít k vygenerování stejného bloku hesla vícekrát $\Rightarrow$ obsah čítače nesmí být stejný.
- Jinak dvojí použití hesla $\rightarrow$ rozluštění OT.

Operační módy blokových šifer (9)

- Předpis pro šifrování v modu CTR je:

$$CTR_i = |IV + i - 1|_{2^B}, H_i = E_K(CTR_i), \check{S}T_i = OT_i \oplus H_i, \quad i = 1, 2, \dots, n$$

a pro dešifrování:

$$CTR_i = |IV + i - 1|_{2^B}, H_i = E_K(CTR_i), OT_i = \check{S}T_i \oplus H_i, \quad i = 1, 2, \dots, n$$

- Výstupní blok lze použít celý nebo jen jeho část. Smyslem modu je zaručit maximální periodu hesla, což je zaručeno periodou čítače.
- Výhoda: heslo může být vypočítáno jen na základě pozice otevřeného textu a IV, nezávisle na ničem jiném.
- Tuto vlastnost má i mod OFB, ale k vypočítání hesla v tomto případě pro nějaký blok předchází výpočet předešlých hesel.
- Naproti tomu u modu CTR se vypočte hodnota čítače a provede se jen jedna transformace $E_K: E_K(counter)$.

Operační módy blokových šifer (9)

Metoda solení

- U operačních modů CBC, OFB, CFB i CTR je možné využívat metodu solení IV.
- Komunikujícímu protějšku se předává hodnota IV, ale k šifrování se použije jiná hodnota IV' ("osolený IV").
- Tato hodnota se na obou stranách vypočítá z IV a klíče K nějakým definovaným způsobem. Např. to může být hašovací hodnota vypočítaná ze zřetězení obou hodnot.
- Bezpečnostní výhodou je, že skutečně použitá inicializační hodnota IV' se neobjevuje nikde na komunikačním kanálu.

Mod CBC-MAC (Message Authentication Code)

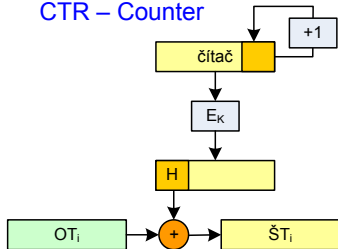
- Proudové a blokové šifry zajišťují důvěrnost, **ne integritu zpráv**.
- Mody CBC a CFB sice způsobí mírnou propagaci chyby (chyba v jednom bloku ŠT naruší 2 bloky OT), ale v systémech, kde není ve vlastním OT zajištěna nějaká redundance, mohou být zpracována chybná data.

Operační módy blokových šifer (10)

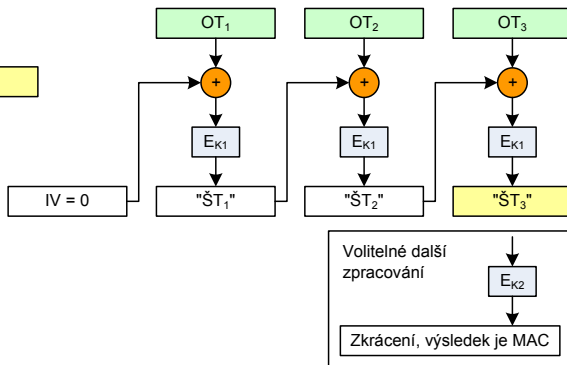
- Autentizační kód zprávy (CBC-MAC) řeší právě zajištění neporušenosti dat. Tento zabezpečovací kód autentizuje původ zprávy a řeší obranu proti náhodným i úmyslným změnám nebo chybám na komunikačním kanálu.
- CBC-MAC je krátký kód, který vznikne zpracováním zprávy s tajným klíčem (K_1). **Klíč je jiný, než k šifrování zprávy.**
- Výpočet CBC-MAC probíhá tak, že se zpráva jakoby šifruje v modu CBC s nulovým IV, ale průběžný ŠT se nikam neodesílá.
- CBC-MAC je pak tvořen až posledním blokem $\check{S}T_n$, přičemž je možné ještě jedno přídatné šifrování navíc, tj.
 $CBC - MAC = E_{K_2}(\check{S}T_n)$. Z výsledného bloku se někdy bere jen určitá část (polovina bloku) o délce potřebné k vytvoření odolného zabezpečovacího kódu.
- CBC-MAC zajišťuje službu **autentizace původu dat**. Protože je to symetrická technika, nezaručuje **nepopiratelnost**.

Operační módy blokových šifer (11)

CTR – Counter



MAC – Message Authentication Code



Kryptografie a bezpečnost

5. Hašovací funkce, SHA-x, HMAC

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLÓGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Hašovací funkce
- SHA-x
- HMAC

Hašovací funkce (1)

- Silný nástroj moderní kryptologie.
- Jedna z klíčových kryptologických myšlenek počítačové revoluce. Přinesly řadu nových použití.
- Základní pojmy: jednosměrnost a bezkoliznost.

Definice – Jednosměrné funkce

Funkce $f : X \rightarrow Y$, pro něž je snadné z jakékoli hodnoty $x \in X$ vypočítat $y = f(x)$, ale pro nějaký náhodně vybraný obraz $y \in f(X)$ nelze (je to pro nás výpočetně nemožné) najít její vzor $x \in X$ tak, aby $y = f(x)$. Přitom víme, že minimálně jeden takový vzor existuje.

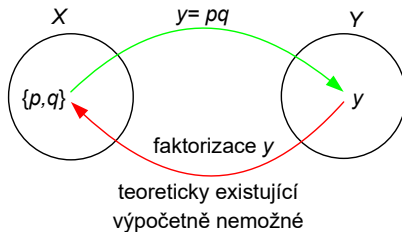
Jednosměrné funkce dělíme na:

- 1 Jednosměrné, pro které je výpočetně nemožné, ale teoreticky proveditelné, najít vzor z obrazu.
- 2 Jednosměrné funkce s padacími dvířky, u kterých lze najít vzor z obrazu, ale jen za předpokladu znalosti „padacích dvířek“ – klíče.

Hašovací funkce (2)

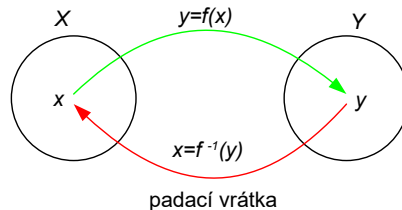
Jednosměrné funkce 1. typu

Jednosměrnost je dána složitostí operace a její inverze, např. násobení dvou prvočísel (p a q) vs. faktorizace.



Jednosměrné funkce 2. typu

Jednosměrnost je dána znalostí „padacích vrátek“, například u asymetrické kryptografie je to klíč.



Hašovací funkce (3)

Kryptografická hašovací funkce

- Původní význam hašovací funkce byl označením funkce, která libovolně velkému vstupu přiřazovala krátký hašovací kód o pevně definované délce.
- V současnosti se pojem hašovací funkce používá v kryptografii pro kryptografickou hašovací funkci, která má oproti původní definici ještě navíc vlastnost **jednosměrnosti a bezkoliznosti**.

Definice – Hašovací funkce

Mějme přirozené číslo d a množinu X všech binárních řetězců délky 0 až d (prázdný řetězec je platným vstupem a má délku 0).

Funkci $h : X \rightarrow \{0, 1\}^n$ nazveme hašovací, jestliže je jednosměrná **1. typu** a bezkolizní.

Říkáme, že každému binárnímu řetězci z množiny X přiřadí binární **hašovací kód** (haš, hash) délky n bitů.

Hašovací funkce (4)

Vstup a výstup hašovací funkce

- Hašovací funkce h zpracovává prakticky neomezeně dlouhá vstupní data M na krátký výstupní hašový kód $h(M)$ pevné a předem stanovené délky n bitů.
- Například u hašovacích funkcí SHA-1/SHA-256/SHA-512 je to 160/256/512 bitů.

Hašovací funkce jako náhodné orákulum

Z hlediska bezpečnosti požadujeme, aby se hašovací funkce chovala jako náhodné orákulum. Odtud se odvozují bezpečnostní vlastnosti.

- **Orákulum** nazýváme libovolný stroj (“podivuhodných vlastností”), který na základě vstupu odpovídá nějakým výstupem. Má pouze tu vlastnost, že na tentýž vstup odpovídá stejným výstupem.
- **Náhodné orákulum** je orákulum, které na nový vstup odpovídá **náhodným výběrem** výstupu z množiny výstupů.

Hašovací funkce (5)

Odolnost proti kolizi 1. a 2. řádu, bezkoliznost 1. a 2. řádu

Definice – Bezkoliznost 1. řádu

Hashovací funkce h je odolná proti kolizi 1. řádu, jestliže je výpočetně nezvládnutelné nalezení libovolných dvou různých (byť nesmyslných) zpráv M a M' tak, že $h(M) = h(M')$. Pokud se toto stane, říkáme, že jsme našli kolizi.

- Bezkoliznost se běžně využívá k digitálním podpisům.
- Nepodepisuje se přímo zpráva (často dlouhá), ale pouze její haš.
- Bezkoliznost zaručuje, že není možné nalézt dva dokumenty se stejnou haší. Proto můžeme podepisovat haš.

Definice – Bezkoliznost 2. řádu

Hašovací funkce h je odolná proti kolizi 2. řádu, jestliže pro jakýkoliv vzor x je výpočetně nezvládnutelné nalézt 2. vzor $y \neq x$ tak, že $h(x) = h(y)$.

Hašovací funkce (6)

Odolnost proti nalezení kolize 1. řádu

- Pokud se hašovací funkce $f : \{0, 1\}^{0\dots d} \rightarrow \{0, 1\}^n$ bude chovat jako náhodné orákulum $\Rightarrow$ složitost nalezení kolize s 50% pravděpodobností je $\approx 2^{\frac{n}{2}}$, namísto očekávaných 2^{n-1}
- Tato složitost nalezení kolize $2^{\frac{n}{2}}$ vyplývá z tzv. narozeninového paradoxu (viz dále).
- Například pro 160 bitový hašový kód bychom očekávali $2^{(160-1)}$ zpráv, paradoxně je to pouhých 2^{80} zpráv.
- U náhodného orákula neznáme rychlejší cestu nalézání kolizí.

Odolnost proti nalezení kolize 2. řádu

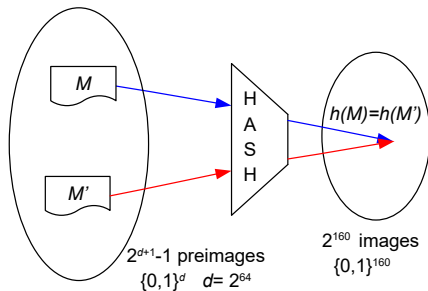
- Pokud se hašovací funkce $f : \{0, 1\}^{0\dots d} \rightarrow \{0, 1\}^n$ bude chovat jako náhodné orákulum $\Rightarrow$ složitost nalezení 2. vzoru je $\approx 2^n$.

Když víme, jak vzory nalézat **jednodušeji**, a to buď v případě kolizí 1. nebo 2. řádu $\Rightarrow$ hovoříme o prolomení hašovací funkce.

Hašovací funkce (7)

Pojem bezkoliznosti

Pokud máme hašovací funkci SHA-1 $\Rightarrow$ možných zpráv je mnoho ($2^0 + 2^1 + \dots + 2^d$), kde $d = 2^{64} - 1$, a hašovacích kódů pouze $2^{160} \Rightarrow$ existuje ohromné množství zpráv vedoucích na tentýž hašový kód, v průměru je to řádově 2^{d-159} .



Kolizí existuje velké množství, ale nalezení i jediné kolize je nad naše výpočetní možnosti, i když zohledníme narozeninový paradox.

Z principu definice hašovací funkce plyne:

- jak existence obrovského množství kolizí,
- ale stejně tak skutečnost, že nalezení těchto kolizí je pro nás příliš těžká a výpočetně neproveditelná úloha.

Hašovací funkce (8)

Narozeninový paradox

Otázka: Jak velká musí být množina náhodných zpráv, aby v ní s nezanedbatelnou pravděpodobností existovaly dvě různé zprávy se stejnou haší?

Narozeninový paradox říká, že pro n -bitovou hašovací funkci nastává kolize s cca 50% pravděpodobností v množině $2^{\frac{n}{2}}$ zpráv, namísto očekávaných 2^{n-1} .

Tvrzení

Mějme množinu M n různých koulí a provedme výběr k koulí po jedné s vrácením do množiny M . Potom pravděpodobnost, že vybereme některou kouli dvakrát nebo vícekrát, je

$$P(n, k) = 1 - \frac{n(n-1) \cdots (n-k+1)}{n^k}.$$

Pro $k = O(n^{\frac{1}{2}})$ a n velké je $P(n, k) \approx 1 - \exp(\frac{-k^2}{2n})$.

Důsledek

Pro n velké se ve výběru $k = (2n \ln 2)^{\frac{1}{2}} \approx n^{\frac{1}{2}}$ prvků z M s cca 50% pravděpodobností naleznou dva prvky shodné. Obecně pro hašovací funkce s b bitovým hašovacím kódem postačí zahašovat $n^{\frac{1}{2}} = (2^b)^{\frac{1}{2}} = 2^{\frac{b}{2}}$ zpráv, abychom přibližně s 50% pravděpodobností našli kolizi.

Platí: $P(365, 23) = 0.507$. $\Rightarrow$ 23 náhodně vybraných lidí postačí k tomu, aby se mezi nimi s cca 50% pravděpodobností našla dvojice, slavicí narozeniny tentýž den. U skupiny 30 lidí je $P(365, 30) = 0.706$.

Paradox, protože obvykle to vnímáme ve smyslu “kolik lidí je potřeba, aby se k danému člověku našel jiný, slavicí narozeniny ve stejný den”. V této podbízející se interpretaci hledáme jedny konkrétní narozeniny, nikoli “jakékoliv shodné” narozeniny. Oba přístupy odráží rozdíl mezi kolizí 1.řádu (libovolní 2 lidé) a 2. řádu (nalezení 2. člověka k danému).

Konstrukce moderních hašovacích funkcí

- U moderních hašovacích funkcí může být zpráva velmi dlouhá, například $d = 2^{64} - 1$ bitů $\Rightarrow$
- zpracovávání po částech, ne najednou. Také v komunikacích zprávu dostáváme po částech a nemůžeme ji z paměťových důvodů ukládat celou pro jednorázové zahašování $\Rightarrow$
- nutnost **hašování zprávy postupně po blocích** $\Rightarrow$
- nutnost **zarovnání vstupní zprávy na celistvý počet bloků** před hašováním.
- Zarovnání musí být bezkolizní a také proto musí umožňovat **jednoznačné odejmutí**.
- Dále budeme předpokládat, že máme hašovací funkci o n -bitovém hašovacím kódu a zprávu M zarovnanou na m -bitové bloky $M_1, \dots, M_N$.

Hašovací funkce (12)

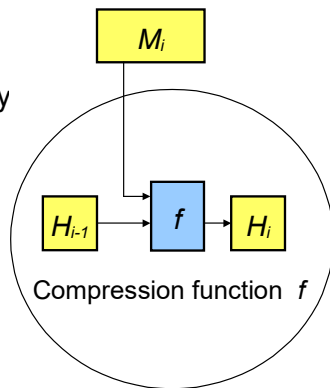
Zarovnání u hašovacích funkcí se definuje jako doplnění bitem 1 a poté potřebným počtem bitů 0 $\Rightarrow$ jednoznačné odejmutí doplňku.

Damgård-Merklovo zesílení – doplnění o délku původní zprávy

- Doplnění ilustruje obrázek (viz dále) pro blok délky $n = 512$, který je použit u SHA-1, SHA-256.
- Zpráva M je nejprve doplněna bitem 1 $\Rightarrow$ bity 0 (může jich být 0 — 511) tak, aby do celistvého násobku 512 bitů zbývalo ještě 64 bitů.
- Posledních 64 bitů je vyplněno 64 bitovou hodnotou # bitů původní zprávy M .
- Délka zprávy je součástí hašovacího procesu. 64 bitů vyjadřující délku zprávy umožňuje hašovat zprávy až do délky $d = 2^{64} - 1$ bitů.
- Informace o délce původní zprávy v paddingu eliminuje některé útoky.

Hašovací funkce (13)

- Současné prakticky používané hašovací funkce (kromě SHA-3) používají Damgård-Merklovův (DM) princip iterativní hašovací funkce s využitím kompresní funkce f .
- f zpracovává aktuální blok zprávy M_i a výsledkem je určitá hodnota, tzv. **kontext**, značíme H_i .
- Hodnota H_i nutně tvoří vstup do f v dalším kroku. f má tedy dva vstupy H_{i-1} a aktuální M_i . Výstupem je nové H_i .
- **Damgård-Merklova konstrukce**
Tato iterativní konstrukce je popsána vzorcem
$$H_i = f(H_{i-1}, M_i),$$
$$H_0 = IV.$$

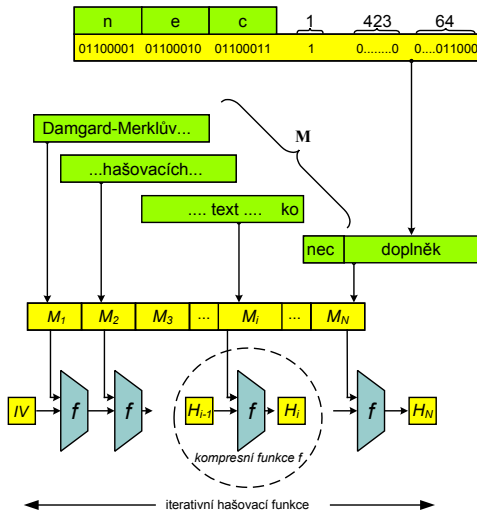


Hašovací funkce (14)

- Hašování probíhá postupně po jednotlivých blocích M_i v cyklu podle i od 1 do N .
- f v i -tém kroku ($i = 1, \dots, N$) zpracuje vždy dané H_{i-1} a blok M_i na nové H_i . Šíře H_i je většinou stejná jako šíře výstupního kódu.
- Počáteční hodnota kontextu H_0 se nazývá **inicializační hodnota – IV**. Je dodefinována jako konstanta daná v popisu každé iterativní hašovací funkce.
- f zpracovává širší vstup (H_{i-1}, M_i) na mnohem kratší $H_i \Rightarrow f$ je skutečně kompresní.
- Blok M_i se promítne do H_i , ale současně dochází ke ztrátě informace (šířka kontextu $H_0, H_1, \dots, H_i, \dots$ zůstává stále stejná).
- Kontextem bývá obvykle několik 16b nebo 32b slov.
- Zhašováním posledního bloku M_N dostáváme H_N , z něhož bereme buď celou délku nebo část jako výslednou haš. (SHA-1 má šířku kontextu 160 bitů a výslednou haš tvoří všech 160 bitů H_N).

Hašovací funkce (15)

Doplňování, kompresní funkce a iterativní hašovací funkce



Hašovací funkce (16)

Kolize kompresní funkce

Kolize kompresní funkce f spočívá v nalezení inicializační hodnoty H a dvou různých bloků B_1 a B_2 tak, že $f(H, B_1) = f(H, B_2)$.

Bezpečnost Damgård-Merklovy konstrukce

- Pokud je hašovací funkce bezkolizní, vyplývá odtud i bezkoliznost kompresní funkce.
- Důkaz je triviální pro zprávu tvořící jeden blok, ale obecně je-li kompresní funkce bezkolizní, obecně odtud nevyplývá, že je bezkolizní i hašovací funkce.
- U DM konstrukce to bylo dokázáno, že pokud je kompresní funkce bezkolizní $\Rightarrow$ bezkolizní také iterovaná hašovací funkce, konstruovaná z ní výše uvedeným postupem.
- Tato vlastnost DM konstrukce je bezpečnostním základem všech moderních hašovacích funkcí $\Rightarrow$ je možné se soustředit na nalezení kvalitní kompresní funkce.

Konstrukce kompresní funkce

- Kompresní funkce musí být **velmi robustní**, aby zajistila dokonalé promíchání bitů zprávy a jednosměrnost.
- Konstruovat kompresní funkce na bázi blokových šifer splňuje požadavek na **jednosměrnost** a požadavek na jejich chování jako **náhodné orákulum**.
- Kvalitní bloková šifra $E_k(x)$ je jednosměrná vzhledem k pevnému klíči $k \Rightarrow$ při znalosti jakékoliv množiny vstupů-výstupů, tj. OT-ŠT (x, y) , nemůžeme odtud určit (díky složitosti) klíč k .
- Přesněji, pro každé k je funkce $x \rightarrow E_k(x)$ jednosměrná.
- Odtud vyplývá možnost konstrukce kompresní funkce:

$$H_i = f(H_{i-1}, M_i,) = E_{M_i}(H_{i-1}).$$

- Vezmeme hašování krátké zprávy, která i se zarovnáním tvoří 1 blok. Máme $H_1 = E_{M_1}(IV) \Rightarrow$ z hodnoty hašového kódu H_1 nejsme schopni určit M_1 , čili máme zaručenu vlastnost jednosměrnosti.

Davies-Meyerova konstrukce kompresní funkce

- Davies-Meyerova konstrukce kompresní funkce pak zesiluje vlastnost jednosměrnosti ještě přičtením (XOR) vzoru před výstupem:

$$H_i = f(H_{i-1}, M_i) = E_{M_i}(H_{i-1}) \oplus H_{i-1}.$$

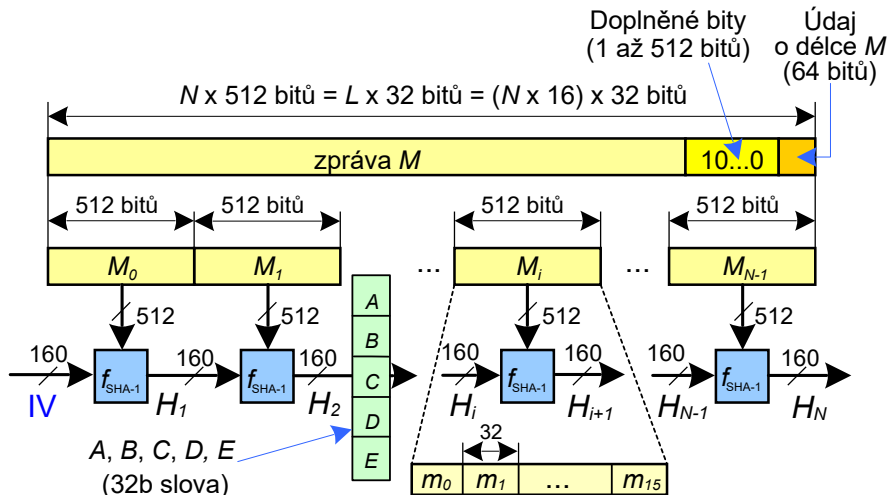
- Výstup je zde tedy navíc **maskován vstupem**, což ještě více ztěžuje případný zpětný chod.
- Blok zprávy M_i je obvykle větší než klíče blokových šifer $\Rightarrow$ se aplikuje bloková šifra několikrát za sebou – v **rundách**.
- Rundovní klíč je postupně čerpán z bloku M_i .

Algoritmus SHA-1

- Algoritmus SHA (Secure Hash Algorithm) byl zavedený NIST a publikovaný v normě FIPS 180 (1993). Revidovaná verze SHA-1 v normě FIPS–180–1 (1995).
- SHA-1 byla prolomená díky rostoucímu vypočetnímu výkonu a nalezení teoretických slabín.
- SHA-1 umožňuje zpracovat zprávu M s maximální délkou $2^{64} - 1$ bitů a vrací výstupní hašový kód s délkou **160b**. Vstupní zpráva M je zpracována po blocích M_i s velikostí 512b.
- Z obrázku na následujícím slidu můžeme vidět způsob předzpracování zprávy, kde šířka kontextu, tj. ve velikosti hašového kódu, je 160b.
- Prostupující kontext se skládá z 5×32 bitových slov A, B, C, D, E .

SHA-x (2)

Kompresní funkce SHA-1 – zpracování bloku zprávy M_i



SHA-x (3)

- Na obrázku na dalším slidu vidíme zapojení kompresní funkce SHA-1, která zpracovává jeden 512b blok M_i zprávy M .
- Blok zprávy M_i je zpracováván v 4×20 rundách. 160 bitový kontext (začínáme s inicializačním vektorem IV) je postupně “zašifrováván” 32b slovem m_0, m_1 až m_{15} pro rundy 0, 1, ..., 15 a pro rundy 16, 17, ..., 79 32b slovem w_i , pro které platí:

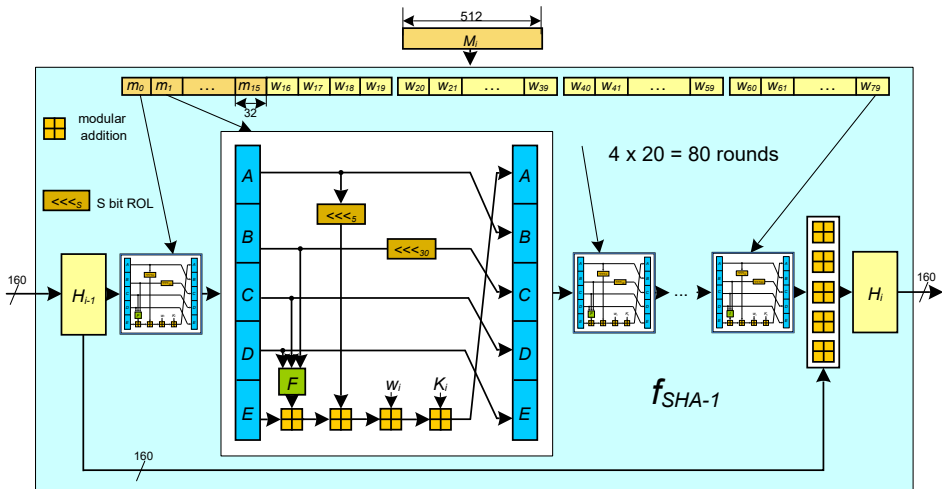
$$w_i = \lll_1 (w_{i-16} \oplus w_{i-14} \oplus w_{i-8} \oplus w_{i-3}), \text{ kde } i = 16, 17, \dots, 79.$$

- Na místě funkce F v obrázku se po 20 rundách střídají 4 různé funkce (F, G, H, I) a v každé rundě se využívá jiná konstanta K_i . Pro výpočet hodnot logických funkcí F, G, H, I platí:

$F(b, c, d)$	$(b \wedge c) \vee (\bar{b} \wedge d)$
$G(b, c, d)$	$b \oplus c \oplus d$
$H(b, c, d)$	$(b \wedge c) \vee (b \wedge d) \vee (c \wedge d)$
$I(b, c, d)$	$b \oplus c \oplus d$

SHA-x (4)

Kompresní funkce SHA-1 – zpracování bloku zprávy M_i



SHA-x (5)

- Po 80 rundách je i u této kompresní funkce k výsledku přičten modulo 2^{32} původní kontext H_{i-1} (podle Davies-Meyerovy konstrukce). Přičítání je prováděno po 32b slovech.
- Po zpracování bloku M_N máme v registrech A, B, C, D, E výslednou 160b haš H_N .
- Pro konstantu K_i platí:

Runda	Hexadecimální K_i	Celočíselná část
$0 \leq i \leq 19$	$K_i=5A927999$	$(2^{30} \times \sqrt{2})$
$20 \leq i \leq 39$	$K_i=6ED9EBA1$	$(2^{30} \times \sqrt{3})$
$40 \leq i \leq 59$	$K_i=8F1BBCDC$	$(2^{30} \times \sqrt{5})$
$60 \leq i \leq 79$	$K_i=CA62C1D6$	$(2^{30} \times \sqrt{10})$

- Inicializační vektor IV má hodnoty (hexadecimálně):
 $A= 67452301$, $B= EFCDA8B9$, $C= 98BADCFE$
 $D= 10325476$, $E= C3D2E1F0$

SHA-x (6)

- Inovací standardu FIPS 180-1 na FIPS 180-2 byly zavedeny 3 nové hašovací algoritmy SHA-2: SHA-256, SHA-384 a SHA-512.

Základní vlastnosti hašovacích funkcí SHA-x

	SHA-1	SHA-256	SHA-384	SHA-512
Délka haš. kódu	160	256	384	512
Délka zprávy	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Velikost bloku	512	512	1024	1024
Velikost slova	32	32	64	64
# rund f	80	64/80	80	80
Bezpečnost v bitech	80	128	192	256

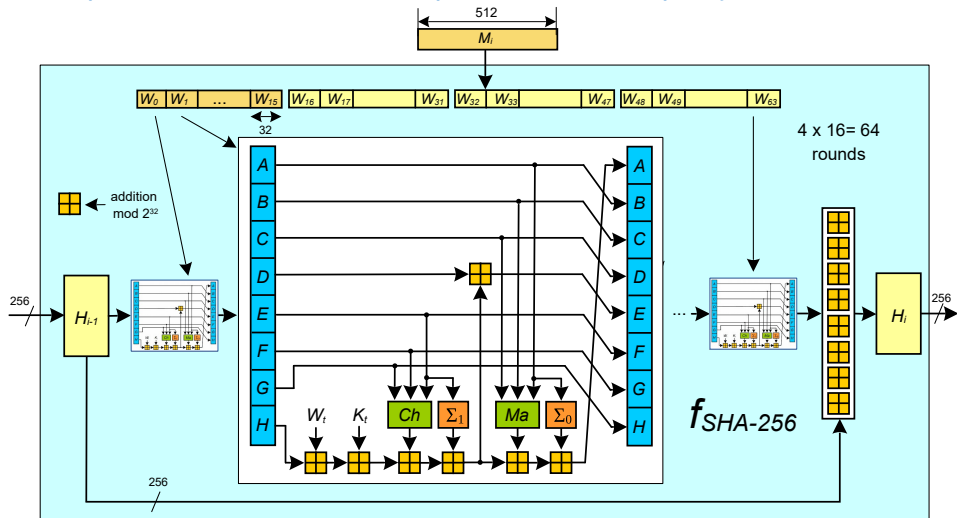
- Nejvýznamnější rozdíly jsou v délce hašového kódu, který určuje odolnost hašového kódu vůči nalezení kolizí 1. a 2. řádu.
- Na druhé straně struktura hašovacích funkcí (kompresních funkcí) je téměř stejná.

SHA-256

- Prolomení algoritmu SHA-1.
- SHA-256 je novější, bezpečná kryptografická hašovací funkce. Od roku 2000 jako nová generace funkcí SHA (2002 standard FIPS).
- SHA-256 má stejnou základní strukturu a používá stejné typy operací modulární aritmetiky a logických binárních operací jako algoritmus SHA-1, je ale bezpečný.
- Algoritmus SHA-256 generuje 256bitovou hašovací hodnotu z bloků zpráv o velikosti 512 bitů s paddingem totožným s pro SHA-1.
- Původní velikost zprávy je až $2^{64} - 1$ bit.
- SHA-256 interně počítá 256bitový kontext (kvůli bezpečnosti). Výstupní hash lze zkrátit na 196/128 bitů. Zkrácený SHA-256 je praktičtější a není předmětem žádných známých útoků.

SHA-x (8)

Kompresní funkce SHA-256 – zpracování bloku zprávy M_i



SHA-x (9)

- Kompresní funkce SHA-256 zpracovává jeden 512b blok M_i zprávy M . Blok zprávy M_i je zpracováván v 4×16 rundách. 256 bitový kontext (na začátku je IV) je postupně “zašifrováván” 32b slovem W_0, W_1 až W_{63} (64 rund).
- Pro rundy $t = 0, 1, \dots, 15$ je $W_t = m_t$ (rozdělení bloku zprávy M_i).
- Pro rundy $t = 16, 17, \dots, 79$ je to 32b slovo W_t , pro které platí:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

Pro výpočet hodnot funkcí $Ch, Ma, \Sigma_0, \Sigma_1, \sigma_0, \sigma_1$ platí:

$Ch(x, y, z)$	$(x \wedge y) \oplus (\bar{x} \wedge z)$
$Ma(x, y, z)$	$(x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$
$\Sigma_0(x)$	$\ggg_2(x) \oplus \ggg_{13}(x) \oplus \ggg_{22}(x)$
$\Sigma_1(x)$	$\ggg_6(x) \oplus \ggg_{11}(x) \oplus \ggg_{25}(x)$
$\sigma_0(x)$	$\ggg_7(x) \oplus \ggg_{18}(x) \oplus \ggg_3(x)$
$\sigma_1(x)$	$\ggg_{17}(x) \oplus \ggg_{19}(x) \oplus \ggg_{10}(x)$

Kryptografické využití hašovacích funkcí

- Kontrola integrity (kontrola shody velkých souborů dat).
- Ukládání a kontrola přihlašovacích hesel. Pro vyloučení slovníkových útoků se používá ještě metoda solení. S otiskem hesla se vygeneruje náhodný řetězec "sůl", který je dohromady hašován s heslem. Databáze obsahuje dvojici: (sůl, haš(heslo, sůl)).
- Jednoznačná identifikace dat (jednoznačná reprezentace vzoru, digitální otisk dat, jednoznačný identifikátor dat, to vše zejména pro digitální podpisy).
- Prokazování znalosti.
- Autentizace původu dat.
- Pseudonáhodné generátory, odvozování klíčů.

Algoritmus HMAC

- Klíčované hašové autentizační kódy zpráv HMAC zpracovávají hašováním nejen zprávu M , ale spolu s ní i nějaký tajný klíč K .
- Jsou proto podobné autentizačnímu kódu zprávy MAC, ale místo blokové šifry využívají hašovací funkci.
- Používají se jak k nepadělatelnému zabezpečení zpráv, tak k autentizaci (prokazováním znalosti tajného klíče K).
- HMAC je obecná konstrukce, která využívá obecnou hašovací funkci. Podle toho, jakou hašovací funkci používá konkrétně, se označuje výsledek, například HMAC-SHA-1(M , K) používá SHA-1, M je zpráva a K je tajný klíč.
- HMAC je definován ve standardu FIPS-PUB-198. Definice závisí na tom, kolik bajtů má blok kompresní funkce. Například u SHA-1 je 64B (512b), u SHA-384 a SHA-512 to je 128B (1024b).

Algoritmus HMAC

- Definujeme konstantní řetězce *ipad* jako řetězec $b/8$ bajtů s hodnotou 0×36 (0011 0110) a *opad* jako řetězec $b/8$ bajtů s hodnotou $0 \times 5C$ (0101 1100), kde b je velikost bloku v bitech.
- Klíč K v případě, že $\log_2 K < b$, doplníme bity 0 vlevo od MSB bitu klíče do délky b -bitů a označíme ho K^+ .
- Definujeme hodnotu $HMAC_K(M)$ jako

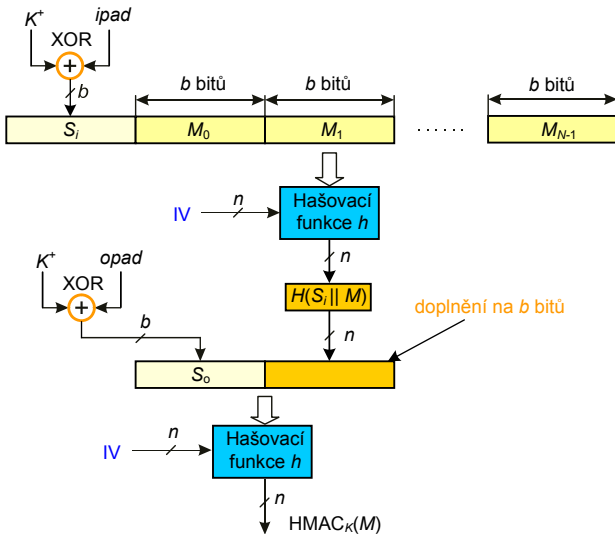
$$HMAC_K(M) = H\left((K^+ \oplus opad) \| H((K^+ \oplus ipad) \| M)\right),$$

kde $\|$ označuje zřetězení.

- Operace “doplnění na b bitů” (viz. obrázek) je prováděna stejným způsobem jako doplnění zprávy u hašovací funkce do celého bloku.
- Doplnění bude vždy jenom do 1 bloku, protože při použití SHA-x platí: *délka hašového kódu* $<$ *velikost bloku* $- 64$ [b].

HMAC (3)

Struktura algoritmu HMAC



Nepadělatelný integritní kód, autentizace původu dat

- Zabezpečovací kód $HMAC_K(M)$, pokud je připojen za zprávu M , detekuje neúmyslnou chybu při jejím přenosu.
- Zabraňuje útočnickovi změnit zprávu a současně změnit HMAC, protože bez znalosti klíče K nelze nový HMAC vypočítat $\Rightarrow$
- HMAC – nepadělatelný integritní kód (neposkytuje samotná haš).
- Pro komunikujícího partnera je správný HMAC autentizací původu dat, protože odesílatel musel znát hodnotu tajného klíče K .

Průkaz znalosti při autentizaci entit

- HMAC může být použit jako průkaz znalosti tajného sdíleného tajemství klíč K při autentizaci entit.
- Dotazovatel odešle náhodnou výzvu (řetězec) *challenge* a od prokazovatele obdrží odpověď *response* = $HMAC_K(challenge)$ $\Rightarrow$
- prokazovatel zná tajný klíč K . Útočník na komunikačním kanálu z hodnoty response klíč K nemůže odvodit.

Kryptografie a bezpečnost

6. RSA, DSA, El-Gamal

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLÓGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- RSA
- DSA
- El-Gamal

Úvod

- Zabezpečení utajené komunikace v síti $\Rightarrow$ každá komunikující dvojice musí používat šifrovací klíč.
- U symetrické šifry pokud je šifrovací klíč známý $\Rightarrow$ dešifrovací klíč lze generovat s použitím malého počtu operací.
- Šifrovací systém veřejného klíče (VK) je řešení problému s přidělováním klíče pro utajenou komunikaci.
- Šifrovací systém VK má veřejný klíč VK pro šifrování a soukromý klíč SK pro dešifrování.
 - ▶ Vypočítat SK při znalosti VK je výpočetně neschůdné.
 - ▶ Použitím VK je zřízena utajená komunikace v síti s více subjekty.
 - ▶ Pokud má Alice VK_A a soukromý klíč $SK_A \Rightarrow$ kdokoli může Alici poslat zprávu zašifrovanou VK_A , ale **POUZE** Alice umí zprávu dešifrovat, protože jen ona zná SK_A .
 - ▶ Každý subjekt má svůj vlastní VK a SK pro daný šifrovací systém.
 - ▶ Každý subjekt vlastní tajnou informaci – SK šifrovacího systému.

RSA (2)

- Seznam klíčů $VK_1, VK_2, \dots, VK_n$ je veřejný.
- Subjekt 1 vysílá zprávu m subjektu 2:
 - ▶ Zpráva $\rightarrow$ blok (obvykle 1) určité délky; bloku OT m odpovídá blok ŠT, písmena $\rightarrow$ numerické ekvivaleny.
 - ▶ Subjekt 2 s použitím dešifrovací transformace dešifruje blok ŠT.
- **Podmínka:** dešifrovací transformace nemůže být nalezena v rozumném čase někým jiným než subjektem 2 $\Rightarrow$ neautorizované subjekty komunikace nemůžou dešifrovat zprávu bez znalosti soukromého klíče.

Princip RSA šifrovacího systému

- Uveden Rivestem, Shamirem a Adlemanem v roce 1970.
- RSA je šifrovací systém VK a je založený na modulárním umocňování.
- Dvojice (e, n) je VK klíče; e - exponent a n - modul.
- $n \rightarrow$ součin dvou prvočísel p a q , tj. $n = pq$ a $\gcd(e, \Phi(n)) = 1$.

- Zašifrování OT: písmena $\rightarrow$ numerické ekvivalenty, vytváříme bloky s největší možnou velikostí (se sudým počtem číslic).
- Pro zašifrování zprávy m na ŠT c použijeme vztah:

$$E(m) = c = |m^e|_n, \quad 0 < c < n.$$

- K dešifrování požadujeme znalost inverze d čísla e modulo $\Phi(n)$, $\gcd(e, \Phi(n)) = 1 \Rightarrow$ inverze existuje. Pro dešifrování bloku c platí:

$$D(c) = |c^d|_n = |m^{ed}|_n = |m^{k\Phi(n)+1}|_n = |(m^{\Phi(n)})^k m|_n = |m|_n,$$

kde $ed = k\Phi(n) + 1$ pro nějaké celé číslo k ($|ed|_{\Phi(n)} = 1$) a z Eulerovy věty platí $|p^{\Phi(n)}|_n = 1$, kde $\gcd(p, n) = 1$.

Pravděpodobnost, že m a n nejsou nesoudělná je extrémně malá.
Dvojice (d, n) je dešifrovací klíč - tajná část klíče SK .

Příklad

- Šifrovací modul je součinem dvou prvočísel 43 a 59. Potom dostáváme $n = 43 \cdot 59 = 2537$ jako modul.
- $e = 13$ je exponent, kde platí $\gcd(e, \Phi(n)) = \gcd(13, 42 \cdot 58) = 1$.
- Dále platí $\Phi(2537) = (43 - 1) \cdot (59 - 1) = 42 \cdot 58 = 2436$.
- Pro zašifrování zprávy

PUBLIC KEY CRYPTOGRAPHY,

- převedeme OT do číselných ekvivalentů písmen textu $\Rightarrow$ vytvoříme bloky o délce 4 číslic (n je 4ciferné!) a dostáváme:
1520 0111 0802 1004 2402 1724 1519 1406 1700 1507 2423,
Písmeno X = 23 je výplň (padding).
- Pro šifrování bloku OT do bloku ŠT použijme vztah $c = |m^{13}|_{2537}$.
Šifrováním prvního bloku OT 1520 dostáváme blok ŠT

$$c = |(1520)^{13}|_{2537} = 95.$$

- Zašifrováním všech bloků OT dostáváme:
0095 1648 1410 1299 0811 2333 2132 0370 1185 1457 1084.
- Pro dešifrování zprávy, která byla zašifrována RSA šifrou, musíme najít inverzi $e = |13^{-1}|_{\Phi(n)}$, kde $\Phi(n) = \Phi(2537) = 2436$.
- S použitím Euklidova algoritmu získáme číslo $d = 937$, které je multiplikativní inverzí čísla 13 modulo 2436.
- K dešifrování bloku c ŠT použijeme vztah:

$$m = |c^{937}|_{2537}, \quad 0 \leq m \leq 2537,$$

který platí, protože

$$|c^{937}|_{2537} = |(m^{13})^{937}|_{2537} = |m \cdot (m^{2436})^5|_{2537} = m,$$

kde jsme použili Eulerovu větu

$$|m^{\Phi(2537)}|_{2537} = |m^{2436}|_{2537} = 1,$$

když platí $\gcd(m, 2537) = 1$. Pokud to není splněno viz přednášky MI-KRY.

Definice RSA

- Necht' p a q jsou prvočísla.
- Vypočítáme $n = pq$, $\Phi(n) = (p - 1)(q - 1)$.
- Zvolíme e , $1 < e < n$, $\gcd(e, \Phi(n)) = 1$ a spočítáme $d = |e^{-1}|_{\Phi(n)}$.
- Dvojici $VK = (n, e)$ prohlásíme za veřejný klíč (a zveřejníme), dvojici $SK = (n, d)$ prohlásíme za soukromý klíč.

Postup pro generování VK a SK :

- Každý subjekt najde 2 velká náhodná lichá čísla p a q se 340 dekadickými číslicemi.
- Z věty o prvočíslech plyne, že pravděpodobnost toho, že takto vybraná čísla jsou prvočísla, $\approx 2 / \log(10^{340})$.
- Pro nalezení prvočísla potřebujeme v průměru $1 / (2 / \log(10^{340})) \approx 400$ testů takových celých čísel.

- Ke zjištění, jestli jsou takto náhodně vybraná lichá celá čísla prvočísla, použijeme Rabinův-Millerův pravděpodobnostní test.
- 100číslicové celé liché číslo je testováno Rabin-Millerovým testem pro 100 “svědků”.
- Pravděpodobnost, že testované číslo je složené je $\approx 10^{-60}$.
- Každý subjekt provádí daný výpočet pouze dvakrát.
- Jakmile jsou prvočísla p a q nalezena $\Rightarrow$ je vypočítán šifrovací exponent e (platí $\gcd(e, \Phi(pq)) = 1$).
- Doporučení: zvolit e jako nějaké prvočíslu $> p$ a q .
- Pokud $2^e > n = pq \Rightarrow$ znemožnění odkrytí bloku otevřeného textu m následným jednoduchým odmocňováním celého čísla c , kde $c = |m^e|_n, 0 < c < n$, bez provedení redukce modulo n .
- Podmínka $2^e > n$ zaručí, že každý blok otevřeného textu m je zašifrovaný umocněním a následnou redukcí modulo n .

Bezpečnost RSA

- Modulární umocňování potřebné k šifrování zprávy s použitím RSA může být provedeno při VK a m o velikosti ≈ 680 dekadických číslic v řádech milisekund a méně.
- Se znalostí p a q ($\Phi(n) = \Phi(pq) = (q - 1)(p - 1)$) a s použitím Euklidova algoritmu lze najít dešifrovací klíč d , kde $|de|_{\Phi(n)} = 1$,
- K objasnění proč znalost šifrovacího klíče (e, n) , který je veřejný, nevede lehce k nalezení dešifrovacího klíče (d, n) je důležité si uvědomit, že k nalezení dešifrovacího klíče d jako inverzi šifrovacího klíče e modulo $\Phi(n)$ vyžaduje znalost hodnot p a q , které umožní snadný výpočet $\Phi(pq) = (p - 1)(q - 1)$.

V případě, kdy hodnoty p a q jsou neznámé, je nalezení $\Phi(n)$ podobně složité jako faktorizace celého čísla n .

Problem faktorizace a RSA (1)

- Pokud p a q jsou 100číslicová prvočísla $\Rightarrow n$ je 200číslicové.
- Nejrychlejší známé algoritmy pro faktorizaci potřebují ≈ 250 roků počítačového času k faktorizaci takových celých čísel.
- Naopak, pokud známe d , ale neznáme $\Phi(n)$, je možné lehce faktorizovat n , protože víme, že $ed - 1$ je násobkem $\Phi(n)$.
- Pro takovou úlohu existují speciální algoritmy faktorizace celého čísla n s použitím nějakého násobku $\Phi(n)$.
- Dosud nebylo prokázáno dešifrování zprávy zašifrované s použitím RSA bez faktorizace n !
- $\Rightarrow$ pokud neexistuje žádná metoda pro dešifrování RSA bez provedení faktorizace modulu n je RSA šifrovací systém metodou, který používá faktorizaci pro její zabezpečení!

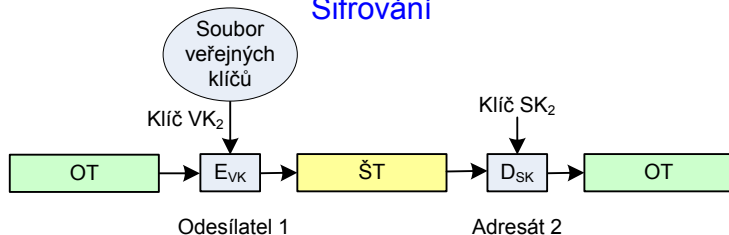
Problem faktorizace a RSA (2)

Výpočetní náročnost je tím větší, čím větší je modul.

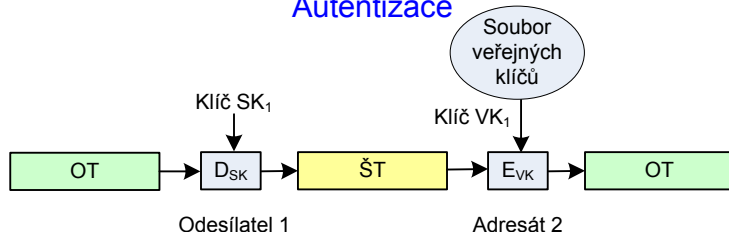
- Zprávy šifrované s použitím RSA systému se stávají zranitelné proti útokům v tom okamžiku, když se faktorizace n stane proveditelnou v “reálných podmínkách”!
- Znamená to zvýšenou pozornost při výběru a používání prvočísel p a q k zajištění ochrany utajení zpráv, které mají být utajeny na desítky a stovky let.
- Ochrana proti speciálním, rychlým technikám pro faktorizaci $n = pq$. Například obě hodnoty $p - 1$ a $q - 1$ by měly mít velký prvočíselný faktor, tedy $\gcd(p - 1, q - 1)$ by mělo být malé a p a q se musí dostatečně lišit. Například FIPS 186-4 požaduje, aby $|p - q| > 2^{\frac{\text{delka } n}{2} - 100}$.

Schémata kryptografie veřejného klíče

Šifrování



Autentizace



Digitální podpis a RSA (1)

- Šifrovací systém RSA lze použít pro vyslání podepsané zprávy.
- Při použití podpisu se příjemce zprávy může ujistit, že zpráva přišla od oprávněného odesílatele, a že tomu tak je na základě nestranného a objektivního testu.
- Takové ověření je potřebné pro elektronickou poštu, elektronické bankovníctví, elektronický obchod atd.

Princip

- Necht' subjekt 1 vysílá podepsanou zprávu m subjektu 2.
- Subjekt 1 spočítá pro zprávu m OT (podpis)

$$S = D_{SK_1}(m) = |m^{d_1}|_{n_1},$$

kde $SK_1 = (d_1, n_1)$ je soukromý dešifrovací klíč pro subjekt 1.

- Když $n_2 > n_1$, kde $VK_2 = (e_2, n_2)$ je veřejný šifrovací klíč pro subjekt 2, subjekt 1 zašifruje S pomocí vztahu

$$c = E_{VK_2}(S) = |S^{e_2}|_{n_2}, \quad 0 < c < n_2.$$

Digitální podpis a RSA (2)

- Když $n_2 < n_1$ subjekt 1 rozdělí S do bloků o velikosti menší než n_2 a zašifruje každý blok s použitím šifrovací transformace E_{VK_2} .
- Pro dešifrování subjekt 2 nejdříve použije soukromou dešifrovací transformaci D_{SK_2} k získání S , protože

$$D_{SK_2}(c) = D_{SK_2}(E_{VK_2}(S)) = S.$$

- K nalezení OT m předpokládejme, že byl vyslán subjektem 1, subjekt 2 dále použije veřejnou šifrovací transformaci E_{VK_1} , protože

$$E_{VK_1}(S) = E_{VK_1}(D_{SK_1}(m)) = m.$$

Zde jsme použili identitu $E_{VK_1}(D_{SK_1}(m)) = m$, která plyne z faktu, že

$$E_{VK_1}(D_{SK_1}(m)) = |(m^{d_1})^{e_1}|_{n_1} = |m^{d_1 e_1}|_{n_1} = m,$$

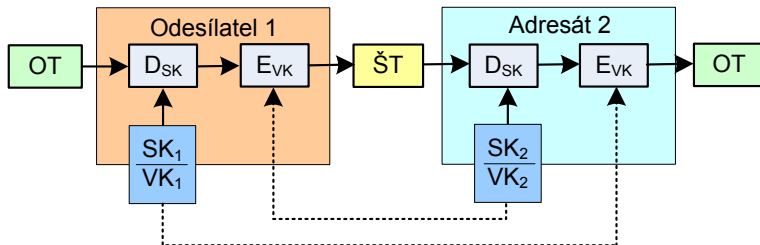
protože

$$|d_1 e_1|_{\phi(n_1)} = 1.$$

Digitální podpis a RSA (3)

- Kombinace OT m a podepsané verze S přesvědčí subjekt 2, že zpráva byla vyslána subjektem 1.
- Také subjekt 1 nemůže odepřít, že on vyslal danou zprávu, protože žádný jiný subjekt než 1 nemůže generovat podepsanou zprávu S z originálního textu zprávy m .

Digitální podpis



Urychlení šifrování

- Pro urychlení šifrování je normou doporučena množina šifrovacích exponentů e .
- Exponenty se vyznačují malou Hammingovou váhou $\Rightarrow$
- šifrování probíhá rychle v několika krocích, viz modulární umocňování.
- Například $e = 11_2, 1011_2, 10001_2, 2^{16} + 1, \dots$

Urychlení dešifrování

- Pro urychlení dešifrování se využívá rozklad pomocí Čínské věty o zbytcích - RSA-CRT
- Na základě tohoto rozkladu se při dešifrování počítá s čísly poloviční délky $\Rightarrow$
- zrychlení 4 až 8 násobné oproti původnímu dešifrovacímu výpočtu.

Definice RSA-CRT

- Necht' p a q jsou prvočísla.
- Vypočítáme $n = pq$, $\Phi(n) = (p - 1)(q - 1)$.
- Zvolíme e , $1 < e < n$, $\gcd(e, \Phi(n)) = 1$ a spočítáme $d = |e^{-1}|_{\Phi(n)}$.
- Vypočítáme $d_p = |d|_{p-1}$, $d_q = |d|_{q-1}$, $q_{inv} = |q^{-1}|_p$.
- Dvojici $VK = (n, e)$ prohlásíme za veřejný klíč (a zveřejníme),
pětici $SK = (p, q, d_p, d_q, q_{inv})$ prohlásíme za soukromý klíč.

Šifrování a dešifrování

- Pro šifrování platí stejný vztah jako pro RSA: $c = |m^e|_n$.
- Pro dešifrování v RSA-CRT musí platit pro d_p a d_q následující kongruence:

$$ed_p \equiv 1 \pmod{p-1}$$

$$ed_q \equiv 1 \pmod{q-1}$$

Dešifrování

1 Vypočteme

$$m_1 = |c^{d_p}|_p$$

$$m_2 = |c^{d_q}|_q$$

2 Vypočteme

$$h = |q_{inv}(m_1 - m_2)|_p$$

3 Vypočteme

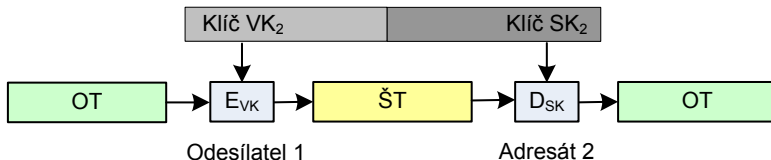
$$m = m_2 + hq$$

- Krok 1 je výpočetně nejnáročnější. Počítáme ale s polovičními délkami čísel než v případě RSA.
- Výpočet m_1 a m_2 je datově nezávislý a lze ho provádět paralelně.
- V kroku 2 je nenáročné násobení rozdílu m_1 a m_2 s předpočítanou konstantou q_{inv} a následná redukce modulo p .
- Poslední krok představuje nejméně náročné násobení a sčítání.

Asymetrická a symetrická kryptografie (1)

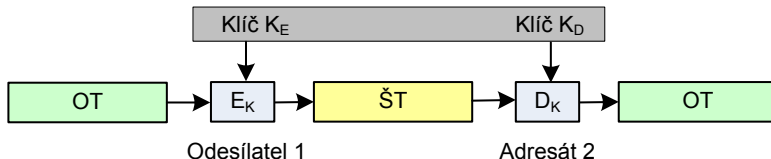
Asymetrické šifrování

SK_2 z VK_2 nelze schůdně vypočítat



Symetrické šifrování

K_E a K_D stejné nebo snadno převoditelné



Odesílatel 1 může i dešifrovat

Digitální podpis (1)

Digitální podpis je obvykle formou asymetrického kryptografického schématu

- Soukromý klíč – podepsání
- Veřejný klíč – ověření

Vlastnosti digitálního podpisu

- Nezfalšovatelnost, autentizace – podpis se nedá napodobit jiným subjektem než podepisujícím
 - ▶ Ověřitelnost – příjemce dokumentu musí být schopen ověřit, že podpis je platný
- Integrita – podepsaná zpráva se nedá změnit, aniž by se zneplatnil podpis
- Nepopíratelnost – podepisující nesmí mít později možnost popřít, že dokument podepsal

Digitální podpis (2)

Další vlastnosti

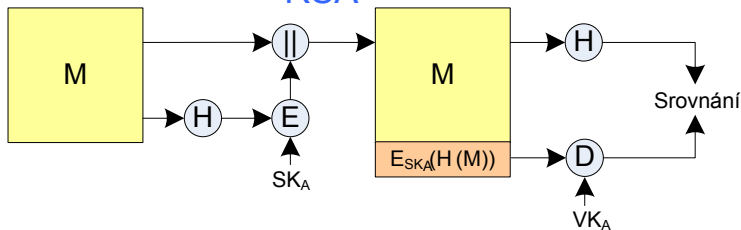
- Skupina bitů, jejichž hodnoty závisí na celém podepisovaném dokumentu
- Využívá informaci, kterou zná jen podepisující (soukromý klíč)
- Implementace digitálního podpisu by měla být snadná, ale...
- falšování digitálního podpisu by mělo být výpočetně obtížné
 - ▶ neschůdné vyrobit falešný podpis pro existující zprávu
 - ▶ neschůdné vyrobit falešnou zprávu pro existující podpis

Kategorie digitálních podpisů

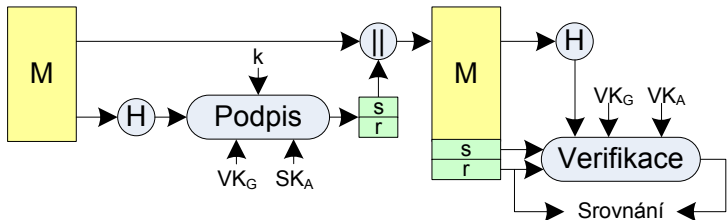
- Přímé digitální podpisy (direct digital signature)
 - ▶ Mezi dvěma subjekty, příjemce zná VK odesílatele
 - ▶ Problém s popiratelností $\Rightarrow$ pokud odesílatel popře podepsání zprávy, příjemce ho nemůže usvědčit (není nikdo třetí, kdo by svědčil proti odesílateli)
- Verifikované digitální podpisy (arbitrated digital signature)
 - ▶ Využívá důvěryhodnou třetí stranu (arbitra), který ověřuje podpisy všech zpráv

Postupy v digitálních podpisech

RSA



DSS



Úvod

- El Gamal - Taher ElGamal
- Algoritmus pro kryptografii s veřejným klíčem
- Založen na Diffie-Hellmanově výměně klíče, resp. problému diskretního logaritmu
- Podobně jako RSA i El Gamal umožňuje šifrování i digitální podpis

Diffie-Hellman připomenutí

- Alice (A) a Bob (B) si veřejně dohodnou prvočíslo m a bázi a , $1 < a < m$ (přesněji: grupu řádu $m - 1$)
- A si náhodně zvolí číslo k_A takové, že $0 < k_A < m$ a $\gcd(k_A, m - 1) = 1$, spočítá $y_A = |a^{k_A}|_m$ a odešle ho B.
- B si náhodně zvolí číslo k_B takové, že $0 < k_B < m$ a $\gcd(k_B, m - 1) = 1$, spočítá $y_B = |a^{k_B}|_m$ a odešle ho A.
- A i B spočítají sdílený klíč
$$K = |y_B^{k_A}|_m = |(a^{k_B})^{k_A}|_m = |a^{k_A \cdot k_B}|_m = |(a^{k_A})^{k_B}|_m = |y_A^{k_B}|_m.$$
- Útočník nedokáže ani z $|a^{k_A}|_m$ ani z $|a^{k_B}|_m$ spočítat $K = |a^{k_A \cdot k_B}|_m$ (tzv. Diffie-Hellmanův problém, DHP)
- DHP není složitější než problém diskretního logaritmu (DLP):
Kdyby útočník uměl vyřešit DLP, dokázal by z $|a^{k_A}|_m$ spočítat k_A , následně k_B z $|a^{k_B}|_m$, a pak triviálně spočítat $K = |a^{k_A \cdot k_B}|_m$
- Je DHP jednodušší než DLP? Nevíme jistě, ale zdá se, že ne.

El Gamal - příprava klíče

- El Gamal vzniká úpravou DH:
- Alice (A) **zvolí** číslo g a prvočíslo m , $1 < g < m$ (přesněji: grupu řádu $m - 1$ a její generátor g).
- A si náhodně zvolí číslo k_A (soukromý klíč) takové, že $0 < k_A < m$, spočítá $y_A = |g^{k_A}|_m$.
- **A zveřejní uspořádanou trojici (m, g, y_A) jako svůj veřejný klíč. k_A je jejím soukromým klíčem.**

El Gamal - šifrování

- Bob chce Alici poslat zprávu p :
- B si náhodně zvolí číslo k_B takové, že $0 < k_B < m$, spočítá $y_B = |g^{k_B}|_m$
- **B spočítá sdílený klíč**
 $K = |y_A^{k_B}|_m = |(g^{k_A})^{k_B}|_m = |g^{k_A \cdot k_B}|_m = |(g^{k_B})^{k_A}|_m = |y_B^{k_A}|_m$
- **B zašifruje zprávu p pomocí vztahu $c = |p \cdot K|_m$**
- **B odešle Alici uspořádanou dvojici (y_B, c) .**

El Gamal - šifrování - příklad

- $m = 2543, g = 5, y_A = |g^{k_A}|_m = 505$ (pro $k_A = 10$, které ale B nezná).
- $k_B = 123$ (náhodná volba)
 $\rightarrow y_B = |g^{k_B}|_{2543} = 308, K = |y_A^{k_B}|_{2543} = 1883$!! Zjednodušení pro demonstrační účely, v praxi by to byla hrubá chyba používat stále stejné y_B !!
- $p = \text{"ELGAMAL RULES"} \rightarrow 0511, 0701, 1301, 1118, 2111, 0519$
- $c = |p \cdot K|_{2543} \rightarrow 0959, 0166, 0874, 2133, 0304, 0765$
- B odesílá A dvojice $(308, 959), (308, 166), (308, 874)...$

El Gamal - dešifrování

- Alice dostala od Boba zprávu (y_B, c)
- A si spočítá sdílený klíč $K = |y_B^{k_A}|_m = |(g^{k_B})^{k_A}|_m = |g^{k_A \cdot k_B}|_m$.
- A si spočítá $|K^{-1}|_m$ (Euklidův rozšířený algoritmus)
- A dešifruje zprávu $p = |c \cdot K^{-1}|_m = |p \cdot K \cdot K^{-1}|_m = |p|_m = p$

El Gamal - dešifrování - příklad

- Alice dostala od Boba zprávu $(308, 959)$, tzn $y_B = 308, c = 959$
- A si spočítá sdílený klíč $K = |y_B^{k_A}|_m = |308^{10}|_{2543} = 1883$.
- A si spočítá $|1883^{-1}|_{2543} = 1337$ (Euklidův rozšířený algoritmus)
- A dešifruje zprávu $p = |959 \cdot 1337|_{2543} = 511 \rightarrow \text{"EL"}$
- Obdobně pro další bloky zprávy

Kryptografie a bezpečnost

7. Úvod do zabezpečení pomocí čipových karet

Ing. Jiří Buček, Ph.D.



**FACULTY
OF INFORMATION
TECHNOLOGY
CTU IN PRAGUE**

Katedra informační bezpečnosti

- Čipová (smart) karta – obvykle plastová kartička obsahující integrovaný obvod
- Nosič informace spojené s nějakým užitečným účelem – token
 - ▶ bezpečná identifikace, autentizace
 - ▶ uchování, zpracování dat
- Odolnost proti padělání
- **Vícefaktorová autentizace** – druhý faktor
 - ▶ Co víme – znalost (heslo, PIN)
 - ▶ **Co máme** – vlastnictví (ČK)
 - ▶ Co jsme – inherence, biometrie (prst, duhovka, sítnice, obličej, hlas...)
- Odvrácená stránka — množství různých karet (životní cyklus karty, škrabka na nádobí)

- Předplacená telefonní karta (minulé tisíciletí?)
- SIM – Subscriber Identification Module (GSM)
 - ▶ UICC – Universal Integrated Circuit Card (UMTS)
 - ▶ R-UIM – Removable User Identity Module (CDMA)
- VISA/MC/EMV — bankovní platební karta
- Elektronická peněženka (drobné nákupy, platby za služby)
- Elektronická jízdenka, vstupenka
- Průkaz na slevu, věrnostní karta
- Průkaz pro vstup do budovy
- Karta pro digitální podpis, dešifrování (soukromý klíč, certifikát)
- NFC – Secure Element v mobilním telefonu nebo SW emulace
- „Čipování“ – označení zvířete, člověka(?) – nemá formát karty, technologie podobná

Napájecí a komunikační rozhraní čipové karty může být zejména:

- **Kontaktní** (např. ISO 7816-2, USB,...)
- **Bezkontaktní** (např. ISO 14443)
 - ⇒ typ RFID (Rádiofrekvenční identifikace)
 - ⇒ základ NFC (Near-field communication)
- Kombinace kontaktního a bezkontaktního
 - ▶ dva čipy → **hybridní**
 - ▶ jeden čip, dvě rozhraní → **duální** (dual interface)

Napájení a přenos dat je hlediska fyzikálního principu:

- Elektrický – pro kontaktní rozhraní
- Elektromagnetický – pro bezkontaktní rozhraní – rádiové signály, indukční vazba
- Optický – potisk, záznam – čárové kódy, MRZ (Machine Readable Zone)

Bezkontaktní rozhraní

Mnoho různých proprietárních i standardních druhů bezkontaktních rozhraní

- LF – nízkofrekvenční – 125 kHz, 135 kHz (EM 4102, TI tagIT)
- HF – vf s vazbou na blízko (proximity) – 13.56 MHz (ISO 14443)
- HF – vf s vazbou na dálku (vicinity) – 13.56 MHz (ISO 15693)
- UHF – 868–928 MHz...

Z hlediska procesorových karet s kryptografickými funkcemi nás zajímá zejména ISO 14443.

- Studentský, zaměstnanecký průkaz ČVUT – MIFARE 1K (Classic) – proprietární protokol nad ISO 14443 (ne podle ISO 7816-4)
- Opencard, In-karta ČD – MIFARE DESFire, DESFire EV1
- Biometrický pas – ICAO (International Civil Aviation Organization)
- **Platební karty – Visa payWave, MC PayPass...**

- Frekvence 13.56 MHz, indukční vazba, dva typy modulace (A, B)
- Častější typ A
- Typ A, komunikace směrem z terminálu do karty:



- Typ A, komunikace směrem z karty do terminálu:



- Nic (skoro) – jen propojené kontakty, rezonanční obvod apod. — to ale není *smart* ani *čipová*
- Paměť
 - ▶ ROM s unikátním sériovým číslem
 - ▶ EEPROM
 - ▶ zabezpečená paměť (např. Philips/NXP MIFARE Classic)
 - ★ Paměť s funkcí autentizace a šifrování, řízení přístupu k datům
- ani ta ještě není *smart* v užším smyslu

● Počítač

- ▶ Konfigurovatelný pomocí API
- ▶ Programovaný při výrobě
- ▶ Programovatelný v terénu (např. Java Cards)
 - ★ Standard GlobalPlatform (dříve OpenPlatform) – správa aplikací

● Vnitřní struktura

- ▶ Výpočetní jednotky – CPU, kryptografický koprocessor
- ▶ Paměti – RAM, ROM, EEPROM
- ▶ Komunikační rozhraní, obvody napájení, senzory

Systém s čipovými kartami – klient/server

- Čipová karta, token...
 - ▶ data (+ aplikace) na kartě, klíče, ...
- Čtečka – terminál
 - ▶ Připojení čtečky k řídicímu počítači
 - ★ např. sériová linka, USB CCID class, ...
 - ▶ API operačního systému
 - ★ např. Windows PC/SC, Linux PCSC-Lite, ...
- Middleware – knihovna → standardní crypto token API
 - ▶ např. PKCS#11, Windows – Crypto API CSP
- Uživatelská aplikace (mail klient, webový prohlížeč, ...)
- Server, ...

- Obecné
 - ▶ ISO/IEC 7816-4, -7, -8, -9...
 - ▶ GlobalPlatform – správa aplikací
- Platební systémy
 - ▶ EMV (Europay, MasterCard, Visa)
 - ▶ EN 1546, CEPS – elektronické peněženky
- Telekomunikace
 - ▶ GSM 11.11, 11.14 (SIM)
 - ▶ TS 31.102, 31.111, ... (UICC, USIM)
- Další – standardizované nebo proprietární

APDU = Application Protocol Data Unit

Command APDU						
Hlavička (povinná)				Tělo (volitelné)		
CLA	INS	P1	P2	Lc	Data	Le

Case 1:

Žádná data

Žádná odpověď

CLA	INS	P1	P2
-----	-----	----	----

Case 2:

Žádná data

Odpověď

CLA	INS	P1	P2	Le
-----	-----	----	----	----

Case 3:

Data

Žádná odpověď

CLA	INS	P1	P2	Lc	Data
-----	-----	----	----	----	------

Case 4:

Data

Odpověď

CLA	INS	P1	P2	Lc	Data	Le
-----	-----	----	----	----	------	----

ISO 7816 – komunikace (2)

Command APDU						
Hlavička (povinná)				Tělo (volitelné)		
CLA	INS	P1	P2	Lc	Data	Le

Response APDU		
Tělo (volitelné)		Zápatí (povinné)
Data	SW1	SW2

Příklad APDU – výběr aplikace

CLA	INS	P1	P2	LC	DATA
00	A4	04	00	06	41 42 43 44 45 46
00					standardní příkaz ISO 7816
	A4				SELECT – vyber (soub./aplik.)
		04			vyber podle jména (AID)
			00		(vrať nepovinnou FCI)
				06	délka datového pole s AID

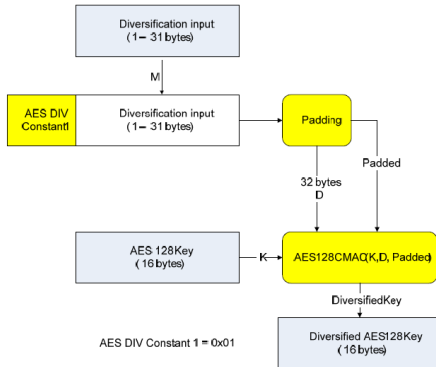
číslo aplikace (AID) 41 42 43 44 45 46

- Nic – doufáme, že nikdo neposlouchá, nenahrává, neopakuje. Spoléháme se, že nevzniknou klony, emulátory.
- **Symetrická** autentizace a šifrování – klíč je tajný, sdílený mezi kartou a terminálem. Možno použít odvozování (diverzifikaci) klíčů
 - ▶ Key derivation function – jednosměrná funkce (např. HMAC, MAC)
 - ▶ Princip: $k = \text{KDF}(\text{MasterKey}, \text{SerialNo})$
 - ▶ Různé karty budou mít různé klíče k
 - ▶ Jednosměrnost KDF → prolomení k neprozradí MasterKey
 - ▶ MasterKey stačí jen jeden (pro jednu sérii karet)
- **Asymetrická** schémata – veřejný a soukromý klíč, PKI, certifikáty, řetěz důvěry
 - ▶ Každá karta má soukromý klíč a odpovídající veřejný klíč
 - ▶ Může mít k VK certifikát → PKI

Diverzifikace klíčů

- Vstup: Hlavní klíč (Master Key), diverzifikační data (číslo karty, identifikace systému, identifikace aplikace...)
- Výstup: Odvozený klíč (Diversified Key)
- Jednosměrnost: Z odvozeného klíče je výpočetně neschůdné zjistit hlavní klíč.

Příklad:

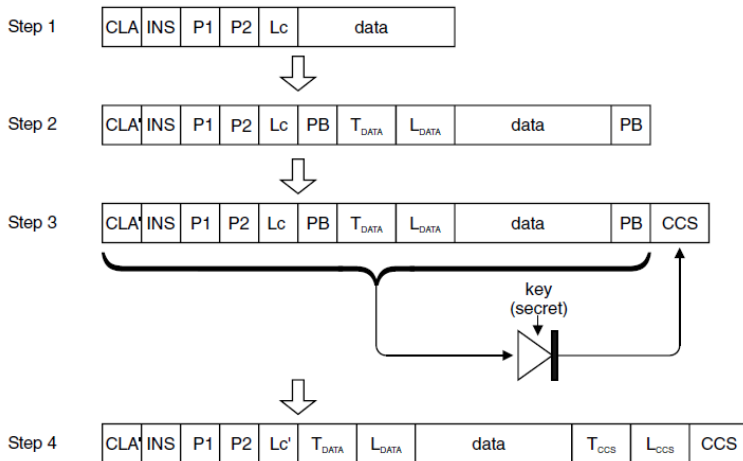


NXP: AN10922 – Symmetric key diversifications

- Bez autentizace, bez utajení
- Zabezpečení proti modifikaci zprávy (autentizace MAC)
- MAC + utajení zprávy (šifrování např. 3DES, AES)
- Princip „zabalení“ do struktur typu TLV – Tag, Length, Value

Zabezpečení přenosu dat – MAC

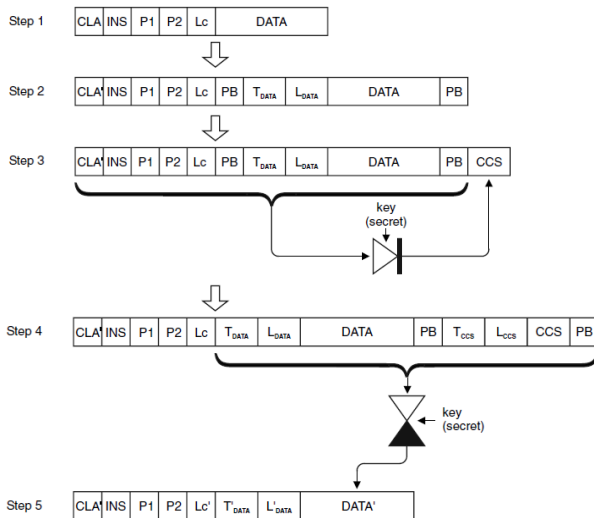
- Zabezpečení proti modifikaci zprávy (autentizace MAC)



W. Rankl, W. Effing: Smart Card Handbook

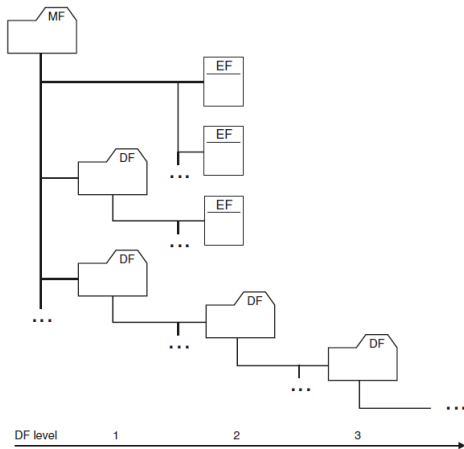
Zabezpečení přenosu dat – MAC + šifrování

● MAC + utajení zprávy (šifrování např. 3DES, AES)



Organizace dat na kartě

- Souborový systém podle ISO 7816-4



W. Rankl, W. Effing: Smart Card Handbook

- MF – Master File
- DF – Dedicated File (Directory File)
- EF – Elementary File

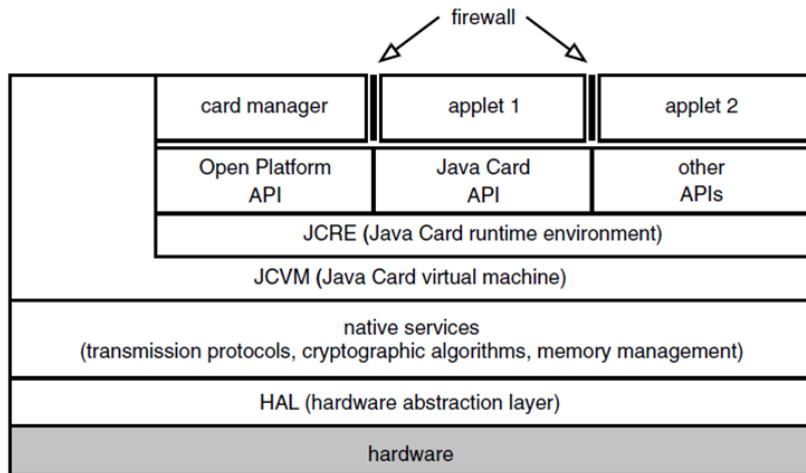
- Identifikátor souboru – FID (File Identifier)
 - ▶ 2 byte, může identifikovat MF, DF i EF
 - ▶ MF – 3F 00
 - ▶ Výběr souboru – SELECT (P1=0)
- Short file identifier (SFI)
 - ▶ 5 bitů, může identifikovat EF
 - ▶ Přímý výběr v příkazech pro manipulaci s daty
 - ▶ Čtení s implicitním výběrem – READ BINARY (P1=100sssss)
- Jméno adresáře – DF name
 - ▶ až 16 bytů, může identifikovat DF (nebo aplikaci)
 - ▶ může obsahovat AID – viz Java karty
 - ▶ Výběr souboru – SELECT (P1=4)

- Výběr – SELECT (00 A4 ...)
 - ▶ Výběr MF, DF nebo EF
- Transparentní soubory (bez vnitřní struktury)
 - ▶ Čtení – READ BINARY (00 B0 ...)
 - ▶ Zápis – UPDATE BINARY (00 D6 ...)
- Soubory organizované po záznamech
 - ▶ Pevná nebo variabilní délka záznamu
 - ▶ Cyklické soubory (s pevnou délkou záznamu)
 - ▶ Čtení záznamu – READ RECORD (00 B2 ...)
 - ▶ Zápis záznamu – UPDATE RECORD (00 DC ...)

- Karta obsahuje počítač + Java VM
- Podmnožina jazyka Java
- Zjednodušený VM, předzpracování (konverze)
- Java interpreter (většinou částečná HW podpora)
- Java Card Framework, runtime
- Persistence dat („heap je v EEPROM“)
- Podpora transakčního zpracování
- Na Java kartách: Aplikace → Applet

- Omezená množina typů:
- ano: byte (8), short (16)
- nepovinně: int (32)
- ne: long, float, double
- Nepodporováno: znaky, řetězce, vícerozměrná pole, dynamické nahrávání tříd, GC a finalizace, serializace, klonování objektů
- runtime `java.lang`, `javacard.framework`, `javacard.security`, `javacardx.crypto`, (`java.rmi`, ...)
- V novějších verzích i float, řetězce, servlety...

Běhové prostředí appletů



W. Rankl, W. Effing: Smart Card Handbook

Kryptografie a bezpečnost

8. Bezpečnost kryptografických systémů z hlediska teorií informace a složitosti

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Hodnocení odolnosti šifry vůči útokům
- Teorie informace
- Teorie složitosti algoritmů

Hodnocení odolnosti šifry vůči útokům (1)

- Určena vnitřní strukturou šifry (nelineární prvky, délka klíče, management klíče, ...).
- Ovlivněna použitou abecedou $\Rightarrow$ určité bigramy, trigramy, atd. jako např. „XZ“, „ZXW“ se nevyskytují, což zjednodušuje útok $\Rightarrow$ **teorie informace**.
- V praxi použitá řešení ovlivňují **teoretickou bezpečnost** (pseudonáhodné generátory čísel jsou predikovatelné, ...).
- Složitost řešení matematického problému, na kterém je šifra postavena $\Rightarrow$ hovoříme o **podmíněné bezpečnosti** a **nepodmíněné bezpečnosti** v rámci **teorie složitosti**.
- **Teorie informace** $\Rightarrow$ za jakých podmínek znalosti (jazykového) kryptografického prostředí lze kryptografický algoritmus rozluštit.
- **Teorie složitosti** $\Rightarrow$ s jakou složitostí bude možné kryptografický algoritmus rozluštit.

Hodnocení odolnosti šifry vůči útokům (2)

Teoretická bezpečnost

O teoretické bezpečnosti kryptosystému hovoříme tehdy, pokud při splnění konkrétních podmínek je šifra považována za bezpečnou, ovšem dané podmínky může být v praxi těžké splnit.

Problémy činí například:

- distribuce tajného klíče
- generování náhodných čísel
- implementace v hardwaru
- ...

Příklad: Sony pro generování podpisu u zařízení PS3 použilo algoritmu *ECDSA*. Soukromý klíč *SK* byl však napevno nastaven v zařízení, stačily tak dva různé digitální podpisy a klíč *SK* se dal snadno dopočítat.

Hodnocení odolnosti šifry vůči útokům (3)

Definice - Nepodmíněná bezpečnost

Kryptosystém je nepodmíněně bezpečný, pokud můžeme dokázat, že jej není možné prolomit **bez ohledu na útočníkem použité prostředky**.

- Pokud útočník nezíská opakovaným použitím šifry žádnou novou užitečnou informaci, hovoříme o **absolutní bezpečnosti**.
- Protože asymetrické šifry ve svém *VK* nesou informace pro zjištění *SK*, nemohou být nikdy **nepodmíněně bezpečné**

Příklad: *Vernamova šifra (one-time-pad)* provádí posuv každého znaku o náhodný počet míst v použité abecedě.

Hodnocení odolnosti šifry vůči útokům (4)

Definice - Podmíněná bezpečnost

Kryptosystém je podmíněně bezpečný, pokud jeho prolomení vyžaduje **prostředky, které nemáme** (výpočetní výkon, paměťový prostor, ...).

- **Podmíněná bezpečnost** má velmi striktní podmínky $\Rightarrow$ zavádíme podtřídy: **prokazatelnou** a **výpočetní bezpečnost**.

Příklad: AES je systémem podmíněně bezpečným, protože při útoku nemáme žádnou informaci o klíči a lze tak provádět pouze útok hrubou silou, což je zatím výpočetně neproveditelné v rozumném čase.

Poznámka - podmíněná vs. nepodmíněná bezpečnost

Podmíněná bezpečnost je slabší variantou nepodmíněné bezpečnosti, opírá totiž svou bezpečnost o prostředky oproti podstatě šifry.

Hodnocení odolnosti šifry vůči útokům (5)

Definice - Prokazatelná bezpečnost

Kryptosystém je prokazatelně bezpečný, pokud je znám matematický důkaz, který říká, že provést útok znamená vyřešit problém výpočetně ekvivalentní s problémem třídy NP.

Příklad: *RSA* je systémem prokazatelně bezpečným, protože je založen na problému faktorizace (spadá do třídy NP).

Poznámka - NP problém

Problém neřešitelný v polynomiálním čase deterministickým Turingovým strojem $\Rightarrow$ více v **teorii složitosti** dále v přednášce.

Hodnocení odolnosti šifry vůči útokům (6)

Definice - Výpočetní bezpečnost

Kryptosystém je výpočetně bezpečný, pokud pro jeho prolomení je třeba vynaložit prostředky nebo čas, které přesahují cenu respektive životnost informace.

Příklad: Prolomení DES na zařízení typu COPACOBANA (2006) je možné v rámci 7 dní s náklady na hardware kolem \$10,000. V takovém případě je šifra výpočetně bezpečná pro informace, které ztrácí význam během kratší doby nebo mají menší hodnotu.

Poznámka - COPACOBANA

Jedná se o zařízení založené na FPGA optimalizované pro běh kryptoanalytických algoritmů, vhodné pro paralelní výpočetní problémy.

Poznámka - Kerckhoffsův princip (1885)

Systém musí být prakticky, pokud ne matematicky, nedešifrovatelný a jeho utajení nesmí být podmínkou jeho bezpečnosti. Dále musí být přenositelný, schopný měnit klíče dle přání uživatele, snadný na používání a nesmí vyžadovat žádné speciální znalosti uživatele.

- Při posuzování bezpečnosti kryptosystému předpokládáme, že útočník zná celý kryptosystém.
- Založení bezpečnosti na *security through obscurity* je nepřijatelné, může sloužit pouze jako další bezpečnostní prvek.

Teorie informace (1)

- V 1948 a 1949 C. E. Shannon zveřejněním svých prací položil základy teorie informace a teorie šifrovacích systémů.
- Stanovil teoretickou míru bezpečnosti šifry pomocí neurčitosti OT.
- Mějme ŠT $\Rightarrow$ jestliže se nic nového o OT nedozvíme (například zúžení prostoru možných zpráv), i když přijmeme jakékoliv množství ŠT $\Rightarrow$ šifra dosahuje **absolutní bezpečnosti** (perfect secrecy), tj. ŠT nenese žádnou informaci o OT.
- Zvyšováním počtu znaků ŠT je u většiny praktických šifer (zvláště u historických šifer) poskytováno více a více **informace o OT**.
- Tato informace nemusí být bezprostředně viditelná.
- **Vzdálenost jednoznačnosti** = **# znaků OT**, pro který množství informace o OT obsažené v ŠT dosáhne takového bodu, že je možný jen jediný OT.

Příklad:

- Mějme jednoduchou substituci. Se znalostí jediného znaku “Z” ŠT nemáme ještě žádnou informaci o OT.
- Pravděpodobnost, že se pod ŠT skrývá písmeno A (resp. B, C, ..., Z) je stejná jako pravděpodobnost výskytu písmene A (B, ..., Z) v OT, tj. nedozvěděli jsme se o OT žádnou informaci.
- Zvýšením počtu písmen ŠT na 2, například “ZP”, máme určitou informaci o OT, tj. OT nemohou být všechny možné bigramy, protože vylučujeme bigramy se stejnými písmeny. Ty by dávaly ŠT typu “ZZ”. Při 50 písmenech už v ŠT můžeme lehce rozeznávat samohláskové a souhláskové pozice a můžeme je určovat.
- Při 1000 znacích ŠT je OT plně nebo až na detaily plně určen. Mezi číslem 1 a 1000 je tedy bod, kdy je OT už jen jeden.
- Z praxe víme, že při určování vzdálenosti jednoznačnosti bude také záležet na jazyku OT $\Rightarrow$ pojem **entropie** zdroje zpráv.

Teorie informace (3)

Definice – Entropie

Entropie je množství informace obsažené ve zprávě.

Teorie informace měří entropii zprávy průměrným počtem bitů nezbytných k jejímu zakódování při optimálním kódování (minimum bitů).

Entropie zprávy ze zdroje X je

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i,$$

$p_1, \dots, p_n$ jsou pravděpodobnosti všech zpráv $X_1, \dots, X_n$ zdroje X a $-p_i \log_2 p_i = \#$ bitů nutných k optimálnímu zakódování zprávy X_i .

V případě, že máme n zpráv se stejnou pravděpodobností $p = \frac{1}{n}$ (náhodné řetězce délky $\log_2 n$) $\Rightarrow$ entropie takového zdroje je $H(X) = -n(\frac{1}{n} \log_2(\frac{1}{n})) = \log_2 n$. Znamená to, že k binárnímu zakódování n zpráv se stejnou pravděpodobností nějakého zdroje potřebujeme přibližně tolik bitů, kolik obsahuje informací — entropie (podle definice).

Poznámka

Lze dokázat, že **maximální entropie** nabývá zdroj, který produkuje všechny zprávy se stejnou pravděpodobností. Má-li zdroj n zpráv se stejnou pravděpodobností $\frac{1}{n}$, pak dosahuje maximální entropie $\log_2 n$.

Doporučení

V případě velkých souborů dat lze získat orientační představu a horní odhad pro entropii těchto souborů použitím kvalitního komprimačního programu $\Rightarrow$ **entropie souboru $\leq$ # bitů komprimovaného souboru**.

Příklad: Mějme 2 možné zprávy: „panna“ nebo „orel“ a necht' mají stejnou pravděpodobnost $\Rightarrow$ entropie zprávy z tohoto zdroje je $H(X) = -0.5 \cdot (-1) - 0.5 \cdot (-1) = 1$, tedy jeden bit.

Příklad: Mějme zdroj, který vydává zprávy: „bílý“ s pravděpodobností $\frac{1}{4}$ a „černý“ s pravděpodobností $\frac{3}{4} \Rightarrow$ entropie zprávy z tohoto zdroje je $H(X) = 0.25 \cdot \log_2 4 + 0.75 \cdot \log_2 \frac{4}{3} = 0.25 \cdot 2 + 0.75 \cdot 0.41 = 0.81$, takže zprávy z tohoto zdroje obsahují v průměru necelý bit informace.

Příklad: Mějme zdroj vydávající pouze jednu zprávu s $p = 1 \Rightarrow$ entropie takového zdroje je $H(X) = -(1 \cdot 0) = 0$, (0 bitů). Daný zdroj nemá žádnou **neurčitost** a zprávy z něj nenesou žádnou informaci.

Neurčitost

Entropie zprávy vyjadřuje také míru její **neurčitosti**. Tato míra neurčitosti je počet bitů, které potřebujeme získat luštěním ŠT, s cílem určit OT.

Například v případě, že část ŠT „4lů9Fg“ vyjadřuje buď výsledek „panna“ nebo „orel“, pak míra neurčitosti této zprávy je rovna právě 1.

Definice – Obsažnost jazyka – Průměrná entropie

Pro daný jazyk uvažujme množinu X všech N -znakových zpráv. Obsažnost jazyka pro zprávy délky N znaků definujeme jako výraz

$$R_N = \frac{H(X)}{N},$$

tj. průměrnou entropii na 1 znak (průměrný # bitů informace v 1 znaku).

- Máme-li dlouhou zprávu, její další písmeno bývá v řadě případů určeno už jednoznačně nebo je možný jen malý počet variant.
- Například máme-li 13 znakovou zprávu „Zítřa odpoled“, je pravděpodobné, že pokračuje písmenem „n“. U 14. znaku nepřibude žádná entropie.
- V případě ostatních možných 13-znakových řetězců bude situace podobná, tj. umožní jen 1 nebo několik málo variant.
- Entropie příliš nevzroste $\Rightarrow R_{N+1} < R_N$.

Teorie informace (7)

- U přirozených jazyků výraz R_N pro zvyšující se N klesá.
- Z předchozího plyne: $\lim_{N \rightarrow \infty} R_N = r$
- Konstanta r je **obsažnost jazyka vzhledem k jednomu písmenu**.
- Pro hovorovou angličtinu je $r = 1.3$ až 1.5 bitů/znak.
- Obsažnost jazyka nám tedy říká, kolik bitů informace ve skutečnosti obsahuje průměrně 1 písmeno jazyka.

Definice – Absolutní obsažnost jazyka R

Mějme stejně pravděpodobné zprávy tvořené v jazyce s L stejně pravděpodobnými znaky $\Rightarrow R_N = \frac{\log_2 L^N}{N} = \log_2 L = R$. R nazýváme absolutní obsažnost jazyka. Absolutní obsažnosti dosahuje takový jazyk, který poskytuje generátor náhodných znaků. Je to maximální neurčitost, kterou přirozené jazyky nemohou dosáhnout, neboť jednotlivé znaky tvoří slova a věty, které mají odlišné pravděpodobnosti.

Teorie informace (8)

Definice – Nadbytečnost jazyka vzhledem k jednomu písmenu D

Nadbytečnost (redundance) jazyka vzhledem k jednomu písmenu nám vyjadřuje, kolik bitů je v jednom znaku daného jazyka nadbytečných, a je dána výrazem

$$D = R - r \quad \text{a číslo} \quad \frac{100D}{R}$$

pak udává, kolik bitů jazyka je nadbytečných procentuálně.

Příklad: Pro angličtinu máme $L = 26$, $R = \log_2 26 = 4.7$ bitů na písmeno, $r = 1.5$ bitů/písmeno, $D = R - r = 4.7 - 1.5 = 3.2$ nadbytečných bitů/písmeno a procentuálně to je

$$\frac{100D}{R} = \frac{100 \cdot 3.2}{4.7} = 68 \% \text{ nadbytečných bitů/písmeno.}$$

Příklad: Anglický text v kódu ASCII má v každém bytu (7b informací a 1b parity) 1.5b informace anglického znaku. 8b informace může nést také 8b informace $\Rightarrow D = R - r = 8 - 1.5 = 6.5$ b redundance.

Výpočet vzdálenosti jednoznačnosti (1)

Uved'me si pro ilustraci následující úvahu (neplatí pro všechny šifry).

- Mějme množinu zpráv M , množinu ŠT C a množinu $2^{H(K)}$ stejně pravděpodobných klíčů K .
- Předpokládejme, že máme ŠT c délky N znaků a že pro klíče $k \in K$ jsou odpovídající OT $D_k(c)$ vybírány z množiny všech zpráv M nezávisle a náhodně.
- V množině M je celkem 2^{RN} zpráv, z toho je 2^{rN} smysluplných zpráv a $U = 2^{RN} - 2^{rN}$ zpráv nesmyslných.
- Pokud provedeme dešifrování ŠT c všemi možnými $2^{H(K)}$ klíči, dostáváme $2^{H(K)}$ zpráv. Z nich je smysluplných průměrně pouze
$$S = 2^{H(K)} \frac{2^{rN}}{2^{RN}} = \frac{2^{H(K)}}{2^{DN}} = 2^{H(K)-DN}.$$
- Abychom dostali pouze jednu zprávu – tu, která byla skutečně zašifrována - musí být $S = 1$.

Výpočet vzdálenosti jednoznačnosti (2)

- Z předchozího plyne: $H(K) = DN$ a $N = \frac{H(K)}{D} = \delta_U$.

Definice – Vzdálenost jednoznačnosti

Vzdálenost jednoznačnosti je definována jako

$$\delta_U = \frac{H(K)}{D},$$

kde $H(K)$ je neurčitost klíče a D je redundance jazyka otevřené zprávy.

Příklad: Mějme obecnou jednoduchou substituci nad anglickou abecedou. Její vzdálenost jednoznačnosti je

$\delta_U = \frac{H(K)}{D} = \frac{\log_2(26!)}{3.2} = \frac{88.3}{3.2} = 27.6$. V ŠT o 28 znacích je tedy dostatečné množství informace na to, aby zbýval v průměru **jediný možný OT**. K rozluštění jednoduché substituce v angličtině postačí tedy v průměru 28 písmen ŠT.

Výpočet vzdálenosti jednoznačnosti (3)

Příklad: Mějme klíč Vigeněrový šifry o délce V náhodných znaků a uvažujme otevřený text z anglické abecedy. Vzdálenost jednoznačnosti

$$\text{je } \delta_U = \frac{H(K)}{D} = \frac{\log_2(26^V)}{3.2} = \frac{V \log_2(26)}{3.2} = \frac{4.7V}{3.2} = 1.5V.$$

To je na první pohled optimistický výsledek, ale ...

- Mějme ŠT v délce 1.5 násobku hesla, tj. oněch $1.5V$ znaků. 1. a 3. třetina textu používá stejné heslo, které lze eliminovat odečtením a poté s určitou pravděpodobností vyluštit 1. a 3. třetinu OT.
- Zůstává neznámá 2. třetina OT, kde je heslo zcela náhodné a neznámé, nemáme tedy z něj žádnou informaci. Zbývá redundance OT, z něhož známe 1. a 3. třetinu.
- δ_U je střední hodnota vzdálenosti jednoznačnosti a nezohledňuje právě takové rozložení informace z OT, které je k dispozici.
- Z hlediska množství informace, pokud máme dvě třetiny OT, zbytek bychom měli být schopni u anglického jazyka doplnit.

Výpočet vzdálenosti jednoznačnosti (4)

- Problém: na krátkých úsecích nedosahuje D hodnoty 3.2 bitu, ale méně, a navíc rozložení informace o OT je v daném případě atypické (známe 1. a 3. třetinu textu) $\Rightarrow$ nelze na něj vztahovat výsledky týkající se průměru a průměrných obvyklých textů.
- Na druhou stranu, pokud bychom měli k dispozici $2V$ znaků, budeme už schopni heslo eliminovat a luštit metodou knižní šifry. Kdybychom měli o pár znaků méně, byli bychom ještě schopni tyto znaky doplnit právě z důvodu nadbytečnosti zprávy.
- Skutečnou vzdálenost jednoznačnosti lze proto v závislosti na konkrétním problému a daném V očekávat v rozmezí $1.5V$ až $2V$.

Upozornění

Vzdálenost jednoznačnosti je odhad množství informace, nutného k vyluštění dané úlohy. Neříká však nic o složitosti takové úlohy.

Konfúze a difúze – podle Shannona základní techniky k potlačení redundance D ve zprávě. **Konfúze** – maří vztahy mezi ŠT a OT.

- Ztěžuje studium redundancí a statistických struktur OT.
- Nejjednodušší — substituce (Caesarova šifra).
- Proudové šifry — konfúze, pokud také zpětné vazby $\Rightarrow$ i difúze.
- Moderní substituční šifry jsou mnohem složitější, nahrazují se dlouhé bloky OT blokem ŠT a mechanismus substituce se mění s bity klíče nebo OT.

Difúze – rozprostírá redundanci OT.

- Vyhledávání rozprostřených redundancí ztěžuje kryptoanalýzu.
- Nejjednodušší — transpozice.
- V současnosti jsou tyto způsoby kombinovány k dokonalejšímu rozptýlu redundance.
- Obecně — difúzi lze snadno luštit, ale už ne například po dvojnásobné transpozici.

Teorie složitosti (1)

Teorie složitosti

- Metodologický základ pro analýzu **výpočetní složitosti** kryptografických technik a algoritmů.
- Porovnává kryptografické techniky a algoritmy a určuje jejich bezpečnost.
- Teorie informace → za jakých podmínek znalosti (jazykového) kryptografického prostředí lze kryptografický algoritmus rozluštit.
- Teorie složitosti → s jakou složitostí bude možné kryptografický algoritmus rozluštit.

Výpočetní složitost algoritmu je dána výpočetním výkonem potřebným pro jeho realizaci a zahrnuje:

- $T(n)$ – časovou náročnost.
- $S(n)$ – prostorovou náročnost (paměťové požadavky).

Proměnná n je rozsah vstupu.

Výpočetní složitost se vyjadřuje **řádem** O (Order) hodnoty výpočetní složitosti.

- Řád složitosti O roste nejrychleji v závislosti na n .
- Všechny prvky nižšího řádu se zanedbávají.
- Pro výpočetní náročnost algoritmu $7n^4 + 5n^2 + 6$ je $O(n^4)$.
- Takové vyjadřování složitosti algoritmu je systémově nezávislé.
- Umožňuje vyhodnotit vliv velikosti vstupu na časové a prostorové požadavky.
- Když $T(n) = O(n) \Rightarrow$ pro dvakrát větší vstup je dvakrát větší časová náročnost.
- V případě $T(n) = O(2^n)$ zvětšením vstupu o jedna se prodlouží doba výpočtu dvojnásobně.
- Časová složitost $T(n) = O(1)$ je nezávislá na n (na vstupech).

Teorie složitosti (3)

Klasifikace algoritmů podle časové nebo prostorové náročnosti

- Konstantní složitost, nezávislá na $n \Rightarrow O(1)$.
- Lineární složitost $\Rightarrow O(n)$.
- Polynomiální složitost $\Rightarrow O(n^m)$, kde m je konstantní. Například kvadratická složitost má $O(n^2)$, kubická složitost má $O(n^3)$, atd.
- Exponenciální složitost $\Rightarrow O(t^{f(n)})$, kde $t > 1$ je konstanta a $f(n)$ je nějaká polynomiální funkce (také lineární).
- Podmnožinou exponenciálních složitostí je superpolynomiální složitost, pro kterou je řád složitosti $O(t^{f(n)})$, kde t je konstanta a $f(n)$ je více než konstanta, ale méně než lineární funkce.

Poznámka

Známé luštitelské algoritmy jsou časově superpolynomiálně složité. Nedá se ale zatím dokázat, že nebude nikdy možné objevit nějaký časově polynomiální luštitelský algoritmus.

Teorie složitosti (4)

Tabulka ukazuje doby zpracování algoritmů s různou časovou složitostí

$T(n)$	# oper. ($n = 10^6$)	Čas výpočtu (1 oper. = $0.1 \mu\text{s}$)
$O(1)$	1	$0.1 \mu\text{s}$
$O(n)$	10^6	100ms
$O(n^2)$	10^{12}	1.2 D
$O(n^3)$	10^{18}	3200 Y
$O(2^n)$	10^{301030}	$10^{301005} \times$ věk vesmíru

- S růstem n může časová složitost výrazně ovlivnit praktickou použitelnost algoritmu.
- Při útoku hrubou silou je časová náročnost luštění úměrná množství klíčů, které je exponenciální funkcí délky klíče.
- Pro n -bitový klíč je časová složitost luštění hrubou silou $O(2^n)$.
- Nechť klíč je 56 bitový $\Rightarrow O(2^{56}) = 7.2 \cdot 10^{16}$ a čas výpočtu je ≈ 81000 dnů (1 oper. = $0.1 \mu\text{s}$). Uvažujme $1000 \times \mu\text{PC}$ s tímto výkonem $\Rightarrow$ nám stačí pro luštění čas kratší než 3 měsíce.

Teorie složitosti (5)

Složitost problémů

- Teorie složitosti klasifikuje kromě složitosti jednotlivých algoritmů použitých k řešení problému také jeho vnitřní složitost.
- **Turingův stroj** (TS) – konečný automat s nekonečnou čtecí – zapisovací páskovou pamětí.
- TS realisticky modeluje výpočetní operace.

Klasifikace problémů podle složitosti

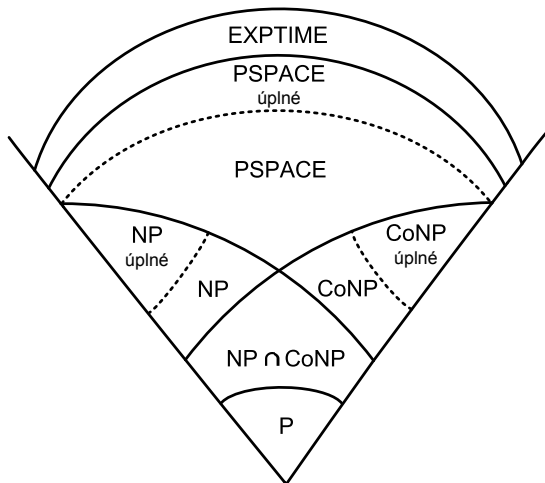
- **Snadno řešitelné problémy** – dají se řešit časově polynomiálními algoritmy za „přijatelně“ dlouhou dobu pro „rozumné“ velikosti n .
- **Obtížné problémy** – nelze řešit v polynomiálním čase za „přijatelně“ dlouhou dobu. Také jsou označovány jako **těžké problémy**. Problémy řešitelné pouze superpolynomiálními algoritmy jsou obtížně řešitelné i při malých hodnotách n .
- **Nerozhodnutelné problémy** – pro jejich řešení nelze navrhnout žádný algoritmus, i v případě, že neuvažujeme časovou složitost.

Dělení problémů podle tříd složitosti

- Třída **P** — Problémy mohou být řešeny v polynomiálním čase.
- Třída **NP** — Problémy mohou být řešeny v polynomiálním čase, ale pouze nedeterministickým TS, tj. může provádět pouze odhad buď správného řešení, nebo paralelně provede všechny odhady (výsledky prověřuje v polynomiálním čase).
- Třída **NP-úplný** — Problém, na který lze převést polynomiálním převodem každý problém ve třídě NP (NP těžký) a náleží do třídy NP.
- Třída **PSPACE** — Problémy mohou být řešeny v polynomiálním prostoru, ale nikoliv nezbytně v polynomiálním čase.
- Třída **EXPTIME** — Problémy, které jsou řešitelné v exponenciálním čase.
- Třída **CoNP** — Zahrnuje problémy, které jsou doplňkem některých problémů NP.

Teorie složitosti (7)

Třídy složitosti a jejich předpokládané vztahy



Vztah kryptologie a tříd složitosti

- Mnohé symetrické algoritmy a všechny algoritmy veřejného klíče se dají vyluštit v nedeterministickém polynomiálním čase.
- Při zadaném ŠT kryptoanalytik odhaduje OT a klíč a v polynomiálním čase nechá zpracovávat šifrovacím algoritmem tento odhad a prověřuje případnou shodu se ŠT.
- Tento postup je důležitý z teoretického hlediska, protože vymezuje horní mez složitosti kryptoanalýzy algoritmu.
- V praxi kryptoanalytik hledá deterministický časově polynomiální algoritmus.
- Pokud by se dokázalo, že $P = NP$ pak ???
- Hodně šifer lze triviálně luštit v nedeterministickém polynomiálním čase, pokud $P = NP$, pak tyto šifry budou luštitelné realizovatelnými deterministickými algoritmy.

Kryptografie a bezpečnost

9. Kryptografie eliptických křivek a kvantová kryptografie

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

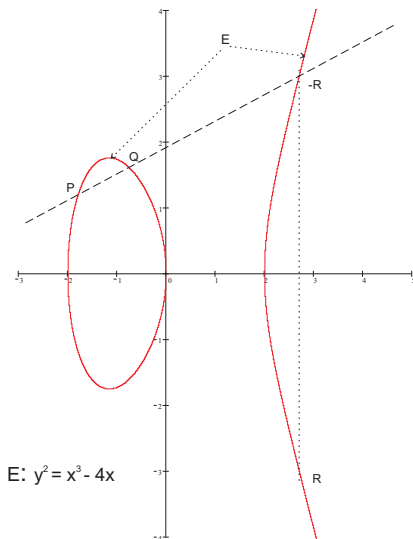
Katedra informační bezpečnosti

- Eliptická křivka nad tělesem $GF(p)$
- ECC a problém diskretního logaritmu
- Šifrování s ECC
- Vlastnosti informace
- Základní charakteristika kvantové kryptografie
- Protokol BB84

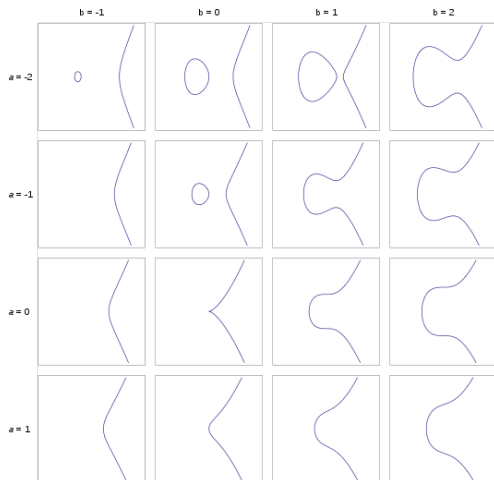
- Kryptografie eliptických křivek (ECC) je moderním a rozšířeným směrem současné kryptografie.
- ECC je další možností pro realizaci elektronického podpisu.
- ECC v některých ukazatelích dává lepší výsledky než předtím běžně používané kryptosystémy.
- V současnosti jsou eliptické kryptosystémy v řadě světových standardů a staly se alternativou k RSA.
- ECC má výhodu v rychlosti a menší náročnosti na hardware.
- Eliptické křivky jsou speciální podtřídou kubických křivek.
- Název eliptické vznikl proto, že kubické rovinné funkce se v minulosti používaly k výpočtu obvodu elipsy.
- Zkoumáním vlastností eliptických křivek se nejvíce zabýval německý matematik K. T. W. Weierstrass (1815 — 1897).
- V. Miller a N. Koblitz přišli nezávisle na sobě na možnost použití eliptických křivek v rámci kryptosystému veřejného klíče (1985).

Matematický základ (1)

- Eliptická křivka E je množina bodů v rovině, která vyhovuje rovnici
$$y^2 = x^3 + ax + b. \quad (1)$$
- Součtem 2 různých bodů P a Q z E bude opět bod ležící na E , a tedy také vyhovující rovnici (1).
- Geometrické interpretace součtu: Spojíme body $P = [x_P, y_P]$ a $Q = [x_Q, y_Q]$ přímkou, ta protne křivku E v bodě $-R$.
- Výsledkem sčítání je potom bod R , který je symetrický k $-R$ podle osy x . Body symetrické podle osy x nazýváme *opačné*.



Matematický základ (2)



$$y^2 = x^3 + ax + b$$

Matematický základ (3)

- Směrnice přímky, která spojuje dva různé body P a Q , je rovna

$$s = \frac{y_Q - y_P}{x_Q - x_P}. \quad (2)$$

- Pro souřadnice bodu $R = [x_R, y_R]$ ($R = P + Q$) platí

$$x_R = s^2 - x_P - x_Q \quad \text{a} \quad y_R = s(x_P - x_R) - y_P. \quad (3)$$

- Když $P = Q \Rightarrow$ jejich spojnice je tečna k E a její směrnice je rovna

$$s = \frac{3x_P^2 + a}{2y_P}. \quad (4)$$

- Sčítáním 2 opačných bodů ($P = -Q$) bychom měli dostat "0 bod".
- Taková přímka nám E už neprotne, resp. ji protne v ∞ . $\Rightarrow$ definitivně k E "bod v ∞ " O přidáme $\Rightarrow$ sčítání 2 opačných bodů definujeme: $P + (-P) = O$. Bod v ∞ je název "0 bodu" křivky E .
- Dodefinujeme sčítání pro O : $P + O = P$, $O + O = O$ a $O = -O$.
- Takto je definováno sčítání pro $\forall$ dvojice bodů na E včetně O .

Eliptická křivka nad tělesem $\text{GF}(p)(1)$

Využití eliptických křivek pro šifrování

Při využití eliptických křivek pro šifrování pracujeme v oblasti diskrétních hodnot (celých čísel, bitových řetězců, m -tice bitů) $\Rightarrow$

- Uvažujeme těleso $\text{GF}(2^m)$ a těleso $\text{GF}(p)$, kde p je prvočíslo.
- Obě tělesa jsou v praxi využívána – každé z nich má své přednosti.
- Pro jednoduchost výkladu dále jen operace nad tělesem $\text{GF}(p)$.
- Eliptická křivka nad tělesem $\text{GF}(p)$ je definována jako bod O v ∞ společně s množinou bodů $P = [x, y]$, kde x a y jsou z tělesa $\text{GF}(p)$ a vyhovují rovnici $y^2 = x^3 + ax + b$ v $\text{GF}(p)$, tj.

$$y^2 \equiv x^3 + ax + b \pmod{p} . \quad (5)$$

Eliptická křivka nad tělesem $\text{GF}(p)(2)$

- Koeficienty a a b jsou také prvky tělesa $\text{GF}(p)$ a musí splňovat podmínku
$$|4a^3 + 27b^2|_p \neq 0. \quad (6)$$
- Takto definovaná množina bodů tvoří grupu, koeficienty a a b volíme libovolně (veřejné parametry příslušného kryptosystému).
- V této grupě definujeme opačný bod k O jako $O = -O$ a pro ostatní nenulové body $P = [x_P, y_P] \in E$ definujeme $-P = [x_P, -y_P|_p]$, dále pro všechny body $P \in E$ definujeme $P + -P = O$ a $P + O = P$.
- Bod O nazýváme také nulovým bodem, vzhledem k jeho roli při sčítání v grupě E . Sčítání stejných nenulových bodů $P + P$ definujeme jako $R = P + P = [x_R, y_R]$, kde směrnice s je rovna

$$s = \left| \frac{3x_P^2 + a}{2y_P} \right|_p \quad (7)$$

- a souřadnice bodu R

$$x_R = \left| s^2 - x_P - x_Q \right|_p \quad \text{a} \quad y_R = \left| s(x_P - x_R) - y_P \right|_p. \quad (8)$$

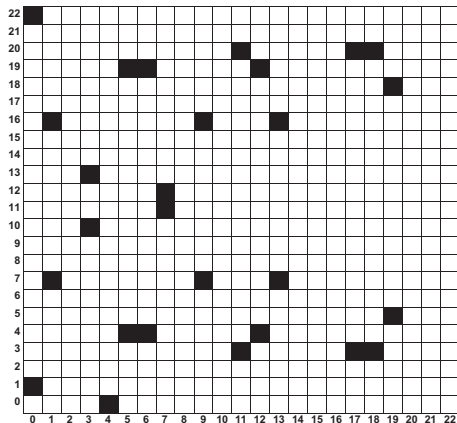
- Sčítáním různých nenulových a vzájemně neinverzních bodů $P = [x_P, y_P]$ a $Q = [x_Q, y_Q]$ křivky E definujeme jako $P + Q = R = [x_R, y_R]$, kde směrnice s je rovna

$$s = \left| \frac{y_Q - y_P}{x_Q - x_P} \right|_p \quad (9)$$

- a souřadnice bodu R

$$x_R = \left| s^2 - x_P - x_Q \right|_p \quad \text{a} \quad y_R = \left| s(x_P - x_R) - y_P \right|_p. \quad (10)$$

Eliptická křivka nad tělesem $GF(p)(4)$



(0,1)	(6,4)	(12,19)	(0,22)
(6,19)	(13,7)	(1,7)	(7,11)
(13,16)	(1,16)	(7,12)	(17,3)
(3,10)	(9,7)	(17,20)	(3,13)
(9,16)	(18,3)	(4,0)	(11,3)
(18,20)	(5,4)	(11,20)	(19,5)
(5,19)	(12,4)	(19,18)	O

28 bodů eliptické křivky $y^2 = x^3 + x + 1$ nad $GF(23)$

Eliptická křivka nad tělesem $\text{GF}(p)(5)$

Příklad:

$$E : y^2 \equiv x^3 + 2x + 2 \pmod{17}, \quad P = [5, 1], \quad 2P = ?$$

$$2P = P + P = [5, 1] + [5, 1]$$

$$s = \left| \frac{3x_P^2 + 2}{2y} \right|_{17} = |(2 \cdot 1)^{-1} \cdot (3 \cdot 5^2 + 2)|_{17} = |9 \cdot 9|_{17} = |81|_{17} = 13$$

$$x_{2P} = |s^2 - x_P - x_P|_{17} = |13^2 - 5 - 5|_{17} = |159|_{17} = 6$$

$$y_{2P} = |s(x_P - x_{2P}) - y_P|_{17} = |13(5 - 6) - 1|_{17} = |-14|_{17} = 3$$

$$2P = [6, 3]$$

Zkouška:

$$3^2 \equiv 6^3 + 2 \cdot 6 + 2 \pmod{17}$$

$$9 \equiv 12 + 12 + 2 \pmod{17}$$

$$9 = 9$$

ECC a problém diskretního logaritmu (1)

- Pro pochopení podstaty šifrování a podepisování v ECC je důležité využití tzv. problému diskretního logaritmu.
- Pro určitý bod P na křivce E postupně vypočítáme body $2P, 3P, 4P, 5P, 6P$ atd., čímž dostaneme obecně různé body xP na E .
- Protože křivka má konečný počet bodů, označíme ho $\#P$, po určitém kroku m se nám musí tato posloupnost opakovat.
- V bodě opakování mP tak platí $mP = nP$, kde nP je některý z předešlých bodů. Odtud dostáváme $mP - nP = O \Rightarrow$
- existuje nějaké $r = m - n, r < m$ takové, že $rP = O$, z toho plyne, že v posloupnosti $P, 2P, 3P, 4P, 5P, \dots$ se vždy dostaneme k bodu O , a poté cyklus začíná znovu od bodu P , protože $(r + 1)P = rP + P = O + P = P$.
- Nejmenší takové r , pro které je $rP = O$, nazýváme **řád bodu** P .

ECC a problém diskrétního logaritmu (2)

- Lze dále dokázat, že řád bodu dělí řád křivky, přičemž *řádem křivky* nazýváme počet bodů na křivce $\#E$.
- Body na křivce E mají různý řád. V kryptografii vybíráme takové body, jejichž řád je roven největšímu prvočíslu v rozkladu čísla $\#E$ nebo jeho násobku. *Kofaktor* je podíl: řád křivky / řád počátečního bodu a měl by být malý.
- U bodu řádu r máme zaručeno, že dojde k opakování v posloupnosti $P, 2P, 3P, \dots$ až po r -tém kroku.
- V případě, že r je velké číslo, např. 2^{256} , je to skutečně dlouhá posloupnost.
- Právě při šifrování a elektronickém podepisování se využívá tak velké posloupnosti a to právě v souvislosti s tzv. problémem diskrétního logaritmu.
- V případě, že si zvolíme jako náš privátní klíč číslo k a vypočteme $Q = kP$, potom body P a Q můžeme zveřejnit jako součást veřejného klíče.

ECC a problém diskretního logaritmu (3)

- Problém diskretního logaritmu je úloha, jak z bodů P a Q získat tajné číslo k tak, aby platilo $Q = kP$.
- Je zřejmé, že pro malý řád bodu P je úloha triviální. Pro velká r je to úloha, která se nedá řešit efektivně, tj. v polynomiálním čase. Z tohoto důvodu mohou být body P a Q zveřejněné.
- Dosud nejúčinnější metodou pro řešení takto definovaného problému diskretního logaritmu je tzv. Pollardova ρ metoda, jejíž složitost je řádově $(\pi r/2)^{1/2}$ kroků.
- Pokud máme $r = 2^{256}$, dostáváme $\approx 2^{128}$ kroků, což je zhruba na úrovni luštitelnosti symetrické blokové šifry se 128 bitovým klíčem.
- Pro nás je to z výpočetního hlediska neřešitelné, a tedy příslušná šifra je výpočetně bezpečná.

Šifrování s ECC

- Podstatu šifrování pomocí ECC si ukážeme na analogii Diffie-Hellmanova schématu výměny klíče.
- Strana i a j si chtějí vyměnit tajnou informaci přes veřejný kanál.
- Každá strana má důvěryhodnou cestou získaný veřejný klíč protistrany. V případě ECC ještě navíc předpokládáme, že oba sdílejí stejnou křivku E a její bod P .
- Označme po řadě d_i a Q_i privátní a veřejný klíč strany i , a obdobně d_j a Q_j pro stranu j , potom si obě strany mohou ustanovit společný klíč — bod Z na křivce E , aniž spolu komunikují.
- Strana i vypočte bod Z jako $d_i Q_j$ a strana j jako $d_j Q_i$. Tyto body jsou ve skutečnosti stejné, protože $Z = d_i Q_j = d_i (d_j P) = (d_i d_j) P$ a současně $Z = d_j Q_i = d_j (d_i P) = (d_j d_i) P$.
- Tedy každá strana vezme bod veřejný klíč — bod protistrany a sečte ho n -krát, kde n je privátní klíč. Protože obě strany vycházejí ze stejného bodu P , dospějí do stejného bodu Z .

Klasické a kvantové pojetí informace

- Klasická informace

- ▶ Lze libovolně kopírovat. Zejména je možné vytvořit zcela identickou kopii dané zprávy.

- Kvantová informace

- ▶ Nelze vytvořit identickou kopii neznámého kvantového stavu.
 - ★ Vychází z [Heisenbergova principu neurčitosti](#) ¹.
 - ★ Čtení zprávy zároveň ovlivňuje její obsah.

¹[Heisenbergův princip neurčitosti](#) je matematická vlastnost dvou kanonicky konjugovaných veličin. Nejznámějšími veličinami tohoto typu jsou *poloha* a *hybnost* elementární částice v kvantové fyzice.

$$\Delta x \Delta p \geq \frac{\hbar}{2},$$

kde $\hbar$ je tzv. redukovaná Planckova konstanta.

Čím přesněji určíme jednu z konjugovaných veličin, tím méně přesně můžeme určit tu druhou, a to bez ohledu na kvalitu přístrojů.

Tvrzení z klasické fyziky: můžeme předpovědět chování systému, pokud známe jeho počáteční stav.

Pro kvantovou fyziku neplatí: počáteční stav systému nikdy nemůžeme zjistit dostatečně přesně $\Leftarrow$ nelze dostatečně přesně zjistit obě tyto konjugované veličiny najednou.

- Klasická kryptografie

- ▶ Musí se vyrovnat s možností neomezeného kopírování nosičů klasické informace.
 - ★ Použití klíčů o extrémních délkách – princip one-time pad.
 - ★ Spoléhání na výpočetní složitost.

- Kvantová kryptografie

- ▶ Zakládá na nemožnosti tvorby identických kopií neznámého kvantového stavu.
 - ★ Nejprve se přenese klíč, který se při pozitivní detekci odposlechu zruší.

- Nepodmíněná bezpečnost

- ▶ V teoretické rovině lze dokázat bezpečnost systému bez ohledu na prostředky útočníka.
- ▶ V teoretické rovině lze dosáhnout i absolutní bezpečnosti.
- ▶ Předpokládá se, že bezpečnost těchto systémů nebude dotčena ani ve věku kvantových počítačů.

- Hlavní pozornost je zatím věnována přenosu zpráv.

- ▶ S uchováváním kvantově šifrované informace jsou spojeny jisté technologické potíže.

- Některé druhy schémat nejsou dostatečně propracovány.

- ▶ Kvantová schémata digitálního podpisu.

Kvantový bit - qubit

- Za fyzikální obraz qubitu považujeme libovolný kvantově mechanicky popsáný objekt, jehož stavy jsou prvky dvourozměrného Hilbertova prostoru.
 - ▶ Foton (polarizace, fázový posun)
 - ▶ Elektron (spin)
 - ▶ Atom (spin)
- Formálně:
 - ▶ Odpovídajícího naměřené hodnotě (kolaps kvantového systému), $|\psi\rangle = \omega_0|0\rangle + \omega_1|1\rangle$, kde:
 - ★ $\omega_0, \omega_1 \in \mathbb{C}$, $|0\rangle, |1\rangle$ jsou báze vektory H_2
 - ★ $|0\rangle, |1\rangle$ nazýváme vlastní stavy qubitu.
 - ▶ Měřením qubitu získáme hodnotu odpovídající právě 1 vlastnímu stavu.
 - ▶ Superpozici nelze „vidět“.
 - ★ Koeficienty ω_0, ω_1 určují rozdělení výsledků měření.
 - ★ Měřením superpozice zaniká a qubit přechází do vlastního stavu.
- Měřením jednoho qubitu získáme nejvýše 1 bit klasické informace.

Polarizační kódování

- Lineární (" + ")
 - ▶ $|0\rangle_{(r)} = "+"$
 - ▶ $|1\rangle_{(r)} = "-"$
 - ▶ $|\psi\rangle = \omega_{(r),0}|0\rangle_{(r)} + \omega_{(r),1}|1\rangle_{(r)}$
- Diagonální (" × ")
 - ▶ $|0\rangle_{(d)} = "\backslash"$
 - ▶ $|1\rangle_{(d)} = "\swarrow"$
 - ▶ $|\psi\rangle = \omega_{(d),0}|0\rangle_{(d)} + \omega_{(d),1}|1\rangle_{(d)}$

Využití v kryptografii

- *Heisenbergův princip neurčitosti*: nelze současně přesně určit stav daného qubitu vzhledem k lineární a diagonální bázi.
 - ▶ Při vhodně zvolené diagonální bázi lze pro ilustraci přímo napsat:
 - ★ $|0\rangle_{(r)} = \sqrt{\frac{1}{2}} (|0\rangle_{(d)} + |1\rangle_{(d)})$
 - ★ $|1\rangle_{(r)} = \sqrt{\frac{1}{2}} (|0\rangle_{(d)} - |1\rangle_{(d)})$
 - ▶ Interpretace: Čím určitější je stav vzhledem k lineární bázi, tím méně určitý je vzhledem k diagonální bázi a naopak.

Absolutně bezpečné kryptosystémy

- Existují perfektní kryptosystémy.
- Například **One-Time Pad (OTP - Vernamova šifra)**
- Zůstává ale problém bezpečné distribuce klíčů.

Řešení:

- Distribuce klíče (zřízení společného klíče) na základě kvantového jevu zaručujícího perfektní utajení.
- Protokol BB84 slouží k dohodě na symetrickém klíči využívající kvantových jevů..
 - ▶ Ten je následně použit pro systém one-time pad.
- Založen na využití Heisenbergova principu neurčitosti ve spojení s polarizačním kódováním.
- S mírnými obměnami je BB84 používán a rozvíjen dodnes.

Protokol BB84 - příklad průběhu komunikace

- 1 Odesílatel (Alice): Generuje náhodnou binární posloupnost a provádí její polarizační kódování dle náhodně volené báze.

1	1	1	1	1	0	0	1	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0
X	+	X	X	X	X	X	+	X	X	+	+	+	+	X	X	+	+	X	+	+	X	X	+	+
/	-	/	/	/	\	\	-	\	/					/	\			\	-		\	\		

- 2 Příjemce (Bob): Dekóduje přijaté fotony dle náhodně volené báze.

/	-	/	/	/	\	\	-	\	/					/	\			\	-		\	\		
+	+	X	+	X	X	+	X	+	X	+	+	X	X	+	X	X	+	X	+	X	+	+	+	X
0	1	1	1	1	0	0	0	1	1	0	0	0	1	0	0	0	0	0	1	1	1	1	0	1

- 3 Odesílatel (Alice): Oznámí Bobovi (veřejně, ovšem s autentizací původu zprávy), jakou bázi v daném kroku použila. To samé učiní Bob. Bity, kde se oba shodli, budou použity pro symetrický klíč.

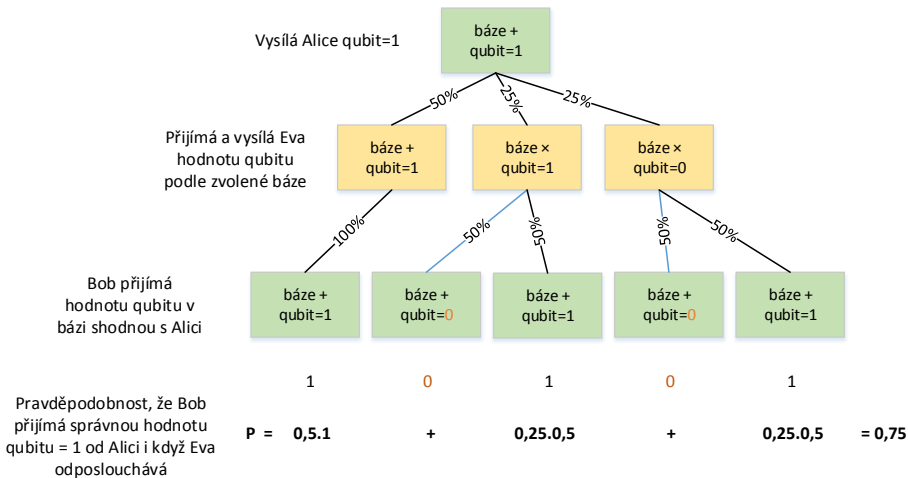
	✓	✓		✓	✓				✓	✓	✓				✓		✓	✓	✓				✓	
1	1		1	0					1	0	0				0		0	0	1				0	

- Dále je třeba provést **detekci odposlechu**.
 - ▶ Základní idea: Útočník (Eva) svým odposlechem ovlivní stav procházejících qubitů.
 - ▶ Alice a Bob obětují část dohodnutého klíče a veřejným kanálem (nutno autentizovat původ zpráv) si porovnají konkrétní přijaté hodnoty.
 - ▶ Odposlech se projeví jako chyba v přenosu.
 - ▶ Při obětování n bitů je pravděpodobnost detekce soustavného odposlechu $1 - (3/4)^n$.
 - ★ Volbou n lze tuto pravděpodobnost limitně přibližovat k hodnotě 1.
 - ▶ V současných systémech se ještě provádí zesílení soukromí (privacy amplification).
 - ★ Cílem je dále minimalizovat Evinu informaci o dohodnutém klíči, která (snad) byla získána nedetekovaným odposlechem.

Protokol BB84 - detekce odposlechu 2

Pozorováním nebo měřením kvantový systém změní svůj stav.

Příklad: **qubit**, $|\psi\rangle = \omega_0|0\rangle + \omega_1|1\rangle$, když je **qubit** pozorován, stav **qubit**u se dostane do kolapsu a to tak, že buď $\omega_0 = 0$ nebo $\omega_1 = 0$.



Dva druhy komunikace:

● Kvantová

- ▶ Nutný kvalitní nerušený komunikační kanál mezi Alicí a Bobem.
 - ★ Optický kabel bez běžných infrastrukturních prvků.
 - ★ Nutno znát model chyb.
 - ★ Synchronizace přenosu.

● Klasická

- ▶ Lze použít běžnou síťovou infrastrukturu.
- ▶ Není třeba zajišťovat důvěrnost přenášených zpráv.
- ▶ Musí být zajištěna autentizace původu kontrolních zpráv mezi Alicí a Bobem.
 - ★ Použití rádiového kanálu nemusí být dostatečné.

- Je schopen do jisté míry nahradit asymetrické systémy ve stávajících aplikacích.
 - ▶ Zamýšleno zejména s ohledem na teoretické hrozby přicházející z oblasti kvantových počítačů.
- V zásadě dva typy uživatelů:
 - ▶ Současní uživatelé asymetrických schémat
 - ★ Získají vyšší teoretickou bezpečnost.
 - ★ Poněkud ztrácejí pohodlí.
 - ★ Ne všechny komponenty lze zatím nahradit (podpis).
 - ▶ Současní uživatelé vojenských a zpravodajských systémů
 - ★ Získají větší pohodlí při zachování přibližně stejné úrovně bezpečnosti, jakou jim poskytují současné mechanismy založené na dlouhých náhodných klíčích.

Experimentální výsledky - dohoda na klíči na vzdálenost 23 km

Muller et al. 1995-96, Ribordy et al. 1998, 2000 (foto: Gisin et al. 2001)



FIG. 13. Geneva and Lake Geneva. The Swisscom optical fiber cable used for quantum cryptography experiments runs under the lake between the town of Nyon, about 23 km north of Geneva, and the centre of the city.

Kryptografie a bezpečnost

10. Generování klíčů, Rabin-Millerův test, náhodné generátory

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Čínská věta o zbytcích
- Kvadratická residua
- Generátory
- Rozklad složených čísel
- Testy prvočíslnosti
- Rabin-Millerův test prvočíslnosti
- Náhodná čísla v kryptografii
- Pseudonáhodné generátory
- Skutečně náhodné generátory
- Testování náhodnosti

Čínská věta o zbytcích (1)

Věta 30 – Čínská věta o zbytcích

Nechť $m_1, m_2, \dots, m_r$ jsou vzájemně nesoudělná kladná celá čísla. Potom systém kongruencí

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$x \equiv a_3 \pmod{m_3}$$

$$\vdots \quad \quad \quad \vdots$$

$$x \equiv a_r \pmod{m_r}$$

má jediné řešení modulo $M = m_1 m_2 \cdots m_r$.

Příklad: Mějme zbytky čísla x :

$$|x|_3 = 1, |x|_5 = 2, |x|_7 = 3 \Rightarrow M = 3 \cdot 5 \cdot 7 = 105.$$

Nechť $M_3 = 7 \cdot 5 = 35$, $M_5 = 3 \cdot 7 = 21$ a $M_7 = 3 \cdot 5 = 15 \Rightarrow$ pro x platí:

$$x = \left| |x|_3 M_3 y_3 + |x|_5 M_5 y_5 + |x|_7 M_7 y_7 \right|_M = |1 \cdot 35 \cdot y_3 + 2 \cdot 21 \cdot y_5 + 3 \cdot 15 \cdot y_7|_{105},$$

Kvadratická residua (1)

kde $35y_3 \equiv 1 \pmod{3}$, $21y_5 \equiv 1 \pmod{5}$ a $15y_7 \equiv 1 \pmod{7}$. Řeším těchto kongruencí dostávámé pro $y_3 = 2$, $y_5 = 1$ a $y_7 = 1$ a $\Rightarrow$ pro x platí: $x = |1 \cdot 35 \cdot 2 + 2 \cdot 21 \cdot 1 + 3 \cdot 15 \cdot 1|_{105} = |157|_{105} = 52$

Kvadratická residua

Definice – Kvadratické residuum

Pokud m je kladné celé číslo $\Rightarrow$ celé číslo a je **kvadratické residuum** modulo m , když $\gcd(a, m) = 1$ a kongruence $x^2 \equiv a \pmod{m}$ má nějaké řešení.

Když tato kongruence nemá žádné řešení $\Rightarrow a$ je **kvadratické nonresiduum** modulo m .

Věta 31 – Kvadratické residuum modulo prvočíslo

Necht' p je liché prvočíslo a a je celé číslo nedělitelné p . Potom kongruence $x^2 \equiv a \pmod{p}$ má buď přesně 2 vzájemně nekongruentní řešení modulo p nebo nemá žádné řešení.

Kvadratická residua (2)

Příklad: Pro $p = 7$ (prvočíslo) jsou kvadratická residua čísla 1, 2 a 4:

$$|1^2|_7 = |1|_7 = |1|_7$$

$$|2^2|_7 = |4|_7 = |4|_7$$

$$|3^2|_7 = |9|_7 = |2|_7$$

$$|4^2|_7 = |16|_7 = |2|_7$$

$$|5^2|_7 = |25|_7 = |4|_7$$

$$|6^2|_7 = |36|_7 = |1|_7$$

Každé kvadratické residuum se vyskytuje $2\times$. Přitom neexistuje žádná hodnota x , která by vyhovovala následujícím kongruencím:

$$|x^2|_7 = |3|_7$$

$$|x^2|_7 = |5|_7$$

$$|x^2|_7 = |6|_7$$

Kvadratická nonresidua

Kvadratická nonresidua jsou čísla, která nejsou kvadratickými residui. Čísla 3, 5 a 6 jsou kvadratickými nonresidui pro případ modula $p = 7$.

Kvadratická residua (3)

Věta 32 – Kvadratického residuum složeného modulu

Nechť $n = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_k^{\alpha_k}$, je kanonický rozklad n , kde $2 < p_1 < p_2 < \cdots < p_k$ jsou prvočísla a $\alpha_1, \dots, \alpha_k$ jsou přirozená čísla. Potom a je kvadratickým residuem modulo $n \Leftrightarrow a$ je kvadratické residuum modulo $p_1^{\alpha_1}, p_2^{\alpha_2}, \dots, p_k^{\alpha_k}$ (platí z čínské věty o zbytcích).

Příklad: $x^2 \equiv a \pmod{15}$, $15 = 5 \cdot 3$

$$|1^2|_{15} = |1|_{15} = |1|_{15} \rightarrow \text{1. kořen}$$

$$|1^2|_5 = |1|_5, |1^2|_3 = |1|_3$$

$$|2^2|_{15} = |4|_{15} = |4|_{15} \rightarrow \text{1. kořen}$$

$$|2^2|_5 = |4|_5, |2^2|_3 = |1|_3$$

$$|3^2|_{15} = |9|_{15} = |9|_{15} \rightarrow \gcd(9, 15) = 3$$

$$|4^2|_{15} = |16|_{15} = |1|_{15} \rightarrow \text{2. kořen}$$

$$|4^2|_5 = |1|_5, |4^2|_3 = |1|_3$$

$$|5^2|_{15} = |25|_{15} = |10|_{15} \rightarrow \gcd(10, 15) = 5$$

$$|6^2|_{15} = |36|_{15} = |6|_{15} \rightarrow \gcd(6, 15) = 3$$

$$|7^2|_{15} = |49|_{15} = |4|_{15} \rightarrow \text{2. kořen}$$

$$|7^2|_5 = |4|_5, |7^2|_3 = |1|_3$$

- 1, 4 jsou 4-násobná kvadratická residua.
- 9, 10, 6 nesplňují podmínku $\gcd(a, n) = 1$.
- 2, 3, 5, 7, 8, 11, 12, 13, 14 jsou kvadratická nonresidua.

Kvadratická residua (4)

- Pro liché prvočíslo p existuje $\frac{p-1}{2}$ kvadratických residuí modulo p a stejný počet kvadratických nonresiduí modulo p .
- Když je a kvadratickým residuem modulo prvočíslo $p \Rightarrow$ existují přesně 2 kořeny odmocniny výrazu $|x^2|_p$ a to:
 - ① číslo a v intervalu $(0, \frac{p-1}{2})$ a
 - ② číslo $| - a|_p$ v intervalu $(\frac{p-1}{2}, p-1)$.
- V případě n , které je součinem 2 lichých prvočísel p a q , bude počet kvadratických residuí mod n roven $\frac{(p-1)(q-1)}{4}$.
- V tomto případě vytváří 4 kvadratická residua tzv. úplnou odmocninu modulo n (perfect square mod n).
- Aby kvadratické residuum bylo "odmocninou" modulo n , musí být odmocninou také kvadratická residua modulo p a modulo q .
- Pro číslo $n = 5 \cdot 7 = 35$ je 6 kvadratických residuí 1, 4, 9, 11, 16, 29. Každé má přesně 4 kořeny odmocniny.

Kvadratická residua (5)

Definice – Legendreův symbol

Nechť p je liché prvočíslo, dále mějme celé číslo a a platí $p \nmid a \Rightarrow$ definujeme **Legendreův symbol** následovně:

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & \text{když } a \text{ je kvadratickým residuem.} \\ -1 & \text{když } a \text{ je kvadratickým nonresiduem.} \end{cases}$$

Příklad:

$$\left(\frac{1}{5}\right) = \left(\frac{4}{5}\right) = 1 \quad \text{a} \quad \left(\frac{2}{5}\right) = \left(\frac{3}{5}\right) = -1$$

Věta 33 – Eulerovo kritérium

Nechť p je liché prvočíslo, a je celé kladné číslo a platí $p \nmid a \Rightarrow$

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}.$$

Kvadratická residua (6)

Důkaz: Předpokládejme, že $\left(\frac{a}{p}\right) = 1 \Rightarrow$ kongruence $x^2 \equiv a \pmod{p}$ má řešení např. $x = x_0$. Použitím Malé Fermatovy věty dostáváme $a^{\frac{p-1}{2}} = (x_0^2)^{\frac{p-1}{2}} = x_0^{p-1} \equiv 1 \pmod{p}$. Odsud, když $\left(\frac{a}{p}\right) = 1 \Rightarrow$ víme, že $\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}$.

V případě, že $\left(\frac{a}{p}\right) = -1$ kongruence $x^2 \equiv a \pmod{p}$ nemá řešení $\Rightarrow$ pro celé číslo $i \in \langle 1, p-1 \rangle$ existuje jediné celé číslo $j \in \langle 1, p-1 \rangle$ tak, že $ij \equiv a \pmod{p}$ (dá se dokázat). Protože $x^2 \equiv a \pmod{p}$ nemá řešení $\Rightarrow$ můžeme seskupit čísla $1, 2, \dots, p-1$ do $\frac{p-1}{2}$ součinných párů rovnajících se a . Vynásobením těchto párů dostáváme $(p-1)! \equiv a^{\frac{p-1}{2}} \pmod{p}$. S použitím Wilsonovy věty: $(p-1)! \equiv -1 \pmod{p}$, $\Rightarrow$ dostáváme $-1 = \left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}$.

Kvadratická residua (7)

Zjednodušující předpisy pro výpočet Legendreovy funkce

❶ Když $a = 1 \Rightarrow \left(\frac{a}{p}\right) = 1$.

❷ Když a je sudé $\Rightarrow \left(\frac{a}{p}\right) = \left(\frac{\frac{a}{2}}{p}\right) \cdot (-1)^{\frac{p^2-1}{8}}$.

❸ Když $a > 1$ je liché $\Rightarrow \left(\frac{a}{p}\right) = \left(\frac{|p|_a}{a}\right) \cdot (-1)^{\frac{(a-1)(p-1)}{4}}$

- Pomocí těchto zjednodušujících předpisů lze efektivněji určit zda a je kvadratickým residuem modulo p , kde p je prvočíslo.
- Tyto předpisy se opírají o řadu vět z teorie čísel, které lze najít i s důkazy v [6].
- Zobecněním Legendreovy funkce pro složené moduly je [Jacobiho funkce](#), popis které lze také nalézt v [6].
- Vlastnosti kvadratických residuí se využívají v testech pro vyhledávání prvočísel.

Kvadratická residua (8)

Jacobiho symbol – funkce

- Je zobecněním Legendreovy funkce pro složené moduly.
- Definuje se pro celé číslo a a lichý celočíselný modul n .

Definice – Jacobiho symbol

Nechť n je liché celé číslo s kanonickým rozkladem $n = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdot \dots \cdot p_k^{\alpha_k}$, kde $p_1 < p_2 < \dots < p_k$ jsou prvočísla a $\alpha_1, \dots, \alpha_k$ jsou přirozená čísla. Dále necht' a je celé číslo nesoudělné s $n \Rightarrow$ pro **Jacobiho symbol** platí:

$$\left[\frac{a}{n} \right] = \left[\frac{a}{p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdot \dots \cdot p_k^{\alpha_k}} \right] = \left(\frac{a}{p_1} \right)^{\alpha_1} \left(\frac{a}{p_2} \right)^{\alpha_2} \dots \left(\frac{a}{p_k} \right)^{\alpha_k}.$$

Příklady: 1. $\left[\frac{2}{45} \right] = \left[\frac{2}{3^2 \cdot 5} \right] = \left(\frac{2}{3} \right)^2 \left(\frac{2}{5} \right) = (-1)^2(-1) = -1.$

2. $\left[\frac{2}{15} \right] = \left(\frac{2}{3} \right) \left(\frac{2}{5} \right) = (-1)(-1) = 1 \Rightarrow$ **Jaké x pro $x^2 \equiv 2 \pmod{15}$?**

Generátory (1)

Definice – Generátor

Pokud p je prvočíslo a celé číslo g je menší než p a bude-li dále pro každé číslo $b \in \langle 1, p-1 \rangle$ existovat nějaké číslo a takové, že platí $g^a \equiv b \pmod{p} \Rightarrow$ číslo g je **generátor** modulo p , tj. g je k p **primitivní**.

Příklad: Mějme $p = 7 \Rightarrow$ číslo 3 je generátorem modulo p :

$$|3^6|_7 = 1$$

$$|3^2|_7 = 2$$

$$|3^1|_7 = 3$$

$$|3^4|_7 = 4$$

$$|3^5|_7 = 5$$

$$|3^3|_7 = 6$$

Každé číslo od 1 do 6 se dá vyjádřit jako $|3^a|_7$. Pro $p = 7$ jsou generátory čísla 3 a 5. Čísla 2, 4 a 6 nejsou generátory.

Generátory (2)

Hledání generátorů je obecně obtížný problém. Generátory modulo prvočíslo p hledáme tak, že náhodně zvolíme číslo z intervalu $\langle 2, p - 1 \rangle$ a testujeme je. V případě znalosti kanonického rozkladu čísla $(p - 1)$ je testování jednodušší.

Věta 34 – Hledání generátorů

Nechť p je prvočíslo a $p - 1 = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdot \dots \cdot p_k^{\alpha_k}$, je kanonický rozklad $p - 1$, kde $p_1 < p_2 < \dots < p_k$ jsou prvočísla a $\alpha_1, \dots, \alpha_k$ jsou přirozená čísla. Celé číslo g je generátorem modulo p , pokud pro všechny hodnoty $p_1, p_2, \dots, p_k$ platí $|g^{p_i^{\frac{p-1}{\alpha_i}}}|_p \neq 1$.

Příklad: Mějme $p = 13$, potom $p - 1 = 12 = 3 \cdot 2^2$. Pro testování čísla 2 jako generátoru vypočítáme: $|2^{\frac{12}{4}}|_{13} = 8$ a $|2^{\frac{12}{3}}|_{13} = 3$. Žádný z výsledků se nerovná 1 $\Rightarrow$ **číslo 2 je generátorem modulo 13**. Pro číslo 3 dostáváme $|3^{\frac{12}{4}}|_{13} = 1$ a $\Rightarrow$ **3 nemůže být generátorem modulo 13**.

Rozklad složených čísel (1)

- Rozklad čísel na prvočinitele $\in$ nejstarší problémy teorie čísel.
- Rozklad není obtížný, je ale časově náročný.

Známé algoritmy pro rozklad čísel

- Síto číselného pole – Number Field Sieve (NFS), rychlý algoritmus pro rozklad 110 a víc místních čísel.
- Kvadratické síto – Quadratic Sieve (QS), rychlý pro čísla do 110 dekadických čísel.
- Eliptická metoda – Ecliptic Curve Method (ECM), do 43 místních čísel.
- Pollardův algoritmus Monte Carlo a Algoritmus řetězových zlomků (Continued Fraction Algorithm) jsou méně používanými algoritmy.
- Zkusmé dělení – Trial Division (TD), nejstarší algoritmus založený na testování každého prvočísla menšího nebo rovného odmocnině testovaného čísla.

Rozklad složených čísel (2)

Pokud n je součin 2 prvočísel $\Rightarrow$ výpočet kořenů odmocniny modulo n je z hlediska náročnosti výpočtu rovna faktorizaci n . Pokud známe prvočíselný rozklad $n \Rightarrow$ lze snadno spočítat kořeny odmocniny modulo n , jinak je výpočet obtížný, jako rozklad čísla na prvočinitele.

- $\pi(2^{512}) \approx 10^{151}$. Vesmír má $\approx 10^{77}$ atomů. Kdyby každý atom spotřeboval od počátku vzniku vesmíru až do dnes každou μs 1 miliardu prvočísel $\Rightarrow$ by to bylo dohromady 10^{109} prvočísel.
- Postup generování prvočísel nezačíná jejich náhodným generováním a následným rozkladem na prvočinitele.
- Správný postup je testování vygenerovaných čísel na prvočíselnost.
- Testy na prvočíselnost určí s danou pravděpodobností skutečnost, že vygenerované číslo je prvočíslo.

Testy prvočíslnosti (1)

Solovay-Strassenův test

Test čísla p na prvočíslnost:

- ① Vybereme náhodné liché $a < p$.
- ② Když $\gcd(a, p) \neq 1 \Rightarrow p$ není prvočíslo.
- ③ Vypočítáme $j = |a^{\frac{p-1}{2}}|_p$.
- ④ Když $j \neq \left[\frac{a}{p}\right] \Rightarrow p$ určitě není prvočíslo.
- ⑤ Když $j = \left[\frac{a}{p}\right] \Rightarrow$ pravděpodobnost, že p je složené, je $\leq 50\%$.
 - a , které dosvědčí, že p není prvočíslo, říkáme **svědek** – Witness.
 - Když p je složené $\Rightarrow$ pravděpodobnost vystupování náhodného čísla a jako svědka je $\geq 50\%$.
 - Opakováním testu t krát pokaždé s jinou hodnotou a docílíme, že pravděpodobnost toho, že složené p projde všemi testy jako prvočíslo je menší než 2^{-t} .

Testy prvočíslnosti (2)

Lehmannův test

Test čísla p na prvočíslnost:

- 1 Vybereme náhodné liché $a < p$.
 - 2 Vypočítáme $j = |a^{\frac{p-1}{2}}|_p$.
 - 3 Když $j \not\equiv \pm 1 \pmod{p} \Rightarrow p$ určitě není prvočíslo.
 - 4 Když $j \equiv \pm 1 \pmod{p} \Rightarrow$ pravděpodobnost, že p je složené, je $\leq 50\%$.
- Je jednodušší test na prvočíslnost.
 - Opět, když p je složené $\Rightarrow$ pravděpodobnost vystupování náhodného čísla a jako svědka je $\geq 50\%$.
 - Opakováním testu t krát vždy s jinou hodnotou a je pravděpodobnost toho, že složené p projde všemi testy jako prvočíslo, menší než 2^{-t} . Přitom se musí vyskytnout minimálně jednou hodnota -1 (krok 2 až 4).

Testy prvočíslnosti (3)

Rabin-Millerův test

Zvolíme náhodně p a spočítáme b a m tak, aby platilo: $p = 1 + 2^b m$, kde m musí být liché $\Rightarrow$

- ❶ Vybereme náhodné liché $a < p$.
- ❷ Necht' $j = 0$ a $z \leftarrow |a^m|_p$.
- ❸ Když $z = 1 \Rightarrow p$ může být prvočíslem, k další iteraci
- ❹ Dokud $z \neq p - 1 \wedge j \leq b - 2 \Rightarrow$ opakuj $z \leftarrow |z^2|_p, j \leftarrow j + 1$.
- ❺ Když $z \neq p - 1 \Rightarrow p$ určitě není prvočíslo
 - Zjednodušená verze testu na prvočíslnost doporučeného normou DSS.
 - Pravděpodobnost průchodu složeného čísla testem jako prvočíslo klesá rychleji než u předchozích testů
 - O $\frac{3}{4}$ hodnot a lze tvrdit, že mohou vystupovat v roli svědků.
 - Znamená to, že složené číslo neprojde t testy častěji než s pravděpodobností 4^{-t} .

Testy prvočíslnosti (4)

Rady pro generování prvočísel

- 1 Vygenerování náhodného čísla p s požadovanou délkou bitů n .
- 2 $\text{MSB} = \text{LSB} = 1$: $\text{MSB} = 1 \Rightarrow$ záruka požadované délky, $\text{LSB} = 1 \Rightarrow$ liché číslo.
- 3 Prověření, zda vygenerované prvočíslo není dělitelné malými prvočíslly (prvočísla < 1000). Testování lichého čísla p na prvočíslnost čísly 3, 5 a 7 vyloučí 54 % složených čísel, testování s prvočíslly < 256 vyloučí 80 % složených čísel.
- 4 Provedení Rabin-Millerova testu s náhodně generovanými čísly a . Volíme menší a . Test opakujeme minimálně 5-krát.
- 5 V případě, že p v některém testu nevyhoví, vygenerujeme jiné p .

Implementace takové metody trvá v závislosti na délce prvočísla řádově sekundy až desítky sekund.

Silná prvočísla

- Když n má být součinem 2 **silných prvočísel** p a $q \Rightarrow$
- Silná prvočísla mají mít vlastnosti ztěžující rozklad čísla n na prvočinitele $\Rightarrow$
- GCD čísel $(p - 1)$ a $(q - 1)$ má být malý.
- $(p - 1)$ a $(q - 1)$ mají mít velké prvočinitele p' a q' .
- $(p' - 1)$, $(q' - 1)$, $(p' + 1)$ a $(q' + 1)$ mají mít velké prvočinitele.
- $(p - 1)/2$ a $(q - 1)/2$ mají být prvočísla.

Použití silných prvočísel je předmětem diskusí:

- Délka prvočísel je důležitější než jejich struktura.
- Struktura může být na škodu nahodilosti.

Mnoho kryptografických aplikací vyžaduje **náhodná čísla**

- Generování kryptografických klíčů
- Generování čísel *nonce*, *salt*, výplní (*padding*)
- Vernamova šifra (*one-time pad*)

Vyžadovaná „kvalita“ náhodnosti se u různých aplikací liší

- *Nonce* v některých protokolech stačí jedinečné
- Generování klíčů vyžaduje vyšší kvalitu
- Záruka nerozluštitelnosti Vernamovy šifry platí jen v případě, že klíč byl získán ze skutečně náhodného zdroje s vysokou entropií

Základní princip:

„Kryptosystém je jen tak silný, jak silný je jeho nejslabší článek.“

Důsledek: Chybně navržený nebo chybně použitý generátor náhodných čísel může představovat fatální slabinu celého kryptosystému.

Náhodné číslo je číslo vygenerované procesem, který má nepředvídatelný výsledek a jehož průběh nelze přesně reprodukovat. Tomuto procesu říkáme *generátor náhodných čísel* (RNG – Random Number Generator).

U jednoho čísla nelze dost dobře diskutovat, zda je nebo není generováno generátorem náhodných čísel. Pracujeme tedy vždy s *posloupností náhodných čísel* neboli též *náhodnou posloupností*.

V počítači se čísla reprezentují pomocí bitů. Namísto náhodných čísel tedy často pracujeme s *náhodnými bity* resp. *generátory náhodných bitů* a *posloupnostmi (řetězci) náhodných bitů*.

Od náhodných posloupností očekáváme *dobré statistické vlastnosti*:

- Rovnoměrné rozdělení – všechny hodnoty jsou generovány se stejnou pravděpodobností
- Jednotlivé generované hodnoty jsou *nezávislé* – není mezi nimi žádná korelace

Důležitým pojmem je *entropie*.

Veličina *entropie* popisuje míru náhodnosti – jak obtížné je hodnotu (náhodné číslo, náhodnou posloupnost, řetězec náhodných bitů) uhodnout.

Entropie se vždy vztahuje k útočníkovi a jeho schopnosti (či neschopnosti) předpovědět vygenerovanou hodnotu. Pokud útočník následující generovanou hodnotu s jistotou zná, entropie je nulová (a nulová je i bezpečnost aplikace, která takto generovanou náhodnou hodnotu využívá).

Kdy je entropie generátoru náhodných bitů maximální?

Entropie generátoru je maximální, pokud se pro danou délku (počet bitů) generují všechny možné posloupnosti, každá z nich se stejnou pravděpodobností.

Počítače pracují deterministicky → jak generovat náhodná čísla?

Generátor pseudonáhodných čísel (PRNG): Algoritmus, jehož výstupem je posloupnost, která sice ve skutečnosti *není* náhodná, ale která se *zdá být* náhodná, pokud útočníkovi nejsou známy některé parametry generátoru.

- Algoritmické $\Rightarrow$ snadno realizovatelné
- Obvykle rychlé
- Zpravidla mají dobré statistické vlastnosti
- Ale: výstup je předvídatelný

Lineární kongruenční generátor (1)

Jeden z nejstarších a nejznámějších způsobů generování pseudonáhodných čísel je *lineární kongruenční generátor*:

$$X_{n+1} = (aX_n + c) \bmod m \quad (1)$$

kde X je posloupnost pseudonáhodných čísel, $m > 0$ je modul (často se používá mocnina dvou), a je násobitel, c je inkrement a X_0 je počáteční hodnota (*seed*).

Pseudonáhodná posloupnost X se opakuje nejvýše po m iteracích. Tohoto maxima dosáhneme, pokud jsou splněny všechny následující podmínky:

- Čísla c a m jsou nesoudělná
- $a - 1$ je dělitelné všemi prvočiniteli m
- Pokud $4|m$, pak také $4|a - 1$

Lineární kongruenční generátor (2)

Kvalita generátoru je velmi závislá na volbě parametrů m , a , c .
Nicméně z kryptologického hlediska jde o velmi problematický RNG:

- Pokud se generovaná čísla použijí jako souřadnice bodů v n -rozměrném prostoru, výsledné body budou ležet na nejvýše $m^{\frac{1}{n}}$ hyperrovinách.
- Když m je mocnina dvou, nízké bity X mají mnohem kratší periodu než celá posloupnost. Generátor může například produkovat střídavě sudá a lichá čísla.
(Pozn.: Proto se z hodnot X_n zpravidla vybírají jen některé bity.)

Lineární kongruenční generátor *není kryptograficky bezpečný*.

Kryptograficky bezpečné PRNG (1)

Požadavky na *kryptograficky bezpečné* pseudonáhodné generátory:

- „*Next-bit test*“: Je-li známo prvních k bitů náhodné posloupnosti, neexistuje žádný algoritmus s polynomiální složitostí, který by dokázal předpovědět $(k + 1)$. bit s pravděpodobností úspěchu vyšší než $1/2$.
- „*State compromise*“: I když je zjištěn vnitřní stav generátoru (ať už celý nebo zčásti), nelze zpětně zrekonstruovat dosavadní vygenerovanou náhodnou posloupnost. Navíc, pokud do generátoru za běhu vstupuje další entropie, nemělo by být možné ze znalosti vnitřního stavu předpovědět vnitřní stav v následujících iteracích.

Většina používaných PRNG tyto požadavky splňuje jen za určitých podmínek.

Kryptograficky bezpečné PRNG (2)

Příklady, jak realizovat kryptograficky bezpečné PRNG:

- **Bezpečná bloková šifra v režimu čítače:** Náhodně se zvolí klíč (*seed*) a počáteční hodnota čítače i . Zvoleným klíčem se postupně šifrují hodnoty $i, i + 1$, atd. Je zřejmé, že perioda u n -bitové blokové šifry je 2^n a nesmí dojít k vyzrazení klíče ani počáteční hodnoty čítače.
- **Kryptograficky bezpečná hešovací funkce aplikovaná na čítač:** Náhodně se zvolí počáteční hodnota čítače i . Postupně se hashuje $i, i + 1$, atd. Opět nesmí dojít k prozrazení počáteční hodnoty čítače.
- **Proudové šifry** jsou v zásadě PRNG, s jehož výstupem se XORuje plaintext.
- **Algoritmy založené na teorii čísel**, u kterých byl proveden (alespoň nějaký) důkaz bezpečnosti.

Blum-Blum-Shub PRNG (1)

Příkladem PRNG, u kterého se má za to, že je kryptograficky bezpečný, je algoritmus Blum-Blum-Shub:

$$X_{n+1} = X_n^2 \bmod m \quad (2)$$

Modul $m = pq$ je součinem dvou velkých prvočísel p a q . Počáteční prvek (*seed*) je $X_0 > 1$. Mělo by platit, že $p, q \equiv 3 \pmod{4}$, a $\gcd(\phi(p-1), \phi(q-1))$ by měl být malý.

Výstupem zpravidla není přímo hodnota X_n , ale její parita nebo několik nejméně významných bitů.

Vlastnosti:

- Pomalý
- Poměrně silný důkaz bezpečnosti (spojuje ji s výpočetní náročností faktorizace celých čísel)
- Lze přímo spočítat i -tý prvek posloupnosti:

$$X_i = (X_0^{2^i \bmod (p-1)(q-1)}) \bmod m \quad (3)$$

Kryptograficky bezpečné PRNG (3)

Zmíněné PRNG vyžadují náhodný a tajný vstup, *seed*:

- Bez nějaké *skutečné* náhody se stejně neobejdeme
- Kvalita PRNG se odvíjí i od kvality hodnoty *seed*

Entropie výstupu PRNG: dána entropií, která vstupuje (*seed*),
algoritmus samotný nikdy nemůže entropii zvyšovat.

Skutečně náhodné generátory (1)

Společná vlastnost kryptograficky bezpečných PRNG: Neobejdou se bez parametrů (zejm. *seed*), které je třeba zvolit náhodně. PRNG tedy samy o sobě v kryptografii nestačí, je třeba umět generovat skutečně náhodné hodnoty resp. bity.

Generátory skutečně náhodných čísel (TRNG – True Random Number Generator) využívají *zdroj entropie*, kterým je zpravidla nějaký fyzikální jev nebo vnější vliv. Například:

- Radioaktivní rozpad (projekt HotBits)
- Atmosférický šum (viz projekt random.org)
- Teplotní šum, např. na analogových součástkách
- Chování uživatele (pohyb myši, prodlevy při psaní na klávesnici)
- ...

Skutečně náhodné generátory (1)

Vlastnosti generátorů skutečně náhodných čísel:

- Výstup není předvídatelný, i když známe všechny parametry
- Výstup má zpravidla horší statistické vlastnosti → je nutné následné zpracování
- Implementace je složitější, často vyžaduje dodatečný dedikovaný hardware
- Zdroj entropie je třeba průběžně testovat, časem se mohou zhoršit jeho vlastnosti

Následné zpracování (*post-processing*) má za cíl vylepšit statistické vlastnosti TRNG, zejména:

- Odstranění nevyváženosti jedniček a nul (*bias*) a zajištění rovnoměrného rozdělení
- Extrakce entropie – zvýšení entropie výstupních bitů za cenu snížení rychlosti jejich generování (*bitrate*)

Skutečně náhodné generátory (2)

Post-processing

John von Neumannův dekorelátor je schopen eliminovat nevyváženost a snížit korelovanost výstupu. Bity se odebírají po dvou. Výstup je následující:

Vstup	Výstup
00, 11	– (vstup se zahodí)
01	0
10	1

Další možnosti, jak zlepšit statistické vlastnosti výstupu z TRNG:

- Výstup TRNG se XOR-uje s výstupem kryptograficky silného PRNG
- Sloučení (XOR) výstupů dvou nebo více různých TRNG („*software whitening*“)
- Hešování výstupu TRNG kryptograficky kvalitní hešovací funkcí

Testování náhodných generátorů (1)

K ověření vlastností náhodných generátorů se používají **statistické testy**. Testy ověřují, zda generovaná posloupnost splňuje některé vlastnosti náhodné posloupnosti.

Pozor: Statistickými testy lze ukázat, že daný generátor nejspíše **NENÍ** kvalitní, ale nelze prokázat, že JE kvalitní. Pokud generátor „projde“ všemi testy, je pořád možné, že obsahuje slabinu, kterou testy (vzhledem k tomu, jak jsou postaveny) neodhalily.

Statistické testy jsou založeny na **testování statistických hypotéz** na určité **hladině významnosti**. Nulovou hypotézou je, že testovaná posloupnost je náhodná. Testy vrací *p-hodnotu*, která vyjadřuje sílu důkazů proti nulové hypotéze. Pokud tato *p-hodnota* překročí určitou mez, považujeme nulovou hypotézu za neplatnou.

Testování náhodných generátorů (2)

Příklady testů náhodnosti:

- **Frekvenční test** – testuje, zda testovaná posloupnost bitů obsahuje přibližně stejný počet nul jako jedniček. Testuje se jednak celá posloupnost, jednak dílčí podposloupnosti.
- **„Runs“ test** – testuje, zda počet a délka řetězců po sobě jdoucích stejných bitů (samých 1 nebo samých 0) v rámci testované posloupnosti bitů odpovídá náhodné posloupnosti. Opět se testuje celá posloupnost i dílčí podposloupnosti.
- **Test hodnotí matic** – zaměřuje se na hodnoty disjunktních podmatic, cílem je odhalit lineární závislost podposloupností pevné délky.
- **Spektrální test** – diskrétní Fourierova transformace, snaží se odhalit periodicitu.
- **Maurerův univerzální statistický test** – testuje, zda lze posloupnost výrazněji bezztrátově zkomprimovat. Výrazně komprimovatelná sekvence neobsahuje dostatek entropie.

Testování náhodných generátorů (3)

Uvedené testy bývají součástí obecně známých a používaných sad („baterií“) testů.

Znamé sady testů:

- **Diehard** (George Marsaglia): 12 různých testů, poměrně silných
- **Dieharder** (Robert G. Brown): re-implementace testů Diehard podle jejich popisu, přidána řada dalších testů
- **NIST** (National Institute of Standards and Technology): 16 testů

Tyto sady byly nicméně vyvinuty převážně pro testování PRNG. Při testování TRNG je třeba **důkladně analyzovat zdroj entropie** a navrhnout a provést cílené testy, které by odhalily případné slabiny specifické pro tento zdroj entropie.

Kryptografie a bezpečnost

11. Infrastruktura veřejného klíče

prof. Ing. Róbert Lórencz, CSc.



**FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE**

Katedra informační bezpečnosti

- Distribuce veřejných klíčů
- Formáty certifikátů podle X.509
- Certifikační strom
- Distribuce tajných klíčů
- Digitální podpis

Distribuce veřejných klíčů (1)

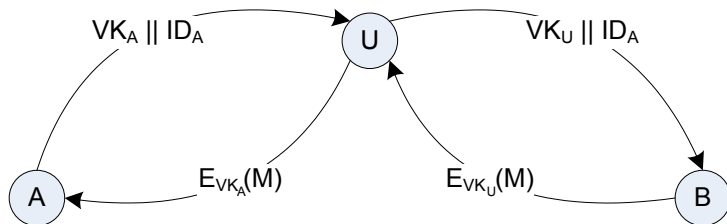
Úvod - distribuce tajných klíčů pomocí kryptografie VK

- 1 Distribuce veřejných klíčů.
- 2 Použití šifrování s VK pro distribuci tajných klíčů

Distribuce veřejných klíčů

- S distribucí VK souvisí hrozba **podvržení veřejného klíče** $\Rightarrow$
- ohrožení bezpečnosti kryptografických systémů s VK.

Podvržení veřejného klíče



Způsob podvržení veřejného klíče

- Subjekt A pošle svůj VK_A a svůj identifikátor ID_A , t.j. zprávu $VK_A || ID_A$, subjektu B.
 - ▶ Předpoklad: útočník U má **aktivní** přístup k veřejnému kanálu.
- U zachytí zprávu $VK_A || ID_A$, vytvoří novou zprávu $VK_U || ID_A$ a odešle ji B - **podvržení** VK_U subjektu B.
- B se domnívá, že VK_U , který dostal, patří A $\Rightarrow$ B zašifruje každou zprávu M klíčem VK_U , t.j. vytvoří $E_{VK_U}(M)$, odešle ji A.
- U zachytí $E_{VK_U}(M)$, dešifruje ji SK_U a získává zprávu M .
- U zná $VK_A \Rightarrow$ zašifruje $M \rightarrow E_{VK_A}(M)$ a pošle ji A.
- **Důsledek: U čte korespondenci zaslanou subjektem B subjektu A. U také může tyto zprávy měnit!**
- Tato činnost nemusí být subjektem B detekována.

Distribuce veřejných klíčů (3)

Distribuce veřejných klíčů lze realizovat technikou:

- 1 zveřejnění veřejných klíčů (public announcement)
- 2 veřejně dostupný adresář (public available directory)
- 3 autorita pro veřejné klíče (public-key authority)
- 4 certifikace veřejných klíčů (public-key certification)

Zveřejnění veřejných klíčů

- Zasíláním VK individuálně nebo hromadně v rámci nějaké komunity.
- Na internetu – připojením k emailu, vystavením na webu apod.
- Rychlý a jednoduchý způsob.
- Nízká bezpečnost! Není odolný proti podvržení VK →
- nepovolený subjekt může číst zprávy dotčeného subjektu až do odhalení.

Distribuce veřejných klíčů (4)

Veřejně dostupný adresář

- Vyšší stupeň bezpečnosti.
- Distribuci VK zabezpečuje důvěryhodná autorita – zodpovídá za obsah, je správcem adresáře.
- Účastníci registrují svůj VK prostřednictvím autorizovaného správce adresáře.
- Bezpečná registrace: osobně nebo přes bezpečnou komunikaci.
- Položky v adresáři jsou tvořeny: [jmeno; VK].
- Účastníci mění VK podle potřeby (bezpečnost, velký objem dat jedním VK, zkompromitovaný VK atd.)
- Správce periodicky aktualizuje adresář – elektronicky nebo fyzicky.
- Tento systém je bezpečnější než předchozí, má ale slabá místa:
- Pokud nepovolaný subjekt získá SK správce adresáře ⇒
- může modifikovat adresář a provádět odposlech jako v předchozí metodě.

Princip veřejně dostupného adresáře

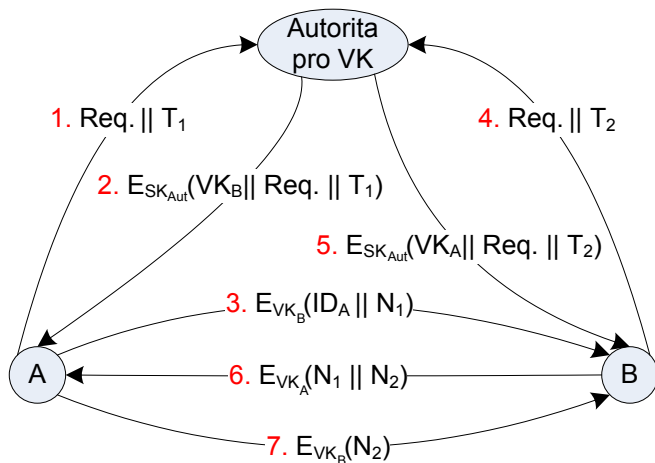


Autorita pro veřejné klíče

- Přísnější dohled na distribuci VK z adresáře znamená vyšší stupeň bezpečnosti.
- Autorita vykonává činnost správce adresáře VK.
- Každý účastník zná veřejný klíč autority VK_{Aut} , která vlastní odpovídající soukromý klíč SK_{Aut} .

Distribuce veřejných klíčů (6)

Distribuce veřejných klíčů pomocí autority pro veřejné klíče



Distribuce veřejných klíčů (7)

A chce inicializovat výměnu zprávy s B – scénář distribuce VK:

- ➊ A ve zprávě Autoritě požaduje (*Req.*- Request) VK_B za účelem komunikace s B. Žádost je doplněná časovou značkou T_1 (Time).
- ➋ Autorita odpoví zprávou zašifrovanou SK_{Aut} . A dešifruje zprávu VK_{Aut} – potvrzení, že zpráva je od Autority. Zpráva obsahuje:
 - ▶ VK_B – A může šifrovat zprávy pro B
 - ▶ kopii žádosti *Req.* od A – A může verifikovat kopii s vyslanou žádostí, tj. zda žádost nebyla modifikována před přijetím Autoritou
 - ▶ kopii T_1 – umožňuje A verifikovat aktualitu žádosti.
- ➌ A použije VK_B pro zašifrování zprávy pro B obsahující identifikátor A ID_A a náhodné číslo N_1 (potvrzuje jedinečnost výměny).
- ➍ B po obdržení zprávy od A vyžádá od Autority VK_A (jako A v 1.).
- ➎ B obdrží VK_A od Autority stejným způsobem jako A v bodě 2.
- ➏ B zašle A zprávu $N_1 || N_2$ zašifrovanou VK_A , N_2 generuje B.
- ➐ A zašle B zprávu N_2 zašifrovanou VK_B .

Distribuce veřejných klíčů (8)

Zvýšení bezpečnosti předaných zpráv

- Postup v bodech 6 a 7 zvyšuje bezpečnost výměny zpráv.
 - B zasláním zprávy $N_1 || N_2$ subjektu A, kde N_2 je náhodné číslo generované B, dá jistotu A, že autorem zprávy je B, protože obsahuje N_1 vygenerované A. Dešifrování na obou stranách je možné jen se znalostí příslušných SK A a B.
 - V kroku 7 získá B jistotu, že zpráva pochází od A, protože obsahuje N_2 .
-
- Distribuce VK pomocí Autority pro VK má své nevýhody. Autorita představuje “úzké hrdlo” této koncepce.
 - Každý účastník na získání VK adresáta musí nejdříve komunikovat s Autoritou pro VK.
 - Alternativní přístup v distribuci VK představuje použití **certifikátů**.

Distribuce veřejných klíčů (9)

Certifikace veřejných klíčů

- Distribuce VK bez kontaktu s třetím důvěryhodným subjektem.
- Tento přístup vyžaduje definici **certifikátu** a **certifikační autority**.

Definice certifikátu

Certifikát je struktura, která obsahuje:

- veřejný klíč žadatele/držitele certifikátu
- identifikační údaje držitele certifikátu
- dobu platnosti certifikátu
- další údaje vytvořené certifikační autoritou.

Tato struktura je podepsána soukromým klíčem certifikační autority SK_{Aut} a každý účastník může verifikovat obsah certifikátu pomocí veřejného klíče certifikační autority VK_{Aut} .

Distribuce veřejných klíčů (10)

Definice certifikační autority

Certifikační autorita (CA) je důvěryhodná třetí strana, která na základě žádosti vydává a aktualizuje certifikáty. Každý účastník může verifikovat to, že certifikát byl vytvořen CA, pomocí jejího veřejného klíče VK_{Aut} .

- Žádost o vydání certifikátu lze CA doručit osobně nebo elektronicky s využitím bezpečné komunikace.
- Přijímání žádostí, kontrola údajů v žádosti a odevzdávání certifikátů žadatelům se realizuje **registrační autoritou**

Princip výměny veřejných klíčů na bázi certifikátů

- Subjekt A před započítím jakékoliv komunikace požádá CA o vydání certifikátu na svůj veřejný klíč VK_A .

Distribuce veřejných klíčů (11)

- CA vydá certifikát C_A pro subjekt A, který obsahuje:
 - ▶ dobu platnosti - T_1
 - ▶ identifikační údaje A - ID_A
 - ▶ veřejný klíč - A - VK_A
- Tuto strukturu certifikátu podepíše svým soukromým klíčem SK_{Aut} a odešle A.
- Certifikát pro A má potom tvar:

$$C_A = E_{SK}(T_1, ID_A, VK_A)$$

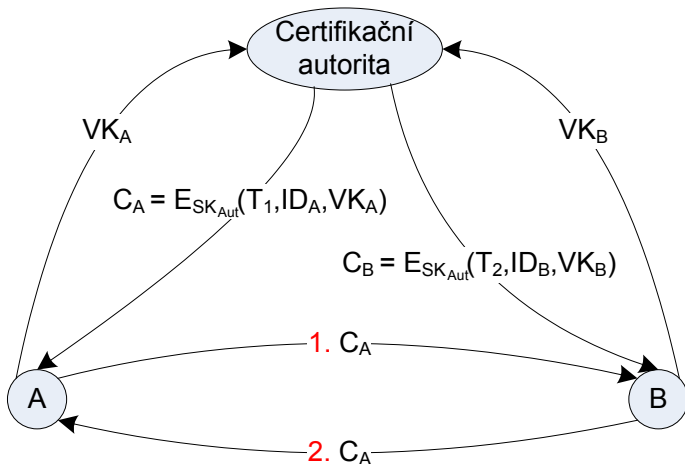
- Certifikát v této podobě lze poskytnout subjektu A, který ho může zveřejnit, protože ho lze číst/verifikovat pomocí veřejného klíče CA - VK_{Aut} :

$$D_{VK_{Aut}}(C_A) = D_{VK_{Aut}}(E_{SK_{Aut}}(T_1, ID_A, VK_A)) = (T_1, ID_A, VK_A)$$

- Tento proces dešifrování klíčem CA je současně ověřením toho, že certifikát byl vytvořen CA.

Distribuce veřejných klíčů (12)

Distribuce veřejných klíčů pomocí certifikátů



Distribuce veřejných klíčů (13)

- Dešifrováním se získají jméno a veřejný klíč držitele certifikátu a také časový údaj o platnosti.
- Analogicky požádá o vydání certifikátu subjekt B.
- Certifikát C_B má analogickou strukturu jako C_A .
- Distribuce veřejných klíčů potom představuje výměnu certifikátů C_A a C_B mezi subjekty A a B (kroky 1. a 2. na obrázku).

Certifikáty podle doporučení X.509

- Formáty certifikátů určuje doporučení ITU-T X.509, které je částí doporučení X.500.
- Doporučení X.509 definuje také autentizační protokoly používané v různých typech sítí a aplikacích síťové bezpečnosti.
- Všeobecný formát podle X.509 je následující:

X.509 - formáty certifikátů

Formát certifikátu	Formát 1	Formát 2	Formát 3	
Sériové číslo certifikátu				
Algorit. vytvoření podpisu certifikátu				
Identifikační údaje CA				
Doba platnosti certifikátu				
Identifikační údaje uživatele				
Veřejný klíč uživatele				
Jednoznačný identifikátor CA				
Jednoznačný identifikátor uživatele				
Rozšíření				
Digitální podpis CA				

Certifikační strom (1)

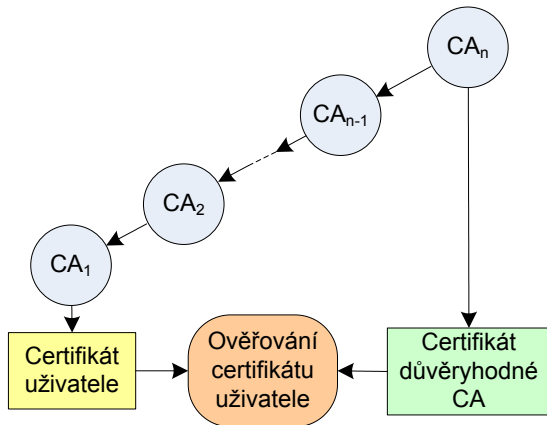
Vlastnosti certifikátů vydávaných CA:

- Kterýkoliv subjekt prostřednictvím veřejného klíče CA může verifikovat veřejné klíče jiných subjektů.
 - Žádný jiný subjekt než CA nemůže modifikovat vydané certifikáty
 - Certifikáty jsou nefalšovatelné $\Rightarrow$ jsou umístěny v adresáři CA přístupném všem subjektům bez nutnosti zvláštní ochrany.
- Uvedená koncepce má nevýhodu při velkém počtu uživatelů.
 - Každý uživatel musí mít kopii veřejného klíče CA.
 - Výhodnější je vytvořit více CA pro různé okruhy uživatelů.
- Vytvoření více CA předpokládá vzájemné propojení mezi CA například ve formě stromové struktury – **certifikačního stromu**.
 - Každý strom reprezentuje **kořenová CA**, která vlastní **kořenový certifikát**.

Certifikační strom (2)

- Posloupnost certifikátů od certifikátu uživatele až k certifikátu kořenové CA se nazývá **řetězec certifikátů**

Řetězec certifikátů

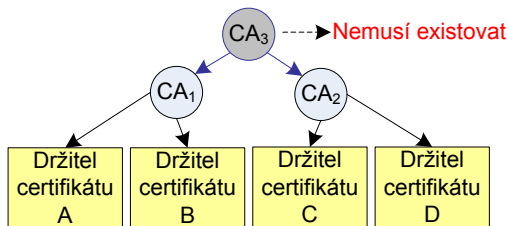


Certifikační strom (3)

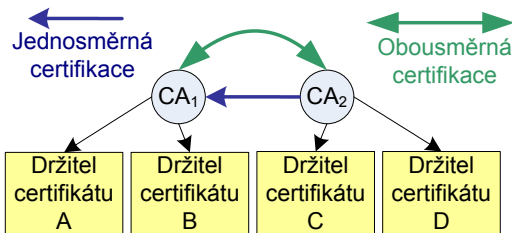
- Certifikát je platný $\Leftrightarrow$ platné všechny certifikáty v řetězci certifikátů.
- Kořenový certifikát musí být ověřen jinou bezpečnou cestou (např. křížová certifikace).
- Je možné prohlásit za důvěryhodný i certifikát v řetězci certifikátů $\Rightarrow$ se ověřování urychlí.
- Řetěz certifikátů předpokládá stromovou strukturu CA.
- Křížovou certifikací deklaruje CA_1 , protože důvěřuje všem certifikátům vystaveným CA_2 .
- Problém vzniká se získáním certifikátů uživatelů jiné stromové struktury.
- V tomto případě získání certifikátů mezi uživateli dvou různých CA umožňuje **křížová certifikace**:
 - ▶ jednosměrná
 - ▶ obousměrná

Certifikační strom (4)

Stromová struktura certifikačních autorit



Křížová certifikace



Certifikační strom (5)

V případě, že C (obrazek Křížová certifikace) chce komunikovat s A, musí A poslat C množinu certifikátů:

- certifikát A, podepsaný CA_1
- certifikát CA_1 podepsaný CA_1 , tj. kořenový certifikát CA_1
- certifikát CA_1 podepsaný CA_2 , tj. **křížový certifikát**

Certifikát CA_2 podepsaný CA_2 , tj. kořenový certifikát CA_2 , subjekt C zná, protože patří do stromové struktury CA_2 .

Touto křížovou certifikaci se CA_1 a její uživatelé stanou důvěryhodnými pro CA_2 a její uživatele, neplatí to naopak!

Aby tento vztah byl obousměrný, je nutné, aby i CA_2 si vyžádala křížový certifikát podepsaný CA_1 .

Obousměrná křížová certifikace znamená, že CA_1 a CA_2 vlastní kromě kořenových certifikátů také křížové.

Certifikační strom (6)

Platnost certifikátu

- Každý certifikát má vymezenou dobu platnosti. Nový certifikát je vydaný těsně před uplynutím této doby platnosti.
- Na žádost držitele certifikátu může certifikát ztratit platnost – žádostí odvolání certifikátu podanou na CA. Motivace:
 - ▶ kompromitace soukromého klíče
 - ▶ uživatel chce změnit aktuální CA
 - ▶ kompromitace certifikátu vydaného CA
- CA zveřejňuje seznam odvolaných certifikátů.

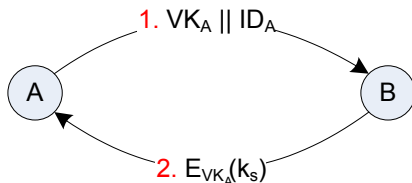
PKI (Public Key Infrastructure)

- Norma v síti Internet vycházející z norem ITU-T X.500
- Specifikace technických a organizačních opatření pro vydávání, správu, používání a odvolávání klíčů a certifikátů.
- Hlavní cíl – zabezpečení kompatibility SW pro Internet.

Distribuce tajných klíčů (1)

- Distribuce tajných klíčů existuje také v kryptografických systémech s VK.
- Kryptografické systémy VK poskytují lepší možnosti pro tuto distribuci než klasická (symetrická) kryptografie.

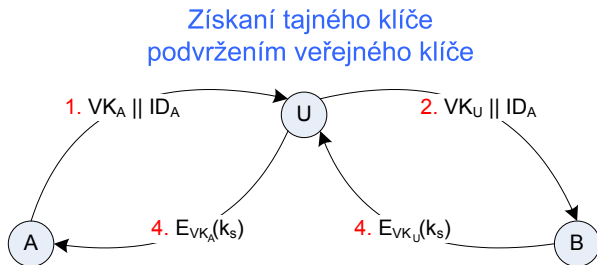
Jednoduchá distribuce tajných klíčů



- Pokud A chce komunikovat s B, musí oba realizovat uvedené kroky.
- po vyslání subjektem A zprávu B obsahující VK_A a identifikátor ID_A obdrží A od B tajný klíč k_s zašifrovaný VK_A .

Distribuce tajných klíčů (2)

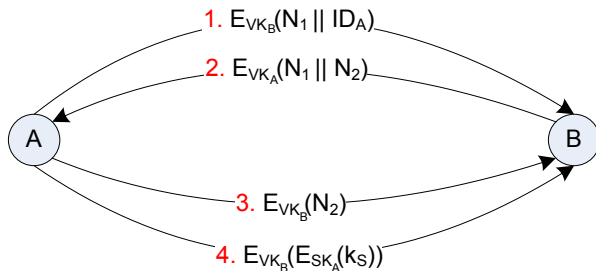
- Podvržením veřejného klíče aktivním útočníkem U lze získat tajný klíč k_s .
- Podvržení je provedeno stejným způsobem jako u podvržení veřejného klíče.



- Výsledek: tajný klíč k_s určený pro tajnou komunikaci mezi A a B zná také U, který může dešifrovat tajnou komunikaci mezi A a B.
- Jednoduchou koncepci distribuce tajných klíčů lze použít jen v prostředí umožňujícím pouze pasivní útoky.

Distribuce tajných klíčů (3)

- Distribuce tajných klíčů v prostředí s aktivními a pasivními útoky lze provést na bázi VK jen za předpokladu:
- **A a B si bezpečným způsobem vyměnili svoje veřejné klíče.**
Výměna tajných klíčů s utajením a autentizací



- Tato koncepce zaručuje důvěrnost a autentizaci při výměně tajného klíče.